POLITEKNIK STATISTIKA STIS
Buku Pedoman Paket R

BUKU PEDOMAN

Menyusun Modul Praktikum dengan modulstis

Pedoman Langkah demi Langkah untuk Dosen

Achmad Syahrul Choir

Untuk paket `modulstis` versi 0.3.11 · 9 November 2026

Menyusun Modul Praktikum dengan modulstis
Pedoman Langkah demi Langkah untuk Dosen

Penyusun: Achmad Syahrul Choir
Politeknik Statistika STIS, Jakarta

Untuk paket `modulstis` versi 0.3.11. Diperbarui 9 November 2026.

Seluruh kode dalam buku ini dijalankan ketika buku dibangun, sehingga keluaran yang tercetak adalah keluaran paket yang sebenarnya. Gambar antarmuka RStudio adalah skema tata letak, bukan tangkapan layar.

Kode sumber paket dan buku ini: https://github.com/madsyair/modulstis

Paket dan buku ini dilisensikan dengan lisensi MIT.

Daftar Isi

Kata Pengantar 1

Cara Membaca Buku Ini 2
- Dua jalur pembacaan . . . 2
- Kotak dan penanda . . . 2
- Kesepakatan penulisan . . . 3
- Prasyarat . . . 3

I. Memulai 4

1. Apa yang Dikerjakan modulstis untuk Anda? 5
- 1.1. Masalah menyusun empat belas modul . . . 5
- 1.2. Satu sumber, tiga keluaran . . . 5
- 1.3. Apa yang tidak dikerjakan paket ini . . . 7
- 1.4. Peta pedoman ini . . . 7
- 1.5. Ringkasan . . . 8
- 1.6. Uji pemahaman . . . 8

2. Menyiapkan Komputer Langkah demi Langkah 9
- 2.1. Daftar periksa kesiapan . . . 9
- 2.2. Memasang R dan RStudio . . . 10
- 2.3. Memasang TinyTeX dan pola pemenggalan Indonesia . . . 11
- 2.4. Memasang font Carlito . . . 12
- 2.5. Memasang paket R pendukung . . . 13
- 2.6. Memasang Quarto (opsional) . . . 13
- 2.7. Memasang modulstis dari GitHub . . . 13
- 2.8. Memeriksa kesiapan dengan satu perintah . . . 14
- 2.9. Galat pemasangan yang paling sering . . . 15
- 2.10. Ringkasan . . . 16
- 2.11. Uji pemahaman . . . 16

3. Modul Pertama dalam Lima Belas Menit 17
- 3.1. Membuat proyek . . . 17
- 3.2. Mengenali isi folder proyek . . . 18
- 3.3. Membaca spesifikasi contoh . . . 18
- 3.4. Memeriksa spesifikasi . . . 19
- 3.5. Merender . . . 20
- 3.6. Melihat hasil . . . 20
- 3.7. Mengubah satu baris, lalu merender ulang . . . 21
- 3.8. Ringkasan . . . 22
- 3.9. Uji pemahaman . . . 23

II. Menyusun Isi **24**

4. YAML Secukupnya untuk Menulis Modul **25**
4.1. Kunci, nilai, dan indentasi . . . 25
4.2. Daftar . . . 26
4.3. Teks panjang: perbedaan > dan | . . . 27
4.4. Kapan teks harus diberi tanda kutip . . . 27
4.5. Komentar . . . 28
4.6. Lima galat YAML yang paling sering . . . 28
4.7. Ringkasan . . . 30
4.8. Uji pemahaman . . . 30

5. Mengisi Identitas Modul dan Mata Kuliah **32**
5.1. Berkas baru untuk Modul 7 . . . 32
5.2. Kunci tingkat atas dan letaknya di PDF . . . 33
5.3. Blok `course` . . . 33
5.4. `short_code` dan nama berkas . . . 34
5.5. Memeriksa dan merender identitas . . . 35
5.6. Satu blok `course` untuk semua modul . . . 36
5.7. Ringkasan . . . 37
5.8. Uji pemahaman . . . 37

6. Sebelas Bagian Isi Modul **38**
6.1. Peta sebelas bagian . . . 38
6.2. Bentuk umum pengisian . . . 39
6.3. Bagian penutup . . . 40
6.4. Merender dan melihat hasilnya . . . 41
6.5. Bagian yang boleh dikosongkan . . . 42
6.6. Mengganti judul bagian Contoh dan Tambahan . . . 42
6.7. Ringkasan . . . 43
6.8. Uji pemahaman . . . 43

7. Contoh Praktikum yang Kodenya Benar-benar Berjalan **44**
7.1. Struktur satu contoh . . . 44
7.2. Menulis contoh pertama . . . 44
7.3. Menulis kode R di dalam YAML . . . 46
7.4. Mengendalikan eksekusi dengan `eval` . . . 48
7.5. Contoh dwibahasa: R dan Python . . . 48
7.6. Menyisipkan kotak di antara contoh . . . 49
7.7. Ringkasan . . . 50
7.8. Uji pemahaman . . . 50

8. Menyiapkan Data Latihan: Simulasi atau Data Nyata **51**
8.1. Untuk apa data simulasi . . . 51
8.2. Anatomi blok `datasets` . . . 51
8.3. Resep `tabular` dan jenis kolomnya . . . 52
8.4. Resep `mvn` dan `grouped_mvn` . . . 55
8.5. Memakai data nyata dengan `kind: file` . . . 56
8.6. Peran `seed` dan mengapa data tidak berubah . . . 57
8.7. Di mana berkas data berada . . . 58
8.8. Ringkasan . . . 59
8.9. Uji pemahaman . . . 59

9. Latihan, Penugasan, dan Kunci Jawaban **60**
9.1. Soal di `body`, solusi di `exercises` . . . 60
9.2. Latihan . . . 60
9.3. Penugasan . . . 61
9.4. Solusi, skor, dan interpretasi . . . 61
9.5. Merender dan memeriksa kunci jawaban . . . 63
9.6. Kunci berupa prosa panjang . . . 65
9.7. Ringkasan . . . 65
9.8. Uji pemahaman . . . 65

10. Kotak, Rumus, Tabel, dan Gambar **66**
10.1. Kapan Markdown cukup dan kapan perlu LaTeX . . . 66
10.2. Menulis LaTeX di dalam spesifikasi . . . 66
10.3. Katalog kotak bawaan . . . 67
10.4. Rumus matematika . . . 68
10.5. Tabel . . . 70
10.6. Gambar . . . 71
10.7. Karakter yang harus diloloskan . . . 71
10.8. Ringkasan . . . 72
10.9. Uji pemahaman . . . 72

III. Menerbitkan **74**

11. Merender: Satu Modul, Banyak Modul, Dua Backend **75**
11.1. Menyiapkan beberapa spesifikasi . . . 75
11.2. Tiga cara merender . . . 75
11.3. Ketika satu modul gagal . . . 78
11.4. Membaca log render . . . 78
11.5. Bookdown atau Quarto . . . 79
11.6. Ringkasan . . . 80
11.7. Uji pemahaman . . . 80

12. Menyusun Buku Modul dan Buku Kunci **81**
12.1. Susunan sebuah buku . . . 81
12.2. Mengapa penomoran perlu dua tahap . . . 81
12.3. Menyiapkan bagian awal . . . 81
12.4. Menjalankan penyusunan . . . 83
12.5. Memeriksa buku hasil gabungan . . . 85
12.6. Ringkasan . . . 86
12.7. Uji pemahaman . . . 86

13. Menyesuaikan Tampilan **88**
13.1. Objek tema dan palet warna . . . 88
13.2. Mengganti font . . . 90
13.3. Memakai tema pada ketiga cara merender . . . 91
13.4. Menambahkan daftar istilah . . . 92
13.5. Ringkasan . . . 94
13.6. Uji pemahaman . . . 94

14. Ketika Render Gagal **96**
14.1. Urutan penelusuran . . . 96

14.2. Galat YAML . . . 97
14.3. Galat isian spesifikasi . . . 97
14.4. Galat kode R . . . 97
14.5. Galat LaTeX . . . 98
14.6. Hasil tampak salah walau tanpa galat . . . 99
14.7. Meminta bantuan . . . 100
14.8. Ringkasan . . . 100
14.9. Uji pemahaman . . . 100

15. Alur Kerja Satu Semester **101**
15.1. Sebelum semester dimulai . . . 101
15.2. Menyunting satu modul di tengah semester . . . 101
15.3. Menyimpan proyek di Git . . . 101
15.4. Daftar periksa sebelum menerbitkan . . . 103
15.5. Berbagi proyek dengan tim pengampu . . . 104
15.6. Penutup . . . 104
15.7. Uji pemahaman . . . 104

Lampiran **106**

A. Rujukan Cepat Spesifikasi YAML **106**

B. Templat Spesifikasi Siap Salin **109**

C. Katalog Kotak, Warna, dan Karakter Khusus **112**

D. Fungsi Paket untuk Penyusun Modul **114**

E. Kunci Uji Pemahaman **116**

Kata Pengantar

Setiap semester, dosen pengampu mata kuliah praktikum menghadapi pekerjaan yang sama: menyiapkan empat belas modul, memperbarui contoh kode, membangkitkan data latihan, lalu menyusun kunci jawaban yang cocok dengan soalnya. Pekerjaan ini sering dikerjakan dengan menyalin berkas tahun lalu dan menyunting bagian yang berubah. Cara itu cepat pada minggu pertama, tetapi lama-kelamaan menimbulkan masalah: keluaran kode di modul tidak lagi sama dengan keluaran di komputer mahasiswa, data latihan berbeda antara modul dan kunci, dan nomor soal di kunci tertinggal ketika soal di modul diubah.

Paket R `modulstis` dibuat untuk memindahkan pekerjaan itu ke satu tempat. Dosen menulis isi modul dalam satu berkas teks berstruktur. Dari berkas itu, paket menyusun modul PDF, kunci jawaban PDF, dan berkas data latihan. Karena ketiganya berasal dari sumber yang sama, ketiganya selalu sinkron.

Buku ini memandu Anda memakai paket tersebut dari awal, mulai dari memasang perangkat di komputer sampai menerbitkan buku modul satu semester. Buku ini ditulis untuk dosen yang sudah memakai R untuk mengajar, tetapi belum pernah menulis YAML, R Markdown, Quarto, atau LaTeX. Setiap istilah baru dijelaskan ketika pertama kali muncul, dan setiap langkah disertai hasil yang seharusnya Anda lihat.

Contoh dalam buku ini memakai mata kuliah **Pemrograman Statistika** (K242310) Program Studi D-IV Statistika. Spesifikasi empat belas modulnya disertakan di dalam paket. Sepanjang buku, kita membangun ulang **Modul 7: Statistik Deskriptif dan Eksplorasi Data** sedikit demi sedikit, sehingga Anda melihat akibat setiap perubahan pada modul yang dihasilkan.

Seluruh kode di buku ini dijalankan ketika buku dibangun dengan `modulstis` versi 0.3.11. Keluaran yang tercetak adalah keluaran paket yang sebenarnya, dan gambar halaman modul adalah tangkapan dari PDF hasil render.

Jakarta, November 2026

Achmad Syahrul Choir

Cara Membaca Buku Ini

Dua jalur pembacaan

Jalur cepat. Bila Anda ingin segera melihat hasil, baca Bab 1 sampai Bab 3. Setelah Bab 3, Anda sudah memiliki satu modul PDF yang dihasilkan dari komputer Anda sendiri. Kembali ke bab lain ketika Anda membutuhkannya.

Jalur lengkap. Bila Anda akan menyusun modul satu semester, baca berurutan. Bab 4 sampai Bab 10 membangun Modul 7 bagian demi bagian. Bab 11 sampai Bab 15 membahas penerbitan, penyesuaian tampilan, penanganan galat, dan alur kerja selama semester.

Kotak dan penanda

Buku ini memakai beberapa kotak berwarna. Masing-masing punya tugas tetap.

> **Hasil yang seharusnya Anda lihat**
>
> Kotak hijau menunjukkan apa yang seharusnya muncul di layar atau di berkas setelah sebuah langkah. Bila yang Anda lihat berbeda, berhentilah dan baca kembali langkah sebelumnya.

> **Perhatian**
>
> Kotak oranye menandai langkah yang mudah keliru.

> **Salah paham umum**
>
> Kotak merah membahas keyakinan yang wajar, tetapi keliru, beserta contoh yang menunjukkan mengapa keliru.

> **Coba sendiri**
>
> Kotak biru tua berisi satu perubahan kecil yang dapat Anda coba sendiri, beserta akibat yang seharusnya terjadi.

Langkah kerja diberi nomor seperti ini:

Langkah 1 **Contoh penanda langkah**

Kalimat sesudah penanda menjelaskan apa yang dikerjakan.

Langkah yang berbeda menurut sistem operasi diberi penanda **Windows**, **macOS**, atau **Linux**. Kerjakan hanya bagian yang sesuai dengan komputer Anda.

Kesepakatan penulisan

- Kode yang Anda ketik di konsol R ditulis dalam blok abu-abu. Baris yang diawali `#>` adalah keluaran R, bukan bagian dari kode; jangan ikut diketik.
- Perintah untuk terminal (bukan konsol R) diberi tanda `bash` di sudut blok kode.
- Nama berkas, folder, fungsi, dan kunci YAML ditulis dengan huruf mesin tik, misalnya `spec/modul-07-deskriptif.yml`.
- Fungsi R ditulis dengan tanda kurung, misalnya `render_module()`, supaya mudah dibedakan dari nama objek.

Prasyarat

Anda diharapkan sudah dapat menjalankan kode R di RStudio, membaca berkas CSV, dan memakai fungsi dasar seperti `mean()` atau `summary()`. Anda tidak perlu menguasai LaTeX, YAML, atau Git. Bagian yang memerlukannya akan menjelaskan secukupnya.

Bagian I

Memulai

1 Apa yang Dikerjakan modulstis untuk Anda?

Mengapa kunci jawaban modul tahun lalu sering tidak cocok lagi dengan soalnya? Biasanya bukan karena dosennya lalai. Soal ada di satu berkas, kunci di berkas lain, dan data di tempat ketiga. Setiap kali satu berkas diubah, dua berkas lain harus diubah dengan tangan. Satu perubahan yang terlewat sudah cukup untuk membuat ketiganya tidak cocok.

Bab ini menjelaskan bagaimana `modulstis` menghilangkan masalah itu, dan apa yang Anda dapatkan ketika memakainya. Setelah membaca bab ini, Anda dapat menyebutkan tiga keluaran yang dihasilkan paket dari satu berkas spesifikasi, dan menjelaskan mengapa ketiganya tidak mungkin saling bertentangan.

1.1. Masalah menyusun empat belas modul

Bayangkan cara yang umum dipakai. Modul ditulis di Word. Contoh kode disalin dari RStudio, lalu keluarannya disalin juga sebagai teks atau tangkapan layar. Data latihan disimpan di folder bersama. Kunci jawaban ditulis di berkas Word lain.

Cara ini punya tiga kelemahan yang baru terasa setelah beberapa semester:

1. **Keluaran kode menjadi usang.** Ketika versi R atau paket berubah, keluaran yang ditempel di modul tidak lagi sama dengan yang dilihat mahasiswa. Tidak ada yang memeriksanya kecuali ada mahasiswa yang bertanya.
2. **Kunci tertinggal.** Soal nomor 3 diganti, tetapi kunci nomor 3 lupa diganti.
3. **Tampilan tidak seragam.** Empat belas modul yang disunting di waktu berbeda lambat laun punya ukuran huruf, warna, dan gaya judul yang berbeda-beda.

Ketiga kelemahan itu berakar pada hal yang sama: **isi yang sama disimpan di beberapa tempat**.

1.2. Satu sumber, tiga keluaran

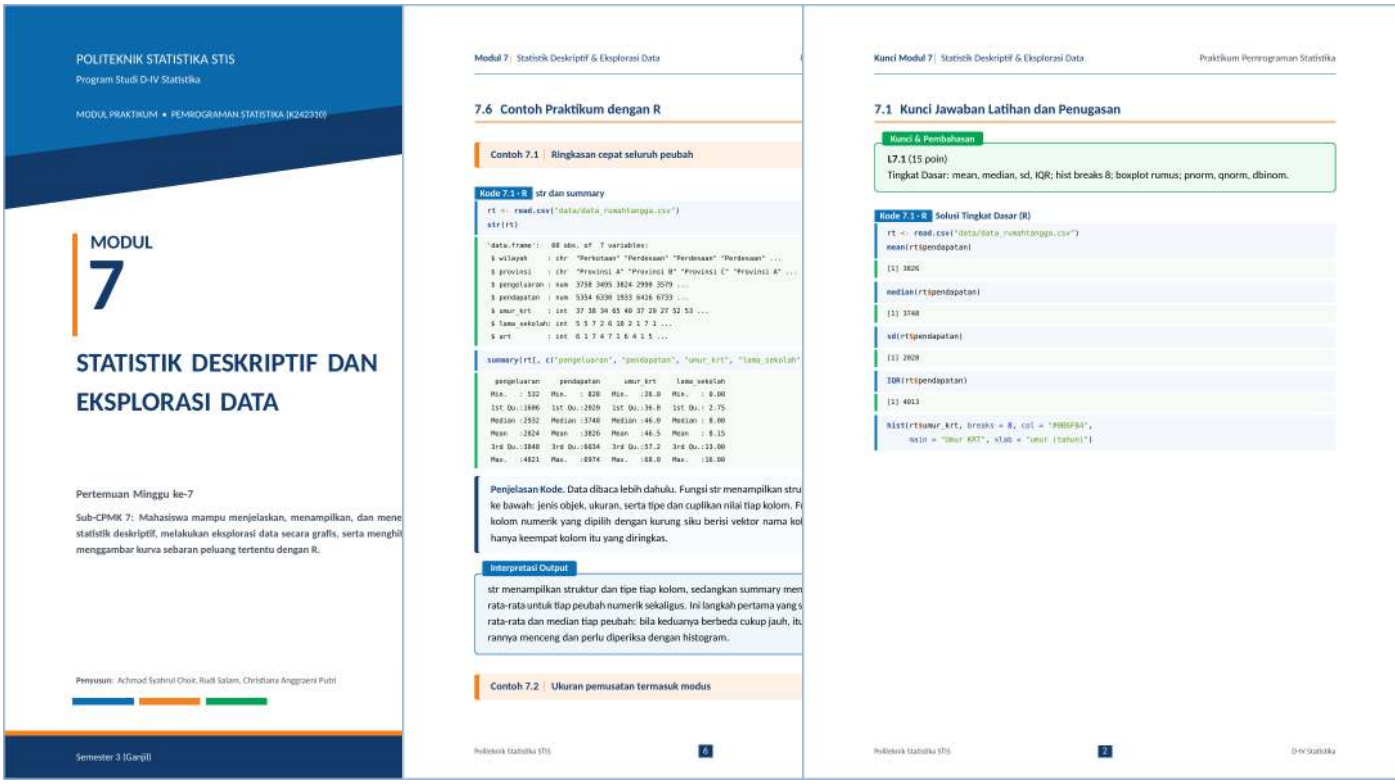
`modulstis` memindahkan seluruh isi modul ke satu berkas teks yang disebut **spesifikasi**. Spesifikasi adalah berkas berakhiran `.yml` yang memuat identitas modul, teks setiap bagian, contoh kode, soal latihan, solusi, dan resep data. Dari berkas itu paket menghasilkan tiga hal sekaligus:

- **modul praktikum** dalam PDF, lengkap dengan sampul, daftar isi, dan sebelas bagian baku;
- **kunci jawaban** dalam PDF, yang hanya memuat solusi dan pembahasan;
- **berkas data latihan** dalam CSV, yang dipakai oleh contoh di modul maupun solusi di kunci.

Kode di dalam spesifikasi **dijalankan** setiap kali modul dibuat. Keluaran yang tercetak di modul adalah

keluaran R di komputer Anda saat itu, bukan teks yang Anda tempel.

Gambar 1.1 memperlihatkan hasilnya untuk Modul 7 Pemrograman Statistika, yang spesifikasinya disertakan di dalam paket. Dari satu berkas sepanjang 655 baris, paket menghasilkan modul 35 halaman dan kunci 13 halaman.



(a) Sampul modul (b) Contoh praktikum dan keluaran kodenya (c) Kunci jawaban

Gambar 1.1.: Tiga halaman yang dihasilkan dari satu spesifikasi Modul 7.

Perhatikan halaman tengah. Blok berwarna di bawah kode adalah keluaran yang dihasilkan R ketika modul dibuat. Bila suatu hari Anda mengubah data atau kodenya, keluaran itu ikut berubah dengan sendirinya pada render berikutnya.

Data latihannya juga ikut dibuat. Berikut lima baris pertama berkas CSV yang dihasilkan untuk modul yang sama:

```
#>     wilayah   provinsi pengeluaran pendapatan umur_krt lama_sekolah art
#> 1 Perkotaan Provinsi A      3757.7     5353.6       37            5   6
#> 2 Perdesaan Provinsi B      3494.8     6330.4       38            5   1
#> 3 Perdesaan Provinsi C      3823.8     1933.2       34            7   7
#> 4 Perdesaan Provinsi A      2990.1     6416.3       65            2   4
#> 5 Perkotaan Provinsi A      3578.8     6732.9       40            6   7
```

1.2.1. Mengapa tidak mungkin tertinggal

Kunci jawaban tidak ditulis sebagai berkas terpisah. Paket merakitnya dari bagian `exercises` pada spesifikasi yang sama dengan soalnya. Data latihannya dibangkitkan dari resep yang sama, dengan bilangan acak awal (*seed*) yang sama, sehingga isinya identik untuk modul dan kunci. Bila Anda mengubah soal, Anda mengubahnya di satu berkas, dan render berikutnya menghasilkan modul serta kunci yang sudah menyesuaikan.

> Salah paham umum: "Saya harus menguasai LaTeX dulu"
>
> Modul `modulstis` memang disusun dengan LaTeX, tetapi Anda tidak menulis LaTeX untuk sebagian besar isinya. Paragraf, daftar bernomor, huruf tebal, dan kode ditulis dengan penanda sederhana seperti `**tebal**`. Sampul, kepala halaman, warna, dan kotak sudah disiapkan paket. Contoh Modul 7 di Bab 6 sampai Bab 9 tidak memuat satu pun perintah LaTeX. LaTeX baru dipakai di Bab 10, untuk kotak rumus dan kotak peringatan, dan bab itu menjelaskan perintah yang diperlukan satu per satu.

1.3. Apa yang tidak dikerjakan paket ini

Supaya harapan Anda tepat, ada beberapa hal yang memang bukan tugas `modulstis`:

- **Menulis isi modul.** Paket menyusun dan merapikan, tetapi teori, contoh, dan soal tetap Anda yang menulis.
- **Menyunting PDF.** PDF adalah hasil akhir. Bila ada yang ingin diubah, ubahlah spesifikasinya lalu render ulang. Menyunting PDF secara langsung akan hilang pada render berikutnya.
- **Menghasilkan Word.** Keluaran paket adalah PDF. Berkas antara (`.Rmd` atau `.qmd`) memang ikut tersimpan, tetapi tidak dirancang untuk diubah ke Word.
- **Menyediakan data resmi.** Resep data menghasilkan data simulasi. Bila Anda memerlukan data nyata, paket dapat memakai berkas data yang Anda sediakan (Bab 8).

1.4. Peta pedoman ini

Buku ini dibagi menjadi tiga bagian yang mengikuti urutan kerja Anda.

Tabel 1.1.: Peta isi buku

Bagian	Bab	Hasil yang Anda peroleh
Memulai	1–3	perangkat terpasang dan modul pertama berhasil dibuat
Menyusun Isi	4–10	spesifikasi Modul 7 yang lengkap: isi, contoh, data, latihan, kunci

Bagian	Bab	Hasil yang Anda peroleh
Menerbitkan	11–15	seluruh modul dirender, buku modul dan buku kunci, alur kerja satu semester

1.5. Ringkasan

`modulstis` menyusun modul praktikum, kunci jawaban, dan data latihan dari satu berkas spesifikasi. Karena ketiganya berasal dari sumber yang sama dan kode dijalankan setiap kali modul dibuat, keluaran di modul selalu mutakhir dan kunci tidak dapat tertinggal dari soal. Anda tetap menulis isinya; paket mengerjakan penyusunan, tampilan, dan sinkronisasi.

Bab berikutnya menyiapkan komputer Anda supaya dapat menjalankan semua ini.

1.6. Uji pemahaman

1. Sebutkan tiga keluaran yang dihasilkan `modulstis` dari satu spesifikasi.
2. Seorang rekan menyunting langsung PDF modul untuk memperbaiki salah ketik. Apa yang terjadi pada perbaikan itu ketika modul dirender ulang? Di mana seharusnya ia memperbaiki?
3. Mengapa data latihan di modul dan di kunci jawaban selalu sama?
4. (Transfer) Anda mengganti versi paket `dplyr` di komputer. Bagian mana dari modul yang mungkin berubah pada render berikutnya, dan mengapa perubahan itu justru diinginkan?

2 Menyiapkan Komputer Langkah demi Langkah

Bagaimana Anda tahu komputer sudah siap sebelum mencoba merender modul pertama? Tanpa pemeriksaan, Anda baru tahu ada yang kurang ketika render gagal dengan pesan panjang yang sukar dibaca. Bab ini memasang perangkat yang diperlukan satu per satu, lalu memeriksanya dengan satu perintah.

Setelah membaca bab ini, Anda dapat memasang semua perangkat di Windows, macOS, atau Linux, dan membaca laporan `cek_kesiapan()` untuk mengetahui apa yang masih kurang.

2.1. Daftar periksa kesiapan

Modul `modulstis` dibuat melalui rantai kerja berikut: R membaca spesifikasi dan menjalankan kode, R Markdown menyusun dokumen, lalu **XeLaTeX** mengubahnya menjadi PDF. XeLaTeX adalah program penyusun dokumen yang mengatur letak huruf, rumus, dan halaman. Setiap mata rantai memerlukan perangkatnya sendiri.

Tabel 2.1.: Perangkat yang diperlukan

No.	Komponen	Untuk apa	Wajib?
1	R (versi 4.1 atau lebih baru)	menjalankan paket dan kode di modul	ya
2	RStudio	tempat Anda bekerja sehari-hari	dianjurkan
3	TinyTeX (XeLaTeX)	menyusun PDF	ya
4	Pola pemenggalan bahasa Indonesia	memenggal kata di akhir baris dengan benar	ya
5	Font Carlito	huruf yang dipakai modul	ya
6	Paket `rmarkdown`, `bookdown`, `pdftools`, `qpdf`	menyusun dokumen dan menggabung PDF	ya
7	Quarto	hanya bila memakai backend Quarto	tidak
8	`modulstis`	paket itu sendiri	ya

Urutan di tabel adalah urutan pemasangan yang dianjurkan. Pasang berurutan, karena beberapa langkah bergantung pada langkah sebelumnya.

Perhatian

Pemasangan memerlukan koneksi internet, dan TinyTeX mengunduh sekitar 100 MB. Pada render pertama, TinyTeX juga mengunduh paket LaTeX tambahan yang dibutuhkan modul. Karena itu, render pertama bisa memakan waktu beberapa menit. Render berikutnya jauh lebih cepat.

2.2. Memasang R dan RStudio

Langkah 1 **Pasang R**

Buka https://cran.r-project.org, lalu pilih tautan untuk sistem operasi Anda.

Windows Pilih **Download R for Windows**, lalu **base**, lalu **Download R-x.y.z for Windows**. Jalankan berkas `.exe` yang terunduh dan terima semua pilihan bawaan.

macOS Pilih **Download R for macOS**. Ada dua berkas `.pkg`: pilih yang berakhiran `arm64` untuk Mac dengan cip Apple (M1, M2, dan seterusnya) dan yang berakhiran `x86_64` untuk Mac dengan prosesor Intel. Bila ragu, buka menu Apple, lalu **About This Mac**, dan lihat baris **Chip** atau **Processor**.

Linux Pada Ubuntu atau Debian, cara paling ringkas adalah melalui terminal:

```
sudo apt update
sudo apt install r-base r-base-dev
```

Versi R dari repositori bawaan distro kadang tertinggal. Bila Anda memerlukan versi terbaru, ikuti petunjuk di halaman **Download R for Linux** pada situs CRAN.

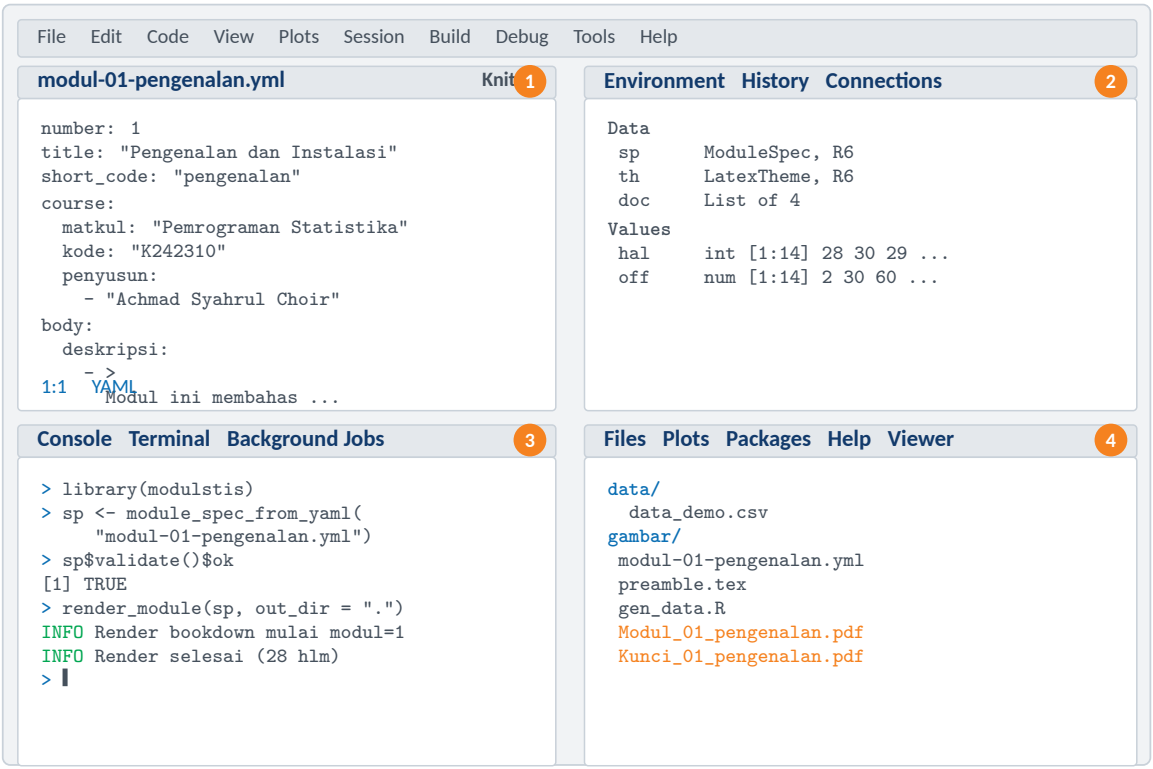
Langkah 2 **Pasang RStudio**

Buka https://posit.co/download/rstudio-desktop/, unduh RStudio Desktop untuk sistem operasi Anda, lalu pasang seperti aplikasi biasa. RStudio otomatis menemukan R yang sudah terpasang.

Hasil yang seharusnya Anda lihat

Buka RStudio. Panel kiri bawah (Console) menampilkan baris pertama seperti `R version 4.x.y`. Angka setelah `4.` harus 1 atau lebih besar.

Gambar 2.1 memperlihatkan tata letak RStudio yang akan dirujuk sepanjang buku ini.



Gambar 2.1.: Skema tata letak RStudio. Panel Source untuk menyunting berkas, Console untuk mengetik perintah R, Environment untuk melihat objek, dan Files untuk menjelajah folder proyek. Gambar ini skema, bukan tangkapan layar.

2.3. Memasang TinyTeX dan pola pemenggalan Indonesia

TinyTeX adalah distribusi LaTeX ringkas yang dipasang dan dikelola dari R. Kelebihannya bagi kita: paket LaTeX yang kurang diunduh sendiri ketika diperlukan, dan cara memasangnya sama di ketiga sistem operasi.

Langkah 3 **Pasang TinyTeX**

Ketik di Console RStudio:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

Tunggu sampai selesai, lalu **tutup dan buka kembali RStudio** supaya RStudio mengenali lokasi XeLaTeX yang baru.

> Perhatian
>
> Bila komputer Anda sudah memiliki distribusi LaTeX lain, misalnya MiKTeX di Windows atau MacTeX di macOS, `install_tinytex()` akan menanyakan apakah Anda tetap ingin memasang TinyTeX. Anda boleh tetap memakai distribusi lama, tetapi langkah pemasangan pola pemenggalan dan pa-

ket LaTeX di buku ini mengikuti TinyTeX.

Langkah 4 **Pasang pola pemenggalan bahasa Indonesia**

```
tinytex::tlmgr_install("hyphen-indonesian")
```

Linux Bila Anda memakai TeX Live dari repositori distro, bukan TinyTeX, pola ini ada di paket `texlive-lang-other`:

```
sudo apt install texlive-xetex texlive-latex-extra texlive-lang-other
```

Salah paham umum: "Modul tersusun tanpa galat, berarti pemenggalannya sudah benar"

Tanpa pola pemenggalan Indonesia, render **tetap berhasil** dan tidak ada pesan galat. Bedanya hanya pada kata yang terpotong di akhir baris: XeLaTeX memakai kaidah bahasa Inggris, sehingga *menyatakan* bisa terpotong menjadi *meny-atakan*, padahal kaidah EYD memenggalnya *me-nya-ta-kan*. Kesalahan ini baru terlihat ketika seseorang membaca modulnya dengan teliti. Karena itulah `cek_kesiapan()` di akhir bab ini memeriksa pola pemenggalan secara khusus.

2.4. Memasang font Carlito

Modul memakai font **Carlito**, yaitu font terbuka yang ukuran hurufnya sama dengan Calibri. Font ini harus terpasang di sistem operasi, bukan di R.

Windows Buka https://fonts.google.com/specimen/Carlito, klik **Get font**, lalu **Download all**. Ekstrak berkas ZIP-nya. Pilih keempat berkas `.ttf`, klik kanan, lalu pilih **Install for all users** (pada Windows 11, pilihan ini ada di bawah **Show more options**).

Perhatian

Di Windows, pilih **Install for all users**, bukan sekadar **Install**. Pilihan **Install** menaruh font di folder milik akun Anda saja, dan XeLaTeX sering tidak menemukannya di sana. Gejalanya adalah pesan `The font "Carlito" cannot be found` ketika render.

macOS Bila Anda memakai Homebrew, jalankan di Terminal:

```
brew install --cask font-carlito
```

Tanpa Homebrew, unduh font dari Google Fonts seperti langkah Windows, lalu klik dua kali setiap berkas `.ttf` dan pilih **Install Font**.

Linux Pada Ubuntu atau Debian:

```
sudo apt install fonts-crosextra-carlito
```

Pada Fedora, nama paketnya `google-carlito-fonts`.

2.5. Memasang paket R pendukung

Langkah 5 **Pasang paket R yang diperlukan**

```
install.packages(c("rmarkdown", "bookdown", "knitr", "pdftools", "qpdf",
↪  "remotes"))
```

Paket `pdftools` dan `qpdf` dipakai untuk menghitung halaman dan menggabung PDF ketika Anda menyusun buku modul (Bab 12).

Linux Pada Linux, `pdftools` dipasang dari kode sumber dan memerlukan pustaka Poppler. Pasang lebih dahulu:

```
sudo apt install libpoppler-cpp-dev
```

2.6. Memasang Quarto (opsional)

Bagian ini hanya perlu bila Anda ingin memakai backend Quarto (Bab 11). Backend bawaan, yaitu bookdown, tidak memerlukannya. Bila ragu, lewati dulu bagian ini.

Unduh pemasang dari https://quarto.org/docs/get-started/ untuk sistem operasi Anda, pasang, lalu jalankan di Console:

```
install.packages("quarto")
```

2.7. Memasang modulstis dari GitHub

`modulstis` belum tersedia di CRAN, sehingga dipasang langsung dari repositori GitHub-nya dengan paket `remotes`.

Langkah 6 **Pasang modulstis**

```
remotes::install_github("madsyair/modulstis")
```

`remotes` akan memasang juga paket yang dibutuhkan `modulstis`, yaitu `R6`, `yaml`, `purrr`, dan `fs`, bila belum ada.

Catatan

Windows Anda mungkin melihat peringatan bahwa **Rtools** diperlukan. Rtools hanya dibutuhkan untuk paket yang berisi kode C atau C++. `modulstis` seluruhnya ditulis dalam R, sehingga peringatan itu boleh diabaikan selama pemasangan diakhiri dengan pesan `* DONE (modulstis).`

2.8. Memeriksa kesiapan dengan satu perintah

Langkah 7 **Jalankan pemeriksaan**

Setelah semua langkah di atas, jalankan:

```
library(modulstis)
cek_kesiapan()
#> Kesiapan komputer untuk modulstis
#> [OK]    R versi 4.1 atau lebih baru
#> [OK]    paket R 'rmarkdown'
#> [OK]    paket R 'knitr'
#> [OK]    paket R 'bookdown'
#> [OK]    paket R 'pdftools' atau 'qpdf' (buku gabungan)
#> [OK]    XeLaTeX
#> [OK]    font Carlito
#> [OK]    pola pemenggalan bahasa Indonesia
#> [OK]    Quarto (hanya untuk backend "quarto")
#> Semua komponen wajib siap. Anda dapat merender modul.
```

Fungsi ini tidak hanya mencari program di komputer Anda. Untuk font dan pola pemenggalan, ia benar-benar menyusun dokumen kecil dengan XeLaTeX. Bila dokumen itu berhasil menjadi PDF, komponen tersebut dianggap siap. Dengan cara ini, hasilnya berlaku sama di Windows, macOS, dan Linux.

Hasil yang seharusnya Anda lihat

Semua baris komponen wajib bertanda `[OK]`, dan baris terakhir berbunyi *Semua komponen wajib siap*. Baris Quarto boleh bertanda `[ - ]`; itu berarti Quarto belum terpasang, dan tidak masalah selama Anda memakai backend bookdown.

Bila ada komponen yang belum siap, laporan menuliskan saran di bawah baris tersebut. Contoh berikut adalah keluaran `cek_kesiapan()` yang dijalankan pada komputer yang belum memiliki XeLaTeX:

```
#> Kesiapan komputer untuk modulstis
#> [OK]    R versi 4.1 atau lebih baru
#> [OK]    paket R 'rmarkdown'
#> [OK]    paket R 'knitr'
```

```
#> [OK]    paket R 'bookdown'
#> [OK]    paket R 'pdftools' atau 'qpdf' (buku gabungan)
#> [BELUM] XeLaTeX
#>           saran: install.packages("tinytex"); tinytex::install_tinytex(); lalu
 ↪  mulai ulang RStudio
#> [BELUM] font Carlito
#>           saran: Pasang XeLaTeX lebih dahulu
#> [BELUM] pola pemenggalan bahasa Indonesia
#>           saran: Pasang XeLaTeX lebih dahulu
#> [ - ]   Quarto (hanya untuk backend "quarto")
#>           saran: Pasang Quarto dari https://quarto.org lalu
 ↪  install.packages("quarto")
#> 3 komponen wajib belum siap. Ikuti saran di atas, lalu jalankan ulang.
```

Kerjakan saran pada baris bertanda `[BELUM]` dari atas ke bawah, lalu jalankan `cek_kesiapan()` sekali lagi. Ulangi sampai semua komponen wajib bertanda `[OK]`.

> **Coba sendiri**
>
> Jalankan `cek_kesiapan()`, lalu simpan hasilnya ke objek: `lap <- cek_kesiapan(tampilkan = FALSE)`. Ketik `lap$status` dan bandingkan dengan laporan yang tercetak. Anda akan melihat bahwa laporan tercetak hanyalah tampilan yang lebih rapi dari tabel yang sama.

2.9. Galat pemasangan yang paling sering

Tabel 2.2.: Galat pemasangan dan penyelesaiannya

Gejala	Penyebab	Penyelesaian
`xelatex not found` setelah memasang TinyTeX	RStudio belum dimulai ulang	tutup dan buka kembali RStudio
`The font "Carlito" cannot be found`	Windows: font dipasang untuk satu pengguna	hapus font, pasang ulang dengan **Install for all users**
`there is no package called 'modulstis'`	pemasangan dari GitHub gagal atau belum dijalankan	jalankan ulang `remotes::install_github()` dan baca pesan terakhirnya
pemasangan `pdftools` gagal di Linux	pustaka Poppler belum ada	`sudo apt install libpoppler-cpp-dev`
kata terpenggal tidak sesuai EYD	pola pemenggalan Indonesia belum dipasang	`tinytex::tlmgr_install("hyphen-indonesian`

2.10. Ringkasan

Modul dibuat lewat rantai R, R Markdown, dan XeLaTeX. Setiap mata rantai memerlukan perangkatnya: R dan RStudio, TinyTeX beserta pola pemenggalan Indonesia, font Carlito, paket R pendukung, dan `modulstis` sendiri. Quarto hanya diperlukan untuk backend Quarto. `cek_kesiapan()` memeriksa semuanya sekaligus dan memberi saran untuk setiap komponen yang belum siap.

Komputer Anda sekarang siap. Bab berikutnya membuat modul pertama.

2.11. Uji pemahaman

1. Mengapa render modul tetap berhasil walaupun pola pemenggalan Indonesia belum dipasang? Bagaimana Anda tahu pola itu belum terpasang?
2. Seorang rekan di Windows sudah memasang Carlito, tetapi render gagal dengan pesan bahwa font tidak ditemukan. Apa dugaan Anda, dan apa yang perlu ia lakukan?
3. Komponen mana di Tabel 2.1 yang boleh dilewati, dan dalam keadaan apa?
4. (Transfer) Laboratorium kampus memasang ulang semua komputer setiap awal semester. Tuliskan urutan perintah paling ringkas yang perlu dijalankan asisten untuk menyiapkan satu komputer, lalu jelaskan perintah mana yang memastikan pekerjaannya sudah benar.

3 Modul Pertama dalam Lima Belas Menit

Apa langkah paling pendek dari folder kosong ke PDF modul pertama? Jawabannya: biarkan paket membuatkan proyek yang sudah berisi satu spesifikasi contoh, lalu render. Bab ini mengerjakan persis itu, kemudian mengubah satu baris untuk melihat bahwa modul memang mengikuti spesifikasinya.

Setelah membaca bab ini, Anda dapat membuat proyek modul baru, merender modul dan kunci jawabannya, serta menemukan berkas hasilnya.

Bab ini memulai contoh berjalan buku: proyek mata kuliah **Pemrograman Statistika**.

3.1. Membuat proyek

Proyek dalam RStudio adalah sebuah folder yang dibuka sebagai satu kesatuan kerja. Ketika proyek dibuka, folder itu otomatis menjadi direktori kerja, sehingga semua nama berkas dapat ditulis pendek, misalnya `spec/modul-07-deskriptif.yml`, tanpa jalur lengkap.

Langkah 1 **Buat proyek dari menu RStudio**

Pilih **File → New Project → New Directory**. Dalam daftar jenis proyek, pilih **Proyek Modul Praktikum (modulstis)**. Pilihan ini muncul setelah `modulstis` terpasang. Isi kotak isian sebagai berikut, lalu klik **Create Project**.

Tabel 3.1.: Isian proyek baru

Isian	Nilai untuk contoh berjalan
Directory name	`pemrograman-statistika`
Nama mata kuliah	Pemrograman Statistika
Kode mata kuliah	K242310
Program studi	D-IV Statistika
Semester	3 (Ganjil)
Penyusun	nama Anda
Banyaknya modul	14
Sertakan spesifikasi contoh	dicentang

RStudio membuka proyek baru dan menampilkan dua berkas di panel Source: `README.md` dan `render_modul.R`.

> **Tanpa menu RStudio**
>
> Perintah berikut menghasilkan proyek yang sama. Jalankan di Console dari folder tempat Anda ingin menaruh proyek, lalu buka proyeknya dengan **File → Open Project**.

```
library(modulstis)
buat_proyek_modul("pemrograman-statistika",
  matkul = "Pemrograman Statistika", kode = "K242310",
  prodi = "D-IV Statistika", semester = "3 (Ganjil)",
  penyusun = "Achmad Syahrul Choir", n_modul = 14)
```

3.2. Mengenali isi folder proyek

Langkah 2 **Lihat isi folder**

Buka panel **Files**, atau jalankan:

```
list.files(recursive = TRUE)
#> [1] "README.md"                "render_modul.R"
#> [3] "spec/modul-01-contoh.yml"
```

Ada tiga folder dan tiga berkas:

- `spec/` menyimpan spesifikasi, **satu berkas `.yml` per modul**. Folder ini sudah berisi satu spesifikasi contoh, `modul-01-contoh.yml`.
- `modul/` menampung hasil render. Isinya dibuat ulang setiap kali Anda merender.
- `buku/` disiapkan untuk bagian awal dan akhir buku gabungan (Bab 12).
- `render_modul.R` adalah skrip yang merender semua spesifikasi di `spec/`.
- `README.md` berisi catatan proyek dan prasyaratnya.
- Berkas `.gitkeep` hanyalah penanda kosong agar folder yang belum berisi tetap ikut tersalin. Anda tidak perlu menyentuhnya.

Bila Anda membuat proyek lewat menu RStudio, ada juga berkas `pemrograman-statistika.Rproj`. Klik dua kali berkas itu untuk membuka proyek di lain waktu.

> Salah paham umum: “Isi modul diubah di PDF atau di berkas .Rmd hasil render”
>
> Folder `modul/` berisi hasil, bukan sumber. Setiap render menulis ulang berkas di sana, termasuk berkas `.Rmd` dan `.tex` antara. Perubahan yang Anda buat langsung di folder itu akan hilang pada render berikutnya. Satu-satunya tempat menyunting isi modul adalah berkas `.yml` di folder `spec/`.

3.3. Membaca spesifikasi contoh

Langkah 3 **Buka spesifikasi contoh**

Klik dua kali `spec/modul-01-contoh.yml` di panel Files. Baris-baris pertamanya seperti ini:

```yaml
number: 1
title: "Judul Modul Pertama"
short_code: "contoh"
topic: "Topik Singkat"
pertemuan: 1
sub_cpmk: >
  Sub-CPMK 1: Mahasiswa mampu menjelaskan dan menerapkan pokok bahasan
  pada modul ini.
packages: [dplyr]
contoh_title: "Contoh Praktikum dengan R"
course:
  matkul: "Pemrograman Statistika"
  matkul_besar: "PEMROGRAMAN STATISTIKA"
  kode: "K242310"
  semester: "3 (Ganjil)"
  prodi: "D-IV Statistika"
  penyusun:
    - "Achmad Syahrul Choir"
```

Anda belum perlu memahami setiap baris. Untuk sekarang cukup kenali polanya: setiap baris berbentuk `nama: nilai`. Nama di kiri titik dua disebut **kunci**, dan isi di kanan disebut **nilai**. Baris yang menjorok ke dalam merupakan bagian dari kunci di atasnya; misalnya `matkul` dan `kode` adalah bagian dari `course`. Aturan lengkapnya dibahas di Bab 4.

3.4. Memeriksa spesifikasi

Langkah 4 **Periksa spesifikasi sebelum merender**

```r
library(modulstis)
sp <- module_spec_from_yaml("spec/modul-01-contoh.yml")
sp$validate()
#> □ Spesifikasi valid.
```

`module_spec_from_yaml()` membaca berkas YAML menjadi objek spesifikasi di R, dan `validate()` memeriksa isinya. Pemeriksaan ini hanya beberapa detik, sedangkan render bisa memakan setengah menit. Karena itu, biasakan memeriksa lebih dahulu.

Hasil yang seharusnya Anda lihat

Konsol menampilkan ✔ `Spesifikasi valid.` Bila yang muncul ✖ `Spesifikasi tidak valid` beserta daftar pesan, baca pesannya: setiap baris menyebut kunci yang bermasalah.

3.5. Merender

Langkah 5 **Render semua spesifikasi**

Buka `render_modul.R` di panel Source, lalu klik tombol **Source** di kanan atas panel. Tombol itu sama dengan menjalankan perintah berikut di Console:

```
source("render_modul.R")
#> INFO  [07:21:07] Render (bookdown/rmarkdown) modul=1 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:21:13] Render (bookdown/rmarkdown) modul=1 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#>   selesai: Modul01_contoh
```

Keluaran ini terdiri atas tiga jenis baris:

- Baris berawalan `INFO` adalah catatan kerja paket. Ada satu baris untuk modul (`jenis=modul`) dan satu untuk kunci jawabannya (`jenis=kunci`).
- Baris `Warning: Package tocloft Warning` berasal dari LaTeX ketika menyusun daftar isi. Peringatan ini muncul pada setiap render dan **tidak memengaruhi hasil**, sehingga boleh diabaikan.
- Baris terakhir, `selesai: Modul01_contoh`, menyebut folder hasilnya.

Render satu modul biasanya memakan waktu sepuluh sampai tiga puluh detik. Render pertama setelah memasang TinyTeX bisa jauh lebih lama karena TinyTeX mengunduh paket LaTeX yang dibutuhkan.

Dari terminal

Skrip yang sama dapat dijalankan dari tab **Terminal** RStudio. Cara ini berguna ketika Anda hanya ingin merender sebagian modul:

```
Rscript render_modul.R          # semua spesifikasi
Rscript render_modul.R 3 5      # modul ketiga sampai kelima dalam urutan
 ↪  berkas
```

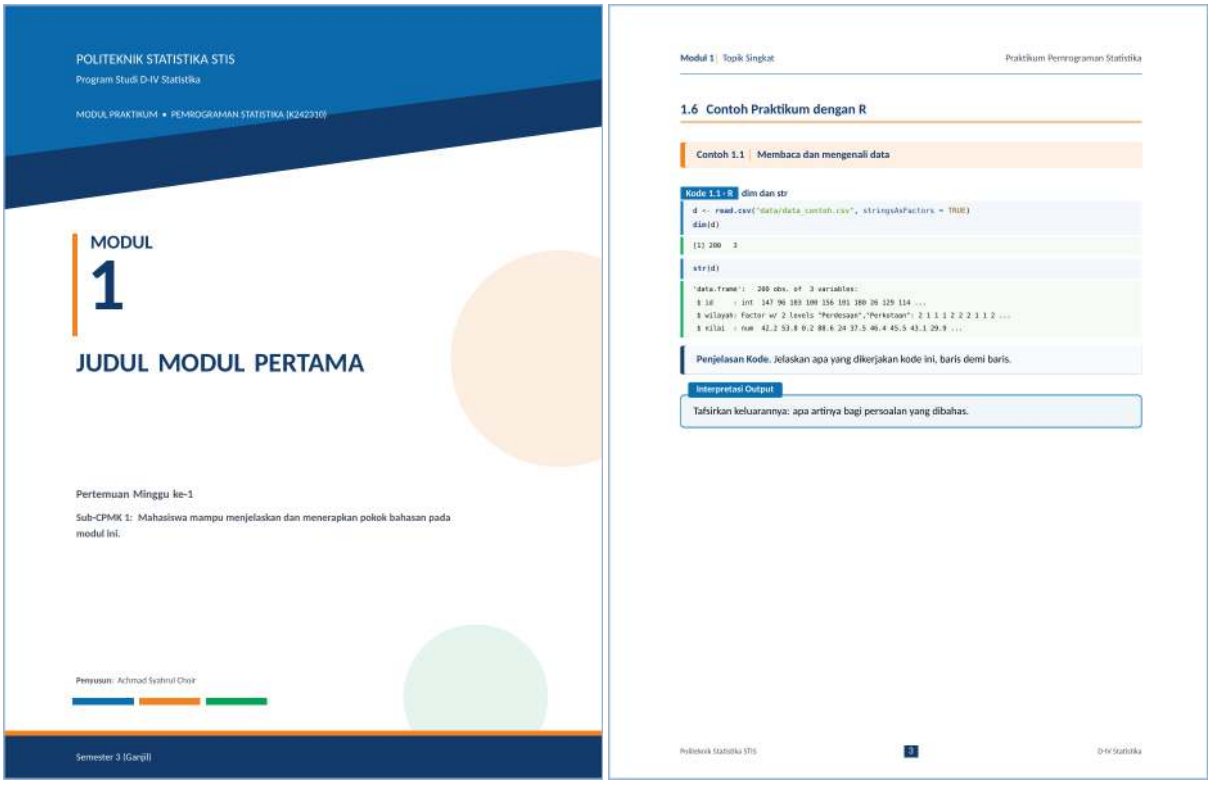
3.6. Melihat hasil

Langkah 6 **Buka berkas hasil**

```
list.files("modul", recursive = TRUE, pattern = "\\.(pdf|csv)$")
#> [1] "Modul01_contoh/data/data_contoh.csv"
#> [2] "Modul01_contoh/Kunci_01_contoh.pdf"
#> [3] "Modul01_contoh/Modul_01_contoh.pdf"
```

Setiap spesifikasi menghasilkan satu folder di `modul/`. Nama foldernya dibentuk dari nomor modul dan kode singkatnya: `Modul01_contoh`. Di dalamnya ada tiga hal yang kita bahas di Bab 1: modul PDF, kunci jawaban PDF, dan folder `data/` berisi berkas CSV.

Klik berkas PDF di panel Files, lalu pilih **View File**. Gambar 3.1 memperlihatkan sampul dan satu halaman isi modul tersebut.



(a) Sampul (b) Contoh praktikum beserta keluaran kodenya

Gambar 3.1.: Modul pertama yang dihasilkan dari spesifikasi contoh.

Hasil yang seharusnya Anda lihat

Sampul menampilkan nomor modul **1**, judul **JUDUL MODUL PERTAMA**, dan nama mata kuliah yang Anda isi saat membuat proyek. Bila nama mata kuliah di sampul berbeda dari isian Anda, buka kembali spesifikasinya dan periksa blok `course`.

3.7. Mengubah satu baris, lalu merender ulang

Sekarang kita buktikan bahwa modul benar-benar mengikuti spesifikasinya.

Langkah 7 Ganti judul modul

Di `spec/modul-01-contoh.yml`, ubah baris kedua menjadi:

```
title: "Pengenalan R dan RStudio"
```

Simpan berkas (**Ctrl+S**, atau **Cmd+S** di macOS), lalu klik **Source** pada `render_modul.R` sekali lagi.



Gambar 3.2.: Sampul setelah judul diubah dan modul dirender ulang.

Judul di sampul, di daftar isi, dan di kepala setiap halaman berubah sekaligus. Anda hanya mengubah satu baris.

> **Coba sendiri**
>
> Ubah nilai `pertemuan` di spesifikasi contoh dari `1` menjadi `2`, simpan, lalu render ulang. Cari di sampul tulisan yang berubah. Kemudian kembalikan nilainya menjadi `1`.

3.8. Ringkasan

Proyek `modulstis` adalah folder dengan susunan tetap: spesifikasi di `spec/`, hasil di `modul/`, dan skrip `render_modul.R` untuk merender semuanya. Alur kerjanya selalu sama: sunting spesifikasi, periksa dengan `validate()`, render, lalu buka hasilnya. Hasil render selalu dibuat ulang dari spesifikasi, sehingga perubahan dilakukan di spesifikasi, bukan di PDF.

Spesifikasi contoh tadi masih berisi teks pengisi. Bagian II menulis spesifikasi sungguhan, yaitu Modul 7 Pemrograman Statistika. Sebelum itu, Bab 4 membahas aturan penulisan YAML yang perlu Anda keta-

hui.

3.9. Uji pemahaman

1. Sebutkan isi ketiga folder dalam proyek `modulstis` dan fungsi masing-masing.
2. Mengapa `validate()` sebaiknya dijalankan sebelum merender?
3. Seorang asisten menyunting berkas `Modul_01_contoh.Rmd` di folder `modul/` untuk memperbaiki salah ketik. Apa yang terjadi pada render berikutnya?
4. (Transfer) Anda baru saja menambahkan berkas `spec/modul-02-dasar.yml`. Tanpa membaca ulang bab ini, tebak nama folder dan nama berkas PDF modul yang akan muncul di `modul/` setelah render, bila nilai `short_code` di berkas itu adalah `dasar`.

Bagian II

Menyusun Isi

4 YAML Secukupnya untuk Menulis Modul

Mengapa satu spasi yang bergeser bisa membuat modul gagal dirender? Karena spesifikasi ditulis dalam **YAML**, dan YAML memakai spasi di awal baris untuk menyatakan bagian mana yang termasuk ke dalam bagian mana. Bagi kita spasi itu hanya kerapian. Bagi YAML, spasi itu adalah struktur.

Bab ini membahas YAML secukupnya untuk menulis modul: lima aturan yang mencegah hampir semua galat, dan cara membaca pesan galat bila aturan itu terlanggar. Setelah membaca bab ini, Anda dapat membaca spesifikasi apa pun di buku ini, menulis teks panjang dan kode dengan benar, serta menemukan letak kesalahan dari pesan galat YAML.

4.1. Kunci, nilai, dan indentasi

YAML adalah cara menulis data berstruktur dalam teks biasa, sehingga mudah dibaca manusia dan mudah dibaca program. Satuan terkecilnya adalah pasangan **kunci** dan **nilai**, dipisah oleh titik dua dan satu spasi:

```yaml
number: 7
title: "Statistik Deskriptif dan Eksplorasi Data"
pertemuan: 7
```

Ketika R membaca tiga baris ini, hasilnya adalah daftar bernama (*list*) dengan tiga unsur. Anda dapat melihatnya sendiri:

```
yaml::yaml.load('
number: 7
title: "Statistik Deskriptif dan Eksplorasi Data"
pertemuan: 7
') |> str()
#> List of 3
#>  $ number   : int 7
#>  $ title    : chr "Statistik Deskriptif dan Eksplorasi Data"
#>  $ pertemuan: int 7
```

Bagaimana bila sebuah nilai terdiri atas beberapa bagian? Tuliskan kuncinya, akhiri dengan titik dua tanpa nilai, lalu tulis bagian-bagiannya di baris berikut dengan **menjorok** ke dalam. Jarak menjorok di awal baris itu disebut **indentasi**.

```yaml
course:
  matkul: "Pemrograman Statistika"
  kode: "K242310"
```

Di sini `matkul` dan `kode` adalah bagian dari `course` karena keduanya menjorok dua spasi di bawahnya. Bila `kode` ditulis tanpa indentasi, ia bukan lagi bagian dari `course`, melainkan kunci tersendiri di tingkat atas.

> **Satu aturan indentasi untuk seluruh buku**
>
> Semua spesifikasi di buku ini memakai **dua spasi** untuk setiap tingkat. YAML sebenarnya menerima jumlah spasi lain, asalkan konsisten dalam satu tingkat. Memakai dua spasi di mana pun adalah cara termudah untuk tetap konsisten.

4.2. Daftar

Daftar ditulis dengan tanda hubung dan satu spasi di awal setiap butir:

```yaml
penyusun:
  - "Achmad Syahrul Choir"
  - "Rudi Salam"
```

Daftar pendek boleh ditulis dalam satu baris dengan kurung siku:

```yaml
packages: [dplyr, ggplot2]
```

Butir daftar boleh berisi pasangan kunci dan nilai. Bentuk inilah yang dipakai untuk contoh praktikum dan latihan. Perhatikan bahwa kunci kedua dan seterusnya sejajar dengan kunci pertama, yaitu sejajar dengan huruf setelah tanda hubung:

```yaml
contoh:
  - contoh: "Ringkasan cepat seluruh peubah"
    judul_r: "str dan summary"
    eval: true
  - contoh: "Ukuran pemusatan"
    judul_r: "mean dan median"
    eval: true
```

Potongan ini berisi satu kunci, `contoh`, yang nilainya daftar dengan dua butir. Setiap butir memiliki tiga kunci.

4.3. Teks panjang: perbedaan > dan |

Paragraf modul biasanya lebih panjang dari satu baris. YAML menyediakan dua penanda untuk teks panjang, dan keduanya sering tertukar.

- `>` **melipat** baris. Baris-baris di bawahnya digabung menjadi satu paragraf, dan pergantian baris diganti spasi. Pakai untuk **prosa**.
- `|` **mempertahankan** pergantian baris apa adanya. Pakai untuk **kode**.

```
teks <- yaml::yaml.load('
prosa: >
  Statistik deskriptif memampatkan
  ratusan amatan menjadi beberapa ukuran.
kode: |
  rt <- read.csv("data/data_rumahtangga.csv")
  summary(rt)
')
cat(teks$prosa)
#> Statistik deskriptif memampatkan ratusan amatan menjadi beberapa ukuran.
cat(teks$kode)
#> rt <- read.csv("data/data_rumahtangga.csv")
#> summary(rt)
```

Pada keluaran pertama, dua baris prosa menjadi satu kalimat. Pada keluaran kedua, dua baris kode tetap dua baris. Bila Anda menulis kode dengan >, kedua baris kode akan digabung menjadi satu baris dan R gagal menjalankannya.

4.4. Kapan teks harus diberi tanda kutip

Sebagian besar teks boleh ditulis tanpa tanda kutip. Namun, ada beberapa keadaan yang membuat YAML salah membaca teks Anda. Aturan paling aman bagi pemula: **beri tanda kutip ganda pada setiap teks yang ditulis dalam satu baris**. Aturan ini juga dipakai di semua spesifikasi bawaan paket.

Tiga keadaan berikut paling sering menimbulkan masalah bila tanda kutip dilupakan.

Teks berisi titik dua diikuti spasi. YAML mengira bagian sesudahnya adalah nilai baru.

Teks berisi tanda pagar didahului spasi. Bagian sesudah # dianggap komentar dan dibuang tanpa peringatan:

```
yaml::yaml.load('title: Modul 7 #revisi')$title
#> [1] "Modul 7"
```

Teks yang kebetulan berarti benar atau salah. YAML membaca `yes`, `no`, `on`, `off`, `y`, dan `n` sebagai nilai logis `TRUE` atau `FALSE`, bukan sebagai teks:

```
yaml::yaml.load('levels: [yes, no, Ya, Tidak]')$levels |> str()
#> List of 4
#>  $ : logi TRUE
#>  $ : logi FALSE
#>  $ : chr "Ya"
#>  $ : chr "Tidak"
```

`Ya` dan `Tidak` aman, tetapi `yes` dan `no` berubah menjadi `TRUE` dan `FALSE`. Bila sebuah kategori data simulasi Anda bernama `yes`, tulislah `"yes"`.

Salah paham umum: “Tanda kutip pada `"n": 60` hanyalah selera penulis”

Resep data di buku ini menulis `"n": 60` dengan tanda kutip, sedangkan kunci lain tidak. Ini bukan selera. Tanpa tanda kutip, YAML membaca kunci `n` sebagai `FALSE`:

```
names(yaml::yaml.load('n: 60\nseed: 2025'))
#> [1] "FALSE" "seed"
```

`modulstis` memperbaiki nama kunci ini secara otomatis untuk resep data, sehingga modul Anda tetap benar. Namun, membiasakan tanda kutip pada `"n"` membuat spesifikasi tetap benar ketika dibaca program lain, dan mengingatkan Anda pada perilaku YAML ini.

4.5. Komentar

Baris yang diawali `#` adalah komentar. YAML mengabaikannya, sehingga Anda bebas menulis catatan untuk diri sendiri:

```
# Revisi semester ganjil 2026: contoh 3 diganti data baru
number: 7
```

4.6. Lima galat YAML yang paling sering

Bagian ini sengaja membuat kesalahan, supaya Anda mengenali pesannya bila kelak melihatnya sendiri. Setiap contoh membaca spesifikasi pendek yang rusak dengan `module_spec_from_yaml()`.

Galat 1: titik dua di dalam nilai tanpa tanda kutip.

```
title: Modul 7: Statistik Deskriptif
```

```
library(modulstis)
module_spec_from_yaml("latihan-yaml/galat-1.yml")
```

```
#> Error: (latihan-yaml/galat-1.yml) Scanner error: mapping values are not allowed
 ↪  in this context at line 2, column 15
```

Pesan *mapping values are not allowed in this context* berarti YAML menemukan titik dua yang tidak ia harapkan. `line 2, column 15` menunjuk titik dua kedua di baris 2. Perbaikannya: `title: "Modul 7: Statistik Deskriptif"`.

Galat 2: indentasi memakai tab.

```
module_spec_from_yaml("latihan-yaml/galat-2.yml")
#> Error: (latihan-yaml/galat-2.yml) Scanner error: while scanning for the next
 ↪  token at line 4, column 1 found character that cannot start any token at line
 ↪  4, column 1
```

found character that cannot start any token pada kolom 1 hampir selalu berarti ada karakter tab di awal baris. YAML tidak menerima tab untuk indentasi.

> Mencegah tab di RStudio
>
> Buka **Tools → Global Options → Code → Editing**, lalu pastikan **Insert spaces for Tab** dicentang dan **Tab width** bernilai 2. Setelah itu, tombol Tab menghasilkan dua spasi.

Galat 3: indentasi tidak sejajar.

```
course:
  matkul: "Pemrograman Statistika"
   kode: "K242310"
```

```
module_spec_from_yaml("latihan-yaml/galat-3.yml")
#> Error: (latihan-yaml/galat-3.yml) Parser error: while parsing a block mapping at
 ↪  line 4, column 3 did not find expected key at line 5, column 4
```

Baris `kode` menjorok tiga spasi, sedangkan `matkul` dua spasi. Pesan *did not find expected key* berarti YAML sedang membaca sekelompok kunci yang sejajar, lalu menemukan baris yang tidak sejajar dengan kelompok itu. Letak yang disebut, baris 5 kolom 4, menunjuk baris yang menjorok berlebih.

Galat 4: tanda kutip tidak ditutup.

```
module_spec_from_yaml("latihan-yaml/galat-4.yml")
#> Error: (latihan-yaml/galat-4.yml) Scanner error: while scanning a quoted scalar
 ↪  at line 2, column 8 found unexpected end of stream at line 3, column 13
```

Tanda kutip pembuka di baris 2 tidak punya pasangan, sehingga YAML terus membaca sampai akhir

berkas. Perhatikan bahwa pesannya menyebut letak **awal** teks yang bermasalah, bukan letak yang seharusnya diberi tanda kutip penutup.

Galat 5: tidak ada spasi sesudah titik dua.

```
module_spec_from_yaml("latihan-yaml/galat-5.yml")
#> Error: (latihan-yaml/galat-5.yml) Scanner error: mapping values are not allowed
 ↪  in this context at line 2, column 6
```

Ini galat yang paling licik. Tanpa spasi sesudah titik dua, `number:7` dibaca sebagai teks biasa, bukan pasangan kunci dan nilai. YAML baru kebingungan pada baris **berikutnya**, ketika menemukan pasangan kunci dan nilai sesudah teks biasa. Karena itu, pesannya menunjuk baris 2, padahal kesalahannya ada di baris 1.

Tabel 4.1.: Pesan galat YAML dan penyebabnya

Pesan yang muncul	Penyebab yang paling mungkin
`mapping values are not allowed in this context`	titik dua dalam nilai tanpa tanda kutip, atau tidak ada spasi sesudah titik dua di baris **sebelumnya**
`found character that cannot start any token`	tab di awal baris
`found unexpected end of stream`	tanda kutip tidak ditutup
`did not find expected key`	indentasi kunci atau butir daftar tidak sejajar

> Coba sendiri
>
> Buka `spec/modul-01-contoh.yml` dari Bab 3. Hapus spasi sesudah titik dua pada baris `number: 1`, simpan, lalu jalankan `module_spec_from_yaml("spec/modul-01-contoh.yml")`. Bandingkan pesan yang muncul dengan Tabel 4.1, lalu kembalikan spasinya.

4.7. Ringkasan

YAML menyusun data dengan pasangan `kunci: nilai`, daftar berawalan tanda hubung, dan indentasi dua spasi untuk menyatakan bagian mana termasuk ke mana. Teks panjang ditulis dengan `>` untuk prosa dan `|` untuk kode. Tanda kutip ganda pada teks satu baris mencegah hampir semua salah baca. Bila terjadi galat, pesan YAML menyebut baris dan kolom tempat pembaca YAML kebingungan, dan Tabel 4.1 memetakan pesan itu ke penyebabnya.

Dengan bekal ini, kita mulai menulis spesifikasi Modul 7 di bab berikutnya.

4.8. Uji pemahaman

1. Apa beda `>` dan `|`? Mana yang Anda pakai untuk menulis kode R, dan apa akibatnya bila tertukar?
2. Mengapa baris `title: Modul 7 #revisi` menghasilkan judul `Modul 7`?

3. Sebuah resep data memiliki kolom kategori dengan taraf `[on, off]`. Apa yang terjadi pada kedua taraf itu, dan bagaimana menuliskannya dengan benar?
4. (Transfer) Pesan galat menyebut `line 12, column 5`, tetapi baris 12 tampak benar. Ke mana Anda mencari berikutnya, dan mengapa? Gunakan galat 4 dan galat 5 sebagai petunjuk.

5 Mengisi Identitas Modul dan Mata Kuliah

Dari mana sampul modul mengambil nama mata kuliah dan nama penyusun? Dari bagian paling atas spesifikasi, yaitu identitas modul dan blok `course`. Bagian ini pendek, tetapi muncul di banyak tempat: sampul, kepala setiap halaman, kaki halaman, dan nama berkas PDF.

Mulai bab ini, kita menulis spesifikasi Modul 7 Pemrograman Statistika dari awal. Setelah membaca bab ini, Anda dapat menulis identitas sebuah modul, menjelaskan di mana setiap nilai muncul di PDF, dan memilih `short_code` yang aman.

5.1. Berkas baru untuk Modul 7

Langkah 1 **Buat berkas spesifikasi**

Di panel Files, buka folder `spec/`, klik **New Blank File → Text File**, lalu beri nama `modul-07-deskriptif.yml`.

Nama berkas boleh bebas, tetapi kebiasaan berikut memudahkan pekerjaan: awali dengan `modul-`, lanjutkan dengan **nomor dua digit**, lalu kata kunci topik. Nomor dua digit (`07`, bukan `7`) membuat berkas tersusun berurutan di panel Files dan ketika dirender, karena komputer mengurutkan nama secara abjad: `modul-10` akan mendahului `modul-7`, tetapi tidak mendahului `modul-07`.

Langkah 2 **Tulis identitas modul**

Ketik isi berikut ke berkas tersebut, lalu simpan.

```yaml
number: 7
title: "Statistik Deskriptif dan Eksplorasi Data"
short_code: "deskriptif"
topic: "Statistik Deskriptif & Eksplorasi Data"
pertemuan: 7
sub_cpmk: >
  Sub-CPMK 7: Mahasiswa mampu menjelaskan, menampilkan, dan menerapkan
  statistik deskriptif serta melakukan eksplorasi data secara grafis dengan R.
packages: [dplyr]
course:
  matkul: "Pemrograman Statistika"
  matkul_besar: "PEMROGRAMAN STATISTIKA"
  kode: "K242310"
  semester: "3 (Ganjil)"
```

```
  prodi: "D-IV Statistika"
  penyusun:
    - "Achmad Syahrul Choir"
```

5.2. Kunci tingkat atas dan letaknya di PDF

Kunci yang tidak menjorok disebut **kunci tingkat atas**. Tabel 5.1 menjelaskan masing-masing.

Tabel 5.1.: Kunci tingkat atas spesifikasi

Kunci	Isi	Muncul di
`number`	nomor modul, bilangan bulat 1 sampai 99	sampul, kepala halaman, nomor subbab (7.1, 7.2, ...)
`title`	judul lengkap	sampul (huruf kapital)
`short_code`	kode singkat tanpa spasi	nama folder dan berkas PDF
`topic`	judul ringkas	kepala setiap halaman; bila kosong, memakai `title`
`pertemuan`	pertemuan ke berapa menurut RPS	sampul: *Pertemuan Minggu ke-7*
`sub_cpmk`	capaian pembelajaran modul	sampul, di bawah nomor pertemuan
`packages`	paket R yang dipakai modul	tidak dicetak; catatan untuk Anda

Hanya `number` dan `title` yang wajib. Kunci lain boleh dihilangkan, tetapi sampul akan terlihat kosong tanpa `pertemuan` dan `sub_cpmk`.

`sub_cpmk` ditulis dengan `>` karena teksnya panjang. Ingat dari Bab 4: `>` menggabungkan baris-baris di bawahnya menjadi satu paragraf.

5.3. Blok `course`

Blok `course` berisi identitas mata kuliah. Tabel 5.2 merangkum isinya.

Tabel 5.2.: Isi blok `course`

Kunci	Isi	Muncul di
`matkul`	nama mata kuliah	kepala halaman kanan: *Praktikum Pemrograman Statistika*
`matkul_besar`	nama mata kuliah dalam huruf kapital	pita atas sampul

Kunci	Isi	Muncul di
`kode`	kode mata kuliah	pita atas sampul, dalam kurung
`semester`	semester penyelenggaraan	pita bawah sampul
`prodi`	program studi	sampul dan kaki halaman kanan
`penyusun`	daftar nama penyusun	bagian bawah sampul, dipisah koma
`halaman_awal`	nomor halaman pertama	nomor halaman; diatur otomatis di Bab 12

`penyusun` ditulis sebagai daftar, sehingga Anda dapat mencantumkan lebih dari satu orang. Semua nama akan dicetak di sampul, dipisah koma.

Salah paham umum: "Blok `course` boleh dihilangkan karena sudah diisi saat membuat proyek"

Isian saat membuat proyek hanya dipakai untuk menulis spesifikasi **contoh**. Setiap spesifikasi membaca identitas mata kuliahnya dari blok `course` miliknya sendiri. Bila blok itu tidak ada, paket memakai identitas bawaan, yaitu mata kuliah **Analisis Peubah Ganda**. Berikut sampul spesifikasi Modul 7 yang blok `course`-nya dihapus:

```
#> POLITEKNIK STATISTIKA STIS
#> Program Studi D-IV Komputasi Statistik
#> MODUL PRAKTIKUM • ANALISIS PEUBAH GANDA (K203318)
```

Sampul menyebut mata kuliah yang salah, dan tidak ada pesan galat apa pun. Karena itu, salin blok `course` yang sama ke setiap spesifikasi dalam satu mata kuliah (Bagian 5.6).

5.4. `short_code` dan nama berkas

`short_code` menentukan nama berkas hasil render. Untuk Modul 7 dengan `short_code: "deskriptif"`, berkasnya bernama `Modul_07_deskriptif.pdf` dan `Kunci_07_deskriptif.pdf`.

`short_code` dipakai apa adanya sebagai bagian nama berkas, termasuk oleh XeLaTeX ketika menyusun PDF. Karena itu, tulislah hanya dengan huruf kecil, angka, dan garis bawah. Bila Anda menghilangkan `short_code`, paket membuatnya sendiri dari judul.

Perhatian

`short_code` yang berisi spasi membuat render gagal. Berikut yang terjadi bila Modul 7 diberi `short_code: "statistik deskriptif"`:

```
sp <- module_spec_from_yaml("spasi.yml")
h <- render_module(sp, output_dir = "spasi")
#> Error: Render gagal: PDF tidak dihasilkan untuk modul 7
```

Pesannya hanya menyebut bahwa PDF tidak dihasilkan, tanpa menyebut spasinya. Gantilah spasi dengan garis bawah: `"statistik_deskriptif"`.

5.5. Memeriksa dan merender identitas

Langkah 3 **Periksa dan render Modul 7**

Kali ini kita merender satu modul saja dengan `render_module()`, supaya tidak perlu menunggu spesifikasi lain:

```
library(modulstis)
sp7 <- module_spec_from_yaml("spec/modul-07-deskriptif.yml")
sp7$validate()
#> □ Spesifikasi valid.
h <- render_module(sp7, output_dir = "build")
#> INFO  [07:21:50] render modul mulai modul=7
#> INFO  [07:21:50] Render (bookdown/rmarkdown) modul=7 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:21:56] render modul selesai modul=7 durasi_dtk=5.48
#> INFO  [07:21:56] render kunci mulai modul=7
#> INFO  [07:21:56] Render (bookdown/rmarkdown) modul=7 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:22:01] render kunci selesai modul=7 durasi_dtk=5.51
h$modul$path
#> [1] "build/modul-07/Modul_07_deskriptif.pdf"
```

`render_module()` menaruh hasilnya di folder yang Anda sebut pada `output_dir`, yaitu `build/`. Susunan foldernya sedikit berbeda dari `render_modul.R`: modul di `build/modul-07/`, kunci di `build/kunci-07/`. Keduanya sama-sama benar; Bab 11 membahas kapan masing-masing dipakai.



(a) Sampul

(b) Halaman isi: kepala halaman memakai topic dan matkul

Gambar 5.1.: Modul 7 yang baru berisi identitas. Semua nilai di sampul berasal dari kunci tingkat atas dan blok course.

Hasil yang seharusnya Anda lihat

Halaman isi menampilkan judul setiap bagian diikuti tulisan miring seperti *(Konten bagian 'deskripsi' belum diisi.)*. Itu tanda bahwa identitas sudah benar dan sebelas bagian isi menunggu diisi, yang kita kerjakan di Bab 6.

5.6. Satu blok `course` untuk semua modul

Dalam satu mata kuliah, blok `course` di keempat belas spesifikasi isinya sama. Cara paling aman adalah menyalin blok itu dari spesifikasi yang sudah benar, bukan mengetiknya ulang. Ketika sesuatu berubah, misalnya ada penyusun baru, ubahlah di semua berkas. Fitur **Find in Files** di RStudio (**Edit → Find in Files**, atau **Ctrl+Shift+F**) dapat mencari teks yang sama di seluruh folder `spec/` sekaligus.

Coba sendiri

Tambahkan nama kedua pada `penyusun`:

```
penyusun:
  - "Achmad Syahrul Choir"
  - "Rudi Salam"
```

Render ulang Modul 7 dan lihat bagian bawah sampul. Lalu coba hapus tanda hubung di depan nama kedua, render lagi, dan amati pesan yang muncul.

5.7. Ringkasan

Identitas modul terdiri atas kunci tingkat atas (`number`, `title`, `short_code`, `topic`, `pertemuan`, `sub_cpmk`) dan blok `course` untuk identitas mata kuliah. Nilai-nilai itu dicetak di sampul, kepala dan kaki halaman, serta dipakai untuk nama berkas. Blok `course` harus ada di setiap spesifikasi, karena tanpanya paket diam-diam memakai identitas bawaan. `short_code` hanya boleh berisi huruf kecil, angka, dan garis bawah.

Modul 7 kini punya sampul yang benar, tetapi isinya kosong. Bab 6 mengisinya.

5.8. Uji pemahaman

1. Sebutkan dua kunci yang wajib ada pada setiap spesifikasi.
2. Kunci mana yang menentukan teks di kepala halaman, dan apa yang terjadi bila kunci itu dihilangkan?
3. Mengapa sampul Modul 7 tanpa blok `course` menampilkan *Analisis Peubah Ganda*?
4. (Transfer) Anda mengampu mata kuliah yang sama untuk program studi D-III dan D-IV, dengan isi modul yang hampir sama. Kunci mana saja yang berbeda di antara dua spesifikasi Modul 7 tersebut? Mengapa lebih aman menyimpan keduanya di dua proyek terpisah daripada di satu folder `spec/`? Petunjuk: perhatikan nama folder hasil di `build/`.

6 Sebelas Bagian Isi Modul

Di mana menulis teori, instruksi, dan ringkasan supaya muncul di tempat yang benar? Semua isi modul ditulis di bawah satu kunci, `body`, yang dibagi menurut bagian modul. Paket sudah menyiapkan sebelas bagian baku dengan judul dan urutan yang tetap, sehingga Anda cukup mengisi teksnya.

Setelah membaca bab ini, Anda dapat mengisi bagian-bagian modul dengan paragraf, daftar, dan subjudul, serta menjelaskan bagaimana paket menentukan urutan bagian di PDF.

6.1. Peta sebelas bagian

Setiap modul `modulstis` tersusun atas sebelas bagian. Daftarnya dapat Anda lihat langsung dari paket:

```
library(modulstis)
bagian <- default_sections()
data.frame(
  kunci = sapply(bagian, `[[`, "id"),
  judul_di_modul = sapply(bagian, `[[`, "title")
)
#>        kunci                          judul_di_modul
#> 1  deskripsi                          Deskripsi Modul
#> 2     tujuan                         Tujuan Praktikum
#> 3       alat                           Alat dan Bahan
#> 4  instruksi                    Instruksi Pengerjaan
#> 5      teori                             Teori Singkat
#> 6     contoh               Contoh Praktikum dengan R
#> 7    latihan                           Latihan Mandiri
#> 8  penugasan                                Penugasan
#> 9   tambahan Tambahan: Penyelesaian dengan Fungsi R
#> 10 ringkasan                           Ringkasan Modul
#> 11 referensi                                Referensi
```

Kolom `kunci` adalah nama yang Anda tulis di bawah `body`. Kolom `judul_di_modul` adalah judul yang tercetak. Di PDF, bagian-bagian ini diberi nomor mengikuti nomor modul: 7.1, 7.2, dan seterusnya.

Sembilan bagian berisi teks biasa. Tiga bagian berisi unsur khusus dan dibahas di bab tersendiri:

- `contoh` dan `tambahan` berisi contoh praktikum yang kodenya dijalankan (Bab 7);
- `latihan` dan `penugasan` berisi soal, sedangkan solusinya ditulis di luar `body` (Bab 9).

6.2. Bentuk umum pengisian

Setiap bagian ditulis sebagai **daftar**, dan setiap butir daftar menjadi satu paragraf di modul. Berikut bagian `deskripsi` dan `tujuan` Modul 7:

```
body:
  deskripsi:
    - >
      Modul ini membahas langkah yang selalu mendahului analisis inferensia:
      mengenali data lewat ringkasan angka dan gambar. Statistik deskriptif
      memampatkan ratusan amatan menjadi beberapa ukuran yang bermakna, sedangkan
      eksplorasi grafis menampakkan pola dan keganjilan yang tidak terlihat dari
      angka ringkasan saja.
    - >
      Keterampilan dari Modul 2 (tipe data), Modul 3 (menulis fungsi), dan
      Modul 5 (meringkas per kelompok dengan `dplyr`) dipakai kembali di sini.
  tujuan:
    - "Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:"
    - "1. **Menghitung** ukuran pemusatan, penyebaran, dan posisi."
    - "2. **Memilih** ukuran yang tepat sesuai bentuk sebaran data."
    - "3. **Membuat** dan **membaca** histogram serta diagram kotak-garis."
```

Perhatikan tiga hal:

1. `body:` adalah kunci tingkat atas. Nama bagian (`deskripsi`, `tujuan`) menjorok dua spasi, dan butir-butirnya menjorok empat spasi.
2. Paragraf panjang memakai `- >` supaya baris-barisnya digabung. Paragraf pendek cukup ditulis dalam tanda kutip pada satu baris.
3. Di dalam teks, Anda dapat memakai penanda Markdown: `**tebal**`, `*miring*`, dan `` `kode` `` untuk nama fungsi atau paket.

6.2.1. Daftar bernomor dan daftar berbutir

Bagian `tujuan` pada contoh di atas menulis setiap tujuan sebagai butir tersendiri yang diawali `1.`, `2.`, dan seterusnya. Karena setiap butir YAML menjadi satu paragraf, butir yang diawali angka dan titik akan tersusun sebagai daftar bernomor di modul. Hal yang sama berlaku untuk tanda hubung: butir yang diawali `-` menjadi daftar berbutir, seperti pada bagian `alat`:

```
  alat:
    - "Perangkat dan bahan yang diperlukan:"
    - "- **R** dan **RStudio**."
    - "- Paket `dplyr`."
```

```
    - "- Berkas data `data/data_rumahtangga.csv`."
```

Perhatikan bahwa tanda hubung pertama di setiap baris adalah penanda butir **YAML**, dan tanda hubung kedua, di dalam tanda kutip, adalah penanda daftar **Markdown**.

6.2.2. Subjudul di dalam bagian

Bagian `teori` biasanya panjang dan perlu dibagi. Tulis butir yang diawali `###` untuk membuat subjudul:

```
  teori:
    - "### Pemusatan, Penyebaran, dan Posisi"
    - >
      **Pemusatan** menjawab "di mana pusat data": rata-rata, median, dan modus.
      **Penyebaran** menjawab "seberapa bervariasi": jangkauan, simpangan kuartil,
      ragam, dan simpangan baku. **Posisi** menjawab "di mana letak suatu
      amatan": kuartil dan persentil.
    - "### Memilih Ukuran yang Tepat"
    - >
      Rata-rata peka terhadap nilai ekstrem, sedangkan median tidak. Pada data
      yang menceng ke kanan, seperti pengeluaran rumah tangga, median biasanya
      lebih mewakili keadaan umum.
```

Di modul, kedua subjudul itu menjadi subbab bernomor 7.5.1 dan 7.5.2.

6.3. Bagian penutup

Bagian `ringkasan` dan `referensi` ditulis dengan cara yang sama:

```
  ringkasan:
    - >
      Statistik deskriptif dan eksplorasi grafis saling melengkapi. Angka
      ringkasan memberi ukuran yang ringkas, gambar menampakkan bentuk dan
      keganjilan. Modul 9 melanjutkan eksplorasi grafis dengan `ggplot2`.
  referensi:
    - "Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data
↪   Science* (2nd ed.). O'Reilly."
    - "Walpole, R. E., Myers, R. H., Myers, S. L., & Ye, K. (2012). *Probability and
↪   Statistics for Engineers and Scientists* (9th ed.). Pearson."
```

Daftar pustaka yang tercetak di modul diambil dari `body` → `referensi`. Kunci tingkat atas `references`, yang Anda lihat di spesifikasi contoh pada Bab 3, saat ini hanya disimpan dan tidak dicetak.

6.4. Merender dan melihat hasilnya

Langkah 1 **Lengkapi bagian isi dan render**

Salin seluruh blok `body` di atas ke `spec/modul-07-deskriptif.yml`, di bawah blok `course`, lalu render:

```
sp7 <- module_spec_from_yaml("spec/modul-07-deskriptif.yml")
sp7$validate()
#> □ Spesifikasi valid.
h <- render_module(sp7, output_dir = "build")
#> INFO  [07:22:03] render modul mulai modul=7
#> INFO  [07:22:03] Render (bookdown/rmarkdown) modul=7 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:22:09] render modul selesai modul=7 durasi_dtk=5.33
#> INFO  [07:22:09] render kunci mulai modul=7
#> INFO  [07:22:09] Render (bookdown/rmarkdown) modul=7 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:22:14] render kunci selesai modul=7 durasi_dtk=5.45
```



(a) Deskripsi, tujuan, dan alat (b) Teori dengan dua subjudul

Gambar 6.1.: Modul 7 setelah bagian isi diisi.

> Hasil yang seharusnya Anda lihat
>
> Bagian 7.1 sampai 7.5 kini berisi teks Anda. Daftar tujuan tercetak bernomor, daftar alat berbutir, dan teori terbagi menjadi dua subbab. Bagian Contoh, Latihan, dan Penugasan masih menampilkan tulisan *belum diisi*; ketiganya diisi pada bab-bab berikutnya.

6.5. Bagian yang boleh dikosongkan

Bagian yang tidak Anda isi tetap tercetak dengan judulnya dan tulisan *(Konten bagian … belum diisi.)*. Tulisan itu sengaja dibuat mencolok supaya bagian yang terlupa mudah terlihat ketika Anda memeriksa PDF.

Satu-satunya pengecualian adalah bagian `tambahan`. Bila tidak diisi, bagian ini dilewati sepenuhnya dan tidak muncul di daftar isi. Karena itu, Modul 7 kita saat ini hanya memiliki sepuluh bagian bernomor, 7.1 sampai 7.10. Bagian ini disediakan untuk materi pengayaan, misalnya penyelesaian ulang contoh dengan fungsi siap pakai.

6.6. Mengganti judul bagian Contoh dan Tambahan

Judul bagian Contoh bawaannya *Contoh Praktikum dengan R*. Bila modul Anda memakai R dan Python, judul itu dapat diganti dengan kunci tingkat atas `contoh_title`:

```
contoh_title: "Contoh Praktikum dengan R dan Python"
```

Dengan cara yang sama, `tambahan_title` mengganti judul bagian Tambahan. Judul sembilan bagian lainnya tetap.

Salah paham umum: "Urutan bagian di YAML menentukan urutan di PDF"

Di berkas Modul 7, Anda boleh menulis `ringkasan` sebelum `deskripsi`; PDF-nya tetap dimulai dari Deskripsi Modul. Paket menyusun bagian menurut urutan tetap dari `default_sections()`, bukan menurut urutan penulisan di `body`. Kunci di bawah `body` hanya menunjukkan bagian mana yang diisi. Konsekuensinya, salah ketik nama bagian, misalnya `deskirpsi`, tidak menimbulkan galat: teksnya tidak tercetak di mana pun, dan bagian Deskripsi tetap bertuliskan *belum diisi*.

Coba sendiri

Ubah nama kunci `ringkasan` menjadi `rangkuman`, render ulang, lalu buka bagian Ringkasan Modul. Apa yang tercetak? Kembalikan namanya menjadi `ringkasan`.

6.7. Ringkasan

Isi modul ditulis di bawah `body`, dibagi menurut sebelas bagian baku. Setiap bagian adalah daftar YAML, dan setiap butirnya menjadi satu paragraf. Penanda Markdown dipakai untuk huruf tebal, kode, daftar, dan subjudul. Urutan dan judul bagian ditetapkan paket; Anda hanya mengisi teksnya, dan bagian yang terlupa terlihat jelas di PDF.

Bagian yang paling penting dalam modul praktikum, yaitu contoh berkode, kita isi di bab berikutnya.

6.8. Uji pemahaman

1. Apa akibatnya bila Anda tidak mengisi bagian `instruksi`? Apa bedanya dengan tidak mengisi bagian `tambahan`?
2. Mengapa setiap tujuan pada bagian `tujuan` ditulis sebagai butir YAML tersendiri?
3. Di bagian `alat`, baris `- "- **R** dan **RStudio**."` memuat dua tanda hubung. Jelaskan peran masing-masing.
4. (Transfer) Seorang rekan mengeluh bahwa teks Ringkasan yang sudah ia tulis tidak muncul di PDF, padahal render berhasil tanpa galat. Sebutkan dua kemungkinan penyebabnya berdasarkan bab ini dan Bab 4.

7 Contoh Praktikum yang Kodenya Benar-benar Berjalan

Bagaimana memastikan keluaran kode di modul sama dengan yang akan dilihat mahasiswa? Caranya adalah tidak menyalin keluaran sama sekali. Di `modulstis`, Anda menulis kodenya saja; paket menjalankan kode itu ketika modul dibuat dan mencetak keluarannya. Bila data atau versi paket berubah, render berikutnya mencetak keluaran yang baru.

Setelah membaca bab ini, Anda dapat menulis contoh praktikum lengkap dengan kode, penjelasan, dan interpretasi, mengendalikan kode mana yang dijalankan, dan menulis contoh dalam dua bahasa, R dan Python.

7.1. Struktur satu contoh

Satu contoh praktikum di modul `modulstis` selalu tersusun atas empat unsur, dengan urutan yang tetap:

1. **judul contoh**, misalnya *Contoh 7.1 Ringkasan cepat seluruh peubah*;
2. **kode** beserta keluarannya, dengan judul kode, misalnya *Kode 7.1 str dan summary*;
3. kotak **Penjelasan Kode**, yang menerangkan apa yang dikerjakan kode baris demi baris;
4. kotak **Interpretasi Output**, yang menafsirkan keluaran dalam konteks persoalan.

Urutan ini disengaja. Mahasiswa membaca kodenya, melihat keluarannya, memahami cara kerjanya, baru kemudian diajak menafsirkan hasilnya. Penjelasan dan interpretasi dipisah karena menjawab dua pertanyaan berbeda: *apa yang dilakukan kode ini* dan *apa arti hasilnya*.

7.2. Menulis contoh pertama

Contoh ditulis di bawah `body` → `contoh`, sebagai daftar. Setiap butir daftar adalah satu contoh. Berikut dua contoh pertama Modul 7:

```yaml
contoh:
  - contoh: "Ringkasan cepat seluruh peubah"
    judul_r: "str dan summary"
    kode_r: |
      rt <- read.csv("data/data_rumahtangga.csv")
      str(rt)
      summary(rt[, c("pengeluaran", "pendapatan", "art")])
    eval: true
```

```
    penjelasan: >
      Data dibaca dengan `read.csv()`. Fungsi `str()` menampilkan struktur
      data: banyaknya baris, nama kolom, dan tipe setiap kolom. Fungsi
      `summary()` diberi tiga kolom numerik saja.
    interpretasi: >
      Bandingkan rata-rata (Mean) dan median setiap peubah. Bila keduanya
      berbeda cukup jauh, sebarannya kemungkinan menceng dan perlu diperiksa
      dengan histogram.
  - contoh: "Ukuran pemusatan termasuk modus"
    judul_r: "mean, median, dan modus buatan"
    kode_r: |
      peng <- rt$pengeluaran
      mean(peng)
      median(peng)
      # R tidak punya fungsi modus bawaan; buat sendiri (Modul 3)
      modus <- function(x) {
        tab <- table(x)
        as.numeric(names(tab)[which.max(tab)])
      }
      modus(rt$art)
    eval: true
    penjelasan: >
      Fungsi `modus()` menghitung frekuensi setiap nilai dengan `table()`,
      lalu mengambil nilai yang frekuensinya terbesar.
    interpretasi: >
      Modus paling bermakna untuk peubah diskret seperti banyaknya anggota
      rumah tangga (`art`).
```

Tabel 7.1.: Kunci pada satu contoh

Kunci	Isi
`contoh`	judul contoh
`judul_r`	judul kotak kode R
`kode_r`	kode R, ditulis dengan `|`
`eval`	`true` bila kode dijalankan; lihat Bagian 7.4
`penjelasan`	isi kotak Penjelasan Kode
`interpretasi`	isi kotak Interpretasi Output
`narasi`	(opsional) paragraf pengantar sebelum kode

Di dalam `penjelasan` dan `interpretasi`, Anda boleh memakai `` `kode` ``, `**tebal**`, dan rumus di antara tanda dolar seperti `$\bar{x}$`. Tanda lain ditulis apa adanya.

7.3. Menulis kode R di dalam YAML

Kode ditulis dengan penanda `|`, yang mempertahankan pergantian baris (Bab 4). Ada tiga hal yang perlu diingat.

Indentasi kode dihitung dari baris pertama. Semua baris kode menjorok enam spasi di bawah `kode_r: |`. YAML membuang enam spasi itu, sehingga R menerima kode yang rata kiri. Indentasi tambahan di dalam kode, misalnya isi fungsi `modus()`, tetap terjaga.

Contoh-contoh dalam satu modul berbagi satu sesi R. Objek `rt` dibuat di Contoh 7.1 dan dipakai lagi di Contoh 7.2 tanpa dibaca ulang. Ini mencerminkan cara mahasiswa bekerja di RStudio. Akibatnya, urutan contoh penting: contoh yang memakai `rt` harus berada sesudah contoh yang membuatnya.

Jalur data ditulis relatif terhadap folder modul. Sebelum menjalankan kode, paket menaruh berkas data di subfolder `data/` di samping modul. Karena itu, kode cukup menulis `read.csv("data/data_rumahtangga.csv")`, persis seperti yang akan ditulis mahasiswa. Blok `datasets` di spesifikasi, yang menghasilkan berkas itu, dibahas di Bab 8. Untuk sekarang, salin saja blok berikut ke spesifikasi Anda, di antara blok `course` dan `body`:

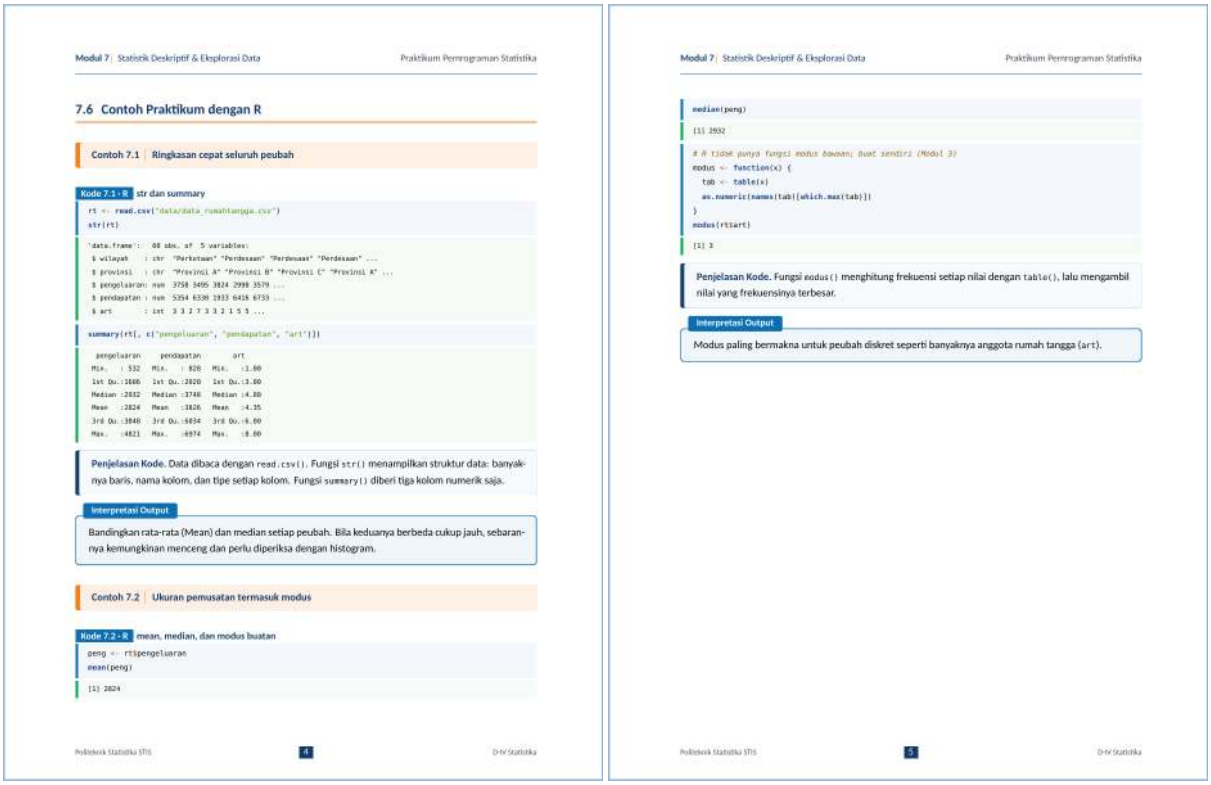
```
datasets:
  - name: data_rumahtangga
    role: contoh
    recipe:
      kind: tabular
      "n": 60
      seed: 2025
      columns:
        wilayah: {type: category, levels: [Perkotaan, Perdesaan], probs: [0.45,
↪  0.55]}
        provinsi: {type: category, levels: [Provinsi A, Provinsi B, Provinsi C],
↪  probs: [0.4, 0.35, 0.25]}
        pengeluaran: {type: numeric, min: 500, max: 5000, digits: 1}
        pendapatan: {type: numeric, min: 800, max: 7000, digits: 1}
        art: {type: integer, min: 1, max: 8}
```

Langkah 1 **Tambahkan dua contoh dan render**

Tambahkan blok `datasets` dan kedua contoh di atas ke `spec/modul-07-deskriptif.yml`, lalu render:

```
library(modulstis)
sp7 <- module_spec_from_yaml("spec/modul-07-deskriptif.yml")
sp7$validate()
#> ✓ Spesifikasi valid.
h <- render_module(sp7, output_dir = "build")
```

```
#> INFO  [07:22:16] Data dibangkitkan modul=7 dataset=1
#> INFO  [07:22:16] render modul mulai modul=7
#> INFO  [07:22:16] Render (bookdown/rmarkdown) modul=7 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:22:22] render modul selesai modul=7 durasi_dtk=5.81
#> INFO  [07:22:22] render kunci mulai modul=7
#> INFO  [07:22:22] Render (bookdown/rmarkdown) modul=7 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:22:27] render kunci selesai modul=7 durasi_dtk=5.24
```



(a) Contoh 7.1 (b) Contoh 7.2

Gambar 7.1.: Dua contoh praktikum Modul 7. Keluaran kode di bawah setiap blok dihasilkan R ketika modul dirender.

Hasil yang seharusnya Anda lihat

Bagian 7.6 *Contoh Praktikum dengan R* dimulai di halaman baru. Setiap contoh berawal dengan pita judul oranye, diikuti kotak kode bernomor, keluaran R, kotak Penjelasan Kode, dan kotak Interpretasi Output. Keluaran `str(rt)` menyebut `60 obs. of 5 variables`, sesuai resep data.

7.4. Mengendalikan eksekusi dengan eval

Kunci `eval` menentukan apakah kode dijalankan:

- `eval: true`: kode dijalankan, dan keluarannya dicetak di bawah kode.
- `eval: false` atau `eval` tidak ditulis: kode hanya ditampilkan, tanpa keluaran.

Gunakan `eval: false` untuk kode yang memang tidak boleh atau tidak dapat dijalankan saat modul dibuat, misalnya kode yang memasang paket, membaca berkas yang hanya ada di komputer mahasiswa, atau kode yang sengaja salah untuk didiskusikan.

> Salah paham umum: "Kode di bagian Contoh selalu dijalankan"
>
> Kode hanya dijalankan bila Anda menulis `eval: true`. Bila kunci `eval` lupa ditulis, paket menganggapnya `false`: kode tercetak rapi, tetapi tanpa keluaran, dan tidak ada pesan apa pun. Modul tetap tampak benar sampai mahasiswa bertanya mengapa contohnya tidak menunjukkan hasil. Biasakan menulis `eval: true` di setiap contoh, dan periksa bahwa setiap kotak kode di PDF diikuti keluaran.

Ketika `eval: true` dan kodenya menimbulkan galat, render berhenti dan PDF modul tidak dibuat. Pesannya menyebut penyebab galat dan letak baris di berkas `.Rmd` antara, misalnya `Quitting from lines 95-100` diikuti `Render gagal: PDF tidak dihasilkan untuk modul 7. Penyebab: ...`. Ada pula kesalahan yang tidak menimbulkan galat: salah ketik nama kolom pada `summary(rt$pengluaran)` tidak menghentikan R, tetapi keluarannya tidak bermakna. Karena itu, setelah mengubah kode, bacalah keluaran di PDF, bukan hanya memastikan render berhasil. Bab 14 membahas cara menemukan kedua jenis kesalahan ini.

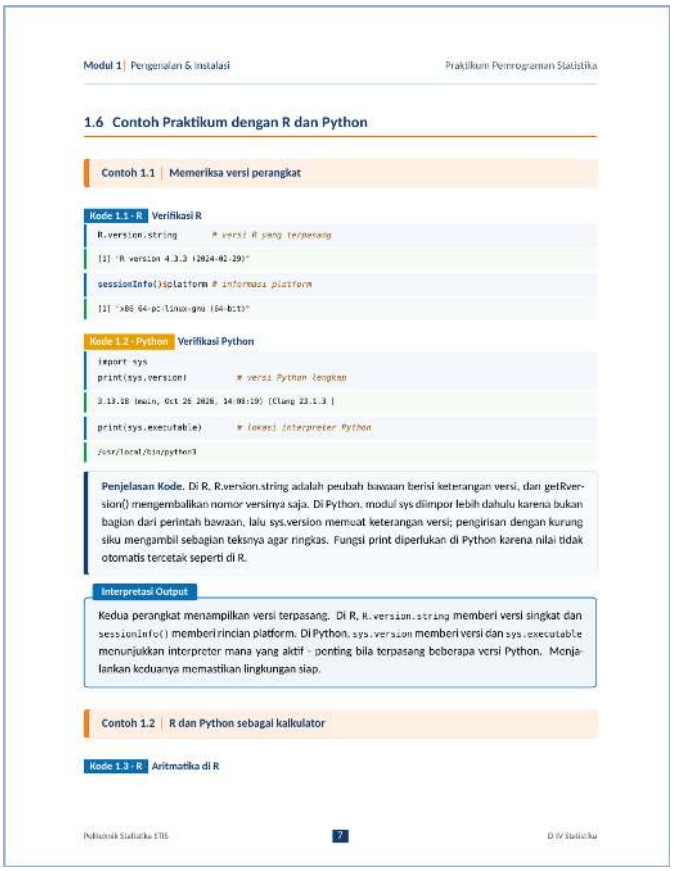
7.5. Contoh dwibahasa: R dan Python

Mata kuliah yang mengajarkan R dan Python dapat menampilkan kedua bahasa dalam satu contoh. Tambahkan `kode_py` dan `judul_py` di samping `kode_r` dan `judul_r`. Kode R dicetak lebih dahulu, lalu kode Python, masing-masing dengan keluarannya. Berikut potongan dari Modul 1 Pemrograman Statistika yang disertakan di paket:

```
    - contoh: "Memeriksa versi perangkat"
      kode_r: "R.version.string       # versi R yang
↪ terpasang\nsessionInfo()$platform # informasi platform"
      judul_r: "Verifikasi R"
      kode_py: "import sys\nprint(sys.version)          # versi Python
↪ lengkap\nprint(sys.executable)       # lokasi interpreter Python"
      judul_py: "Verifikasi Python"
      eval: true
```

Kode Python dijalankan melalui paket R `reticulate`, sehingga komputer yang merender memerlukan

Python dan paket `reticulate`. Paket Python yang dipakai contoh, misalnya `pandas`, juga harus terpasang di Python tersebut.



Gambar 7.2.: Contoh dwibahasa pada Modul 1 Pemrograman Statistika: kode R dan kode Python dalam satu contoh, masing-masing dengan keluarannya.

Jangan lupa mengganti judul bagian Contoh dengan `contoh_title: "Contoh Praktikum dengan R dan Python"` (Bab 6).

7.6. Menyisipkan kotak di antara contoh

Butir pada daftar `contoh` tidak harus berupa contoh. Butir yang berupa teks biasa dicetak apa adanya di antara contoh. Ini berguna untuk menyisipkan catatan atau peringatan, misalnya sebelum contoh yang memakai data baru:

```
contoh:
  - contoh: "Ringkasan cepat seluruh peubah"
    ...
  - "Contoh berikut memakai objek `rt` dari Contoh 7.1."
  - contoh: "Ukuran pemusatan termasuk modus"
    ...
```

Kotak berwarna, seperti kotak peringatan atau kotak tips, ditulis dengan perintah LaTeX pendek. Caranya dibahas di Bab 10.

> **Coba sendiri**
>
> Hapus baris `eval: true` dari Contoh 7.2, render ulang, lalu bandingkan halaman Contoh 7.2 dengan Gambar 7.1. Setelah itu, kembalikan baris tersebut.

7.7. Ringkasan

Satu contoh praktikum terdiri atas judul, kode, penjelasan kode, dan interpretasi keluaran. Kode ditulis dengan `|` dan dijalankan ketika modul dibuat, asalkan contoh itu memuat `eval: true`. Semua contoh dalam satu modul berbagi satu sesi R dan membaca data dari subfolder `data/`. Contoh dwibahasa memakai `kode_py` dan `judul_py`, dan memerlukan Python serta `reticulate` di komputer yang merender.

Contoh-contoh tadi membaca berkas data yang belum kita bahas asal-usulnya. Bab 8 menjelaskannya.

7.8. Uji pemahaman

1. Sebutkan empat unsur satu contoh praktikum sesuai urutannya di modul.
2. Apa yang terjadi bila sebuah contoh tidak memuat kunci `eval`? Bagaimana Anda mengenalinya di PDF?
3. Contoh 7.2 memakai objek `rt` tanpa membacanya. Apa yang terjadi bila Contoh 7.2 dipindah ke urutan pertama?
4. (Transfer) Anda ingin menampilkan kode `install.packages("dplyr")` sebagai contoh, tetapi tidak ingin paket itu benar-benar dipasang setiap kali modul dibuat. Bagaimana Anda menuliskannya, dan bagaimana mahasiswa membedakan kode itu dari contoh lain di PDF?

8 Menyiapkan Data Latihan: Simulasi atau Data Nyata

Bagaimana memberi setiap modul data latihan tanpa mengirim berkas terpisah kepada mahasiswa, dan tanpa khawatir data di modul berbeda dari data di kunci jawaban? Di `modulstis`, data ditulis sebagai **resep** di dalam spesifikasi. Setiap kali modul dirender, paket membuat berkas CSV dari resep itu dan menaruhnya di samping modul. Bila Anda memerlukan data nyata, resep dapat menunjuk berkas data yang Anda sediakan.

Setelah membaca bab ini, Anda dapat menulis resep data simulasi, memilih jenis resep yang sesuai, memakai berkas data nyata, dan menjelaskan mengapa data simulasi tidak berubah dari render ke render.

8.1. Untuk apa data simulasi

Data simulasi adalah data yang dibangkitkan komputer dengan aturan yang Anda tentukan. Untuk modul praktikum, data simulasi punya tiga kelebihan dibanding data nyata:

- **Tidak ada masalah izin.** Data simulasi bebas dibagikan. Data mikro resmi umumnya tidak.
- **Ukurannya dapat diatur.** Enam puluh baris cukup untuk mempelajari `summary()` dan mudah diperiksa dengan mata.
- **Tidak berubah.** Data dibangkitkan dari bilangan acak awal yang tetap, sehingga angka di modul, kunci, dan komputer mahasiswa selalu sama.

Kelemahannya juga perlu disadari: data simulasi tidak membawa pola nyata. Bagian Bagian 8.3.1 membahas akibatnya.

8.2. Anatomi blok `datasets`

Blok `datasets` adalah kunci tingkat atas. Berikut blok Modul 7, yang di bab ini mendapat satu kolom baru, `umur_krt`:

```yaml
datasets:
  - name: data_rumahtangga
    role: contoh
    recipe:
      kind: tabular
      "n": 60
```

```
    seed: 2025
    columns:
      wilayah: {type: category, levels: [Perkotaan, Perdesaan], probs: [0.45,
↪ 0.55]}
      provinsi: {type: category, levels: [Provinsi A, Provinsi B, Provinsi C],
↪ probs: [0.4, 0.35, 0.25]}
      pengeluaran: {type: numeric, min: 500, max: 5000, digits: 1}
      pendapatan: {type: numeric, min: 800, max: 7000, digits: 1}
      art: {type: integer, min: 1, max: 8}
      umur_krt: {type: integer, min: 25, max: 70}
```

Tabel 8.1.: Kunci pada satu data

Kunci	Isi
`name`	nama data; menjadi nama berkas `data/<name>.csv`
`role`	keterangan peran data, misalnya `contoh` atau `latihan`; tidak memengaruhi hasil
`recipe` → `kind`	jenis resep: `tabular`, `mvn`, `grouped_mvn`, atau `file`
`recipe` → `"n"`	banyaknya baris
`recipe` → `seed`	bilangan acak awal; lihat Bagian 8.6
`recipe` → `columns`	daftar kolom beserta jenis dan rentangnya (untuk `tabular`)

Satu modul boleh memiliki lebih dari satu data. Tambahkan butir baru di bawah `datasets`, dengan `name` yang berbeda.

8.3. Resep `tabular` dan jenis kolomnya

Resep `tabular` cocok untuk sebagian besar modul pemrograman dan statistika dasar: setiap kolom ditentukan sendiri-sendiri. Empat jenis kolom tersedia.

Tabel 8.2.: Jenis kolom pada resep `tabular`

`type`	Kunci tambahan	Nilai yang dihasilkan
`numeric`	`min`, `max`, `digits`	bilangan desimal, tersebar **merata** antara `min` dan `max`, dibulatkan `digits` angka
`integer`	`min`, `max`	bilangan bulat, tersebar merata antara `min` dan `max`

`type`	Kunci tambahan	Nilai yang dihasilkan
`category`	`levels`, `probs`	kategori dipilih acak; `probs` adalah peluang tiap kategori
`balanced`	`levels`, `each`	kategori berulang teratur, untuk rancangan percobaan

Kolom `category` tanpa `probs` memberi peluang yang sama kepada setiap kategori. Bila `probs` ditulis, banyaknya angka harus sama dengan banyaknya `levels`, dan jumlahnya sebaiknya 1.

Anda dapat melihat data yang akan dihasilkan tanpa merender modul, dengan `generate_module_data()`:

```
library(modulstis)
sp7 <- module_spec_from_yaml("spec/modul-07-deskriptif.yml")
d <- generate_module_data(sp7)$data_rumahtangga
str(d)
#> 'data.frame':    60 obs. of  6 variables:
#>  $ wilayah    : Factor w/ 2 levels "Perkotaan","Perdesaan": 1 2 2 2 1 2 1 2 1 1
 ↪  ...
#>  $ provinsi   : Factor w/ 3 levels "Provinsi A","Provinsi B",..: 1 2 3 1 1 3 2 1
 ↪  1 1 ...
#>  $ pengeluaran: num  3758 3495 3824 2990 3579 ...
#>  $ pendapatan : num  5354 6330 1933 6416 6733 ...
#>  $ art        : int  3 3 2 7 3 3 2 1 5 5 ...
#>  $ umur_krt   : int  39 39 45 31 41 53 30 29 44 28 ...
table(d$wilayah)
#>
#> Perkotaan Perdesaan
#>        27        33
```

Perhatikan bahwa `wilayah` dan `provinsi` terbaca sebagai `Factor`, sedangkan di berkas CSV keduanya tersimpan sebagai teks biasa. Ketika mahasiswa membaca CSV dengan `read.csv()`, kolom itu menjadi `chr`, seperti yang terlihat pada keluaran Contoh 7.1.

8.3.1. Kolom numerik tersebar merata

Kolom `numeric` dan `integer` dibangkitkan dari sebaran **seragam** (*uniform*): setiap nilai di antara `min` dan `max` sama mungkinnya. Akibatnya, histogram kolom tersebut cenderung datar.



Gambar 8.1.: Contoh 7.3 yang ditambahkan pada bab ini. Histogram pengeluaran dari resep tabular tampak datar, tidak menceng seperti pengeluaran rumah tangga yang sebenarnya.

> Salah paham umum: "Data simulasi meniru bentuk data nyata"
>
> Resep `tabular` hanya mengatur rentang setiap kolom, bukan bentuk sebarannya, dan tidak membangun hubungan antarkolom. Pengeluaran dan pendapatan pada data Modul 7 tidak berkorelasi, dan sebaran pengeluaran tidak menceng ke kanan. Bila sebuah contoh ingin menunjukkan data menceng atau korelasi, pakailah resep `mvn` (Bagian 8.4) atau data nyata (Bagian 8.5), dan sesuaikan kalimat interpretasi dengan data yang benar-benar dihasilkan. Selalu cantumkan kotak *Tentang data* yang menyatakan bahwa datanya simulasi.

Langkah 1 Tambahkan kolom dan contoh grafik

Tambahkan kolom `umur_krt` ke resep, dan tambahkan contoh ketiga berikut di bawah dua contoh sebelumnya:

```
  - contoh: "Histogram dan diagram kotak-garis"
    judul_r: "hist dan boxplot"
    kode_r: |
      hist(rt$pengeluaran, breaks = 8, col = "#0B6FB4", border = "white",
           main = "Pengeluaran rumah tangga", xlab = "ribu rupiah per bulan")
```

```
        boxplot(pengeluaran ~ wilayah, data = rt, col = "#6FB3E0",
                ylab = "ribu rupiah per bulan")
    eval: true
    penjelasan: >
      `hist()` membagi rentang nilai menjadi selang (`breaks`) dan menghitung
      banyaknya amatan di setiap selang. `boxplot()` dengan rumus
      `pengeluaran ~ wilayah` menggambar satu kotak untuk setiap wilayah.
    interpretasi: >
      Histogram menampakkan bentuk sebaran pengeluaran, sedangkan diagram
      kotak-garis membandingkan median dan sebaran kedua wilayah secara
      berdampingan.
```

Grafik yang dibuat kode contoh otomatis tercetak di modul, tepat di bawah kodenya.

8.4. Resep `mvn` dan `grouped_mvn`

Resep `mvn` membangkitkan beberapa peubah numerik dari sebaran **normal ganda**. Anda menentukan rata-rata setiap peubah (`mu`) dan matriks ragam-peragam (`Sigma`), sehingga hubungan antarpeubah dapat diatur.

```
datasets:
  - name: nilai_ujian
    role: contoh
    recipe:
      kind: mvn
      "n": 80
      seed: 2026
      vars: [uts, uas]
      mu: [70, 72]
      Sigma: [[64, 40], [40, 81]]
```

`Sigma` ditulis sebagai daftar baris: baris pertama `[64, 40]` dan baris kedua `[40, 81]`. Angka di diagonal adalah ragam setiap peubah (simpangan baku 8 dan 9), dan angka di luar diagonal adalah peragamnya. Resep ini menyatakan korelasi populasi $40/(8\times 9) \approx 0{,}56$. Korelasi pada 80 amatan yang dibangkitkan akan sedikit berbeda dari angka itu, karena datanya adalah sampel acak:

```
resep <- list(kind = "mvn", n = 80, seed = 2026, vars = c("uts", "uas"),
              mu = c(70, 72), Sigma = list(c(64, 40), c(40, 81)))
nilai <- generate_dataset(resep)
round(cor(nilai$uts, nilai$uas), 2)
#> [1] 0.47
```

Resep `grouped_mvn` membangkitkan data normal ganda untuk beberapa kelompok sekaligus, dengan rata-rata yang berbeda per kelompok. Resep ini cocok untuk modul perbandingan kelompok:

```
    recipe:
      kind: grouped_mvn
      seed: 2026
      vars: [uts, uas]
      groups: {Kelas_A: 30, Kelas_B: 30}
      means: {Kelas_A: [70, 72], Kelas_B: [75, 78]}
      Sigma: [[64, 40], [40, 81]]
```

`groups` menyebut nama dan banyaknya amatan setiap kelompok. Hasilnya memiliki kolom tambahan `kelompok`.

8.5. Memakai data nyata dengan `kind: file`

Bila modul memerlukan data nyata, simpan berkasnya di proyek, lalu tunjuk berkas itu dengan `kind: file`. Paket tidak membangkitkan apa pun; ia membaca berkas Anda dan menyalinnya ke folder `data/` di samping modul.

Sebagai contoh, kita pakai data `airquality` bawaan R, yaitu pengukuran kualitas udara di New York pada 1973, dan menyimpannya di folder `data-asli/` proyek:

```
dir.create("data-asli", showWarnings = FALSE)
write.csv(airquality, "data-asli/kualitas_udara.csv", row.names = FALSE)
```

Resepnya:

```
datasets:
  - name: udara
    role: contoh
    recipe:
      kind: file
      path: "../data-asli/kualitas_udara.csv"
      columns: [Ozone, Temp, Month]
      rename: {Ozone: ozon, Temp: suhu, Month: bulan}
```

Tabel 8.3.: Kunci pada resep `file`

Kunci	Isi
`path`	letak berkas data, **relatif terhadap letak berkas spesifikasi**

Kunci	Isi
`columns`	(opsional) kolom yang diambil
`rename`	(opsional) nama baru kolom, dengan bentuk `lama: baru`
`"n"`	(opsional) ambil hanya sekian baris pertama
`na`	(opsional) teks yang dibaca sebagai data hilang; bawaannya `NA` dan sel kosong

Berkas spesifikasi ada di `spec/`, sedangkan data ada di `data-asli/`. Karena itu `path` ditulis `../data-asli/...`: `..` berarti naik satu folder dari `spec/`. Berkas berakhiran `.csv`, `.tsv`, `.txt`, dan `.rds` dapat dibaca.

```
sp_nyata <- module_spec_from_yaml("spec/uji-data-nyata.yml")
sp_nyata$validate()
#> □ Spesifikasi valid.
head(generate_module_data(sp_nyata)$udara, 4)
#>   ozon suhu bulan
#> 1   41   67     5
#> 2   36   72     5
#> 3   12   74     5
#> 4   18   62     5
```

`validate()` memeriksa apakah berkas di `path` benar-benar ada. Bila jalurnya salah, pesannya menyebut nama data dan jalur yang dicari, sehingga kesalahan ketik terlihat sebelum render.

> **Perhatian**
>
> Data nyata ikut tersalin ke setiap folder hasil render dan ikut terbagi ketika modul dibagikan. Pastikan Anda berhak membagikan data tersebut kepada mahasiswa.

8.6. Peran seed dan mengapa data tidak berubah

Komputer tidak membangkitkan bilangan yang benar-benar acak. Ia menghitung barisan bilangan yang tampak acak dari satu bilangan awal, yaitu `seed`. Dengan `seed` yang sama, barisan yang dihasilkan selalu sama. Itulah sebabnya data Modul 7 identik pada setiap render, di modul, di kunci, dan di komputer mahasiswa:

```
d1 <- generate_module_data(sp7)$data_rumahtangga
d2 <- generate_module_data(sp7)$data_rumahtangga
identical(d1, d2)
#> [1] TRUE
```

Mengganti `seed` menghasilkan data yang berbeda. Gunakan `seed` yang berbeda untuk setiap modul, misalnya tahun ditambah nomor modul, supaya data antarmodul tidak kembar.

Coba sendiri

Ubah `seed: 2025` pada Modul 7 menjadi `seed: 2032`, lalu jalankan `generate_module_data(sp7)$data_rumahtangga |> head(3)` sebelum dan sesudah perubahan. Ingat untuk membaca ulang spesifikasinya dengan `module_spec_from_yaml()` setelah menyimpan berkas.

Perhatian

Bila Anda mengubah resep atau `seed` di tengah semester, semua angka di contoh, latihan, dan kunci ikut berubah. Modul dan kunci tetap cocok satu sama lain karena dirender ulang bersama, tetapi modul yang sudah dibagikan kepada mahasiswa tidak lagi cocok dengan modul baru.

8.7. Di mana berkas data berada

Setelah render, berkas data ada di subfolder `data/` di samping setiap PDF:

```
h <- render_module(sp7, output_dir = "build")
#> INFO  [07:23:04] Data dibangkitkan modul=7 dataset=1
#> INFO  [07:23:04] render modul mulai modul=7
#> INFO  [07:23:04] Render (bookdown/rmarkdown) modul=7 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:23:10] render modul selesai modul=7 durasi_dtk=6.19
#> INFO  [07:23:10] render kunci mulai modul=7
#> INFO  [07:23:10] Render (bookdown/rmarkdown) modul=7 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:23:15] render kunci selesai modul=7 durasi_dtk=5.23
list.files("build", pattern = "\\.csv$", recursive = TRUE)
#> [1] "kunci-07/data/data_rumahtangga.csv" "modul-07/data/data_rumahtangga.csv"
```

Berkas di `build/modul-07/data/` adalah yang dibagikan kepada mahasiswa bersama modulnya. Berkas di folder kunci isinya sama; berkas itu ada supaya kode solusi di kunci jawaban dapat dijalankan.

8.8. Ringkasan

Data latihan ditulis sebagai resep di blok `datasets`. Resep `tabular` mengatur setiap kolom sendiri-sendiri, dengan kolom numerik yang tersebar merata. Resep `mvn` dan `grouped_mvn` membangun peubah numerik yang saling berkorelasi. Resep `file` memakai berkas data nyata yang jalurnya dihitung dari letak spesifikasi. Karena setiap resep memakai `seed` yang tetap, data yang sama dihasilkan pada setiap render, sehingga modul dan kunci jawaban selalu memakai angka yang sama.

Data dan contoh sudah siap. Bab 9 menambahkan latihan untuk mahasiswa beserta kunci jawabannya.

8.9. Uji pemahaman

1. Apa yang menentukan nama berkas CSV yang dihasilkan sebuah resep?
2. Mengapa histogram kolom `pengeluaran` pada Modul 7 tampak datar? Resep apa yang Anda pilih bila ingin menunjukkan dua peubah yang berkorelasi?
3. Spesifikasi Anda ada di `spec/`, dan data nyata di `spec/data/survei.csv`. Bagaimana menulis `path`-nya?
4. (Transfer) Di tengah semester, seorang rekan mengganti `seed` Modul 5 karena ingin data yang “lebih bagus”. Apa akibatnya bagi mahasiswa yang sudah mengerjakan Modul 5, dan apa saran Anda?

9 Latihan, Penugasan, dan Kunci Jawaban

Bagaimana kunci jawaban tetap cocok dengan soal walaupun soalnya direvisi? Di `modulstis`, soal dan solusinya ditulis di berkas yang sama dan dirender bersama. Anda mengubah keduanya di satu tempat, dan solusinya dijalankan ulang terhadap data yang sama dengan data mahasiswa. Bab ini menunjukkan cara menulis keduanya, dan satu hal yang tetap menjadi tanggung jawab Anda.

Setelah membaca bab ini, Anda dapat menulis latihan dan penugasan, menulis solusi berkode beserta skornya, dan memeriksa kunci jawaban hasil render.

9.1. Soal di `body`, solusi di `exercises`

Soal dan solusi ditulis di dua tempat yang berbeda dalam spesifikasi yang sama:

- **soal** untuk mahasiswa ditulis di `body` → `latihan` dan `body` → `penugasan`, dan tercetak di **modul**;
- **solusi** ditulis di kunci tingkat atas `exercises`, dan tercetak hanya di **kunci jawaban**.

Pemisahan ini membuat modul tidak pernah memuat jawaban, sementara kunci jawaban selalu dibuat dari berkas yang sama dengan modulnya.

9.2. Latihan

Latihan ditulis sebagai daftar. Butir berupa teks dicetak sebagai paragraf. Butir yang memiliki kunci `judul` dan `isi` menjadi satu soal bernomor:

````
latihan:
  - "Kerjakan latihan berikut dengan berkas `data/data_rumahtangga.csv`."
  - judul: "Ukuran pemusatan dan penyebaran"
    isi: |
      Lengkapi kode berikut untuk peubah `pendapatan`.

      ```{r eval=FALSE}
      rt <- read.csv("data/data_rumahtangga.csv")
      ____(rt$pendapatan)   # rata-rata
      ____(rt$pendapatan)   # median
      ____(rt$pendapatan)   # simpangan baku
      ```
  - judul: "Membandingkan wilayah"
    isi: >
      Hitung median `pengeluaran` untuk setiap `wilayah` dengan `dplyr`.
````

```
        Wilayah mana yang median pengeluarannya lebih tinggi?
```

Paket memberi nomor otomatis: soal pertama menjadi *Latihan 7.1*, soal kedua *Latihan 7.2*, dan seterusnya. Anda tidak perlu menulis nomornya.

Perhatikan soal pertama. Isinya memuat blok kode dengan garis kosong `____` yang harus diisi mahasiswa. Blok itu ditulis dengan pembuka ```` ```{r eval=FALSE} ````, sehingga kode ditampilkan tetapi tidak dijalankan. Tanpa `eval=FALSE`, R akan mencoba menjalankan `____(rt$pendapatan)` dan render gagal.

> **Latihan berjenjang**
>
> Untuk latihan yang bertingkat, sisipkan subjudul sebagai butir teks, misalnya `"#### Tingkat Dasar (melengkapi kode)"` sebelum soal-soal dasar dan `"#### Tingkat Menengah (menulis sendiri)"` sebelum soal berikutnya. Spesifikasi Modul 7 bawaan paket memakai pola ini dengan tiga tingkat.

9.3. Penugasan

Penugasan ditulis dengan cara yang sama, dan setiap butir ber-`judul` diberi nomor *Tugas 7.1*, *Tugas 7.2*, dan seterusnya:

```yaml
  penugasan:
    - judul: "Profil deskriptif rumah tangga"
      isi: >
        Susun laporan R Markdown berisi ringkasan angka dan grafik untuk
        `pengeluaran` dan `pendapatan`, dipisahkan menurut `provinsi`. Tutup
        dengan ringkasan paling banyak 200 kata.
```

Bagian Penugasan selalu dimulai di halaman baru, sehingga mudah dicetak terpisah bila diperlukan.

9.4. Solusi, skor, dan interpretasi

Solusi ditulis di `exercises`, kunci tingkat atas di bagian akhir spesifikasi:

```yaml
exercises:
  - id: "L7.1"
    points: 40
    solution: "mean, median, dan sd."
    judul_r: "Solusi Latihan 7.1"
    solusi_r: |
      rt <- read.csv("data/data_rumahtangga.csv")
      mean(rt$pendapatan)
      median(rt$pendapatan)
```

```
    sd(rt$pendapatan)
  interpretasi: >
    Tiga fungsi ringkasan dasar dipakai sesuai perannya. Nilai yang dihasilkan
    harus sama dengan keluaran di atas karena datanya dibangkitkan dengan seed
    yang sama.
- id: "L7.2"
  points: 60
  solution: "group_by(wilayah) lalu summarise(median(pengeluaran))."
  judul_r: "Solusi Latihan 7.2"
  solusi_r: |
    suppressPackageStartupMessages(library(dplyr))
    rt <- read.csv("data/data_rumahtangga.csv")
    rt |>
      group_by(wilayah) |>
      summarise(median_pengeluaran = median(pengeluaran))
  interpretasi: >
    Bandingkan kedua median. Karena datanya simulasi, selisih antarwilayah
    tidak mencerminkan keadaan nyata.
```

Tabel 9.1.: Kunci pada satu solusi

Kunci	Isi	Wajib?
`id`	label soal, misalnya `L7.1`	dianjurkan
`points`	skor soal	ya
`solution`	ringkasan jawaban dalam satu kalimat	dianjurkan
`judul_r`, `solusi_r`	judul dan kode solusi R	tidak
`judul_py`, `solusi_py`	judul dan kode solusi Python	tidak
`eval`	`false` bila kode solusi tidak boleh dijalankan	tidak
`interpretasi`	pembahasan untuk pengajar	dianjurkan

`validate()` memeriksa bahwa setiap solusi memiliki `points`. Solusi tanpa skor menghasilkan pesan `exercises[2].points hilang`, misalnya untuk solusi kedua.

Perhatikan perbedaan bawaan `eval`

Kode **contoh** hanya dijalankan bila ditulis `eval: true` (Bagian 7.4). Kode **solusi** sebaliknya: selalu dijalankan, kecuali ditulis `eval: false`. Bawaan yang berbeda ini disengaja, karena kunci jawaban yang kodenya tidak dijalankan tidak dapat memastikan jawabannya benar.

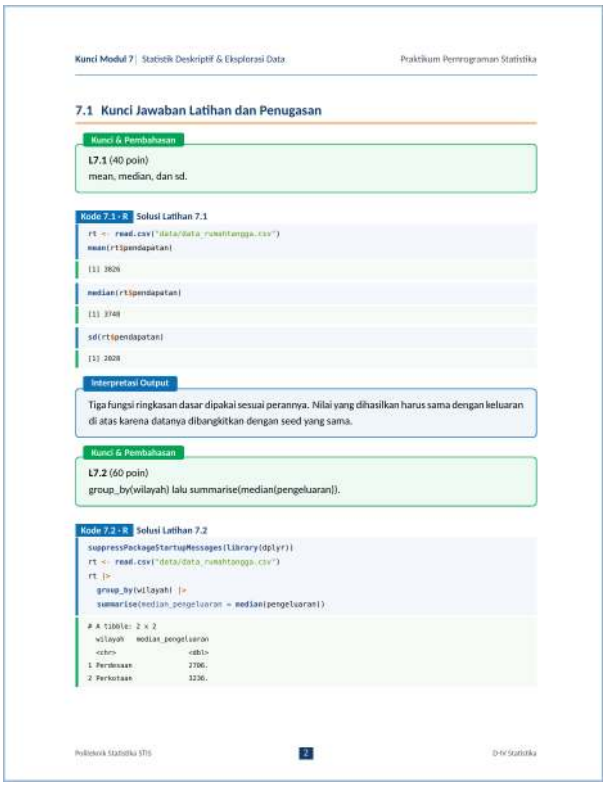
9.5. Merender dan memeriksa kunci jawaban

Langkah 1 **Tambahkan latihan, penugasan, dan solusi, lalu render**

```
library(modulstis)
sp7 <- module_spec_from_yaml("spec/modul-07-deskriptif.yml")
sp7$validate()
#> ✔ Spesifikasi valid.
h <- render_module(sp7, output_dir = "build")
#> INFO  [07:23:17] Data dibangkitkan modul=7 dataset=1
#> INFO  [07:23:17] render modul mulai modul=7
#> INFO  [07:23:17] Render (bookdown/rmarkdown) modul=7 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:23:23] render modul selesai modul=7 durasi_dtk=6.28
#> INFO  [07:23:23] render kunci mulai modul=7
#> INFO  [07:23:23] Render (bookdown/rmarkdown) modul=7 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:23:29] render kunci selesai modul=7 durasi_dtk=5.7
h$kunci$path
#> [1] "build/kunci-07/Kunci_07_deskriptif.pdf"
```



(a) Latihan Mandiri di modul



(b) Kunci jawaban: ringkasan, kode solusi, keluaran, dan pembahasan

Gambar 9.1.: Soal di modul dan solusinya di kunci jawaban, keduanya dari spesifikasi yang sama.

Kunci jawaban Modul 7 kini 4 halaman. Susunannya:

1. sampul hijau berlabel **KUNCI JAWABAN**;
2. untuk setiap solusi: kotak berisi `id`, skor, dan `solution`; kode solusi beserta keluarannya; lalu kotak interpretasi;
3. bagian **Catatan Penskoran untuk Pengajar**, yang ditambahkan otomatis.

Langkah 2 Periksa kunci jawaban

Bacalah kunci jawaban dengan pertanyaan berikut:

- Apakah keluaran kode solusi masuk akal? Untuk Latihan 7.1, rata-rata pendapatan harus berada di antara `min` dan `max` resep, yaitu 800 dan 7000.
- Apakah setiap soal di modul memiliki solusi di kunci, dengan urutan yang sama?
- Apakah skor seluruh soal berjumlah sesuai rencana penilaian Anda? Pada contoh ini, 40 + 60 = 100.

> Salah paham umum: “Paket mencocokkan setiap soal dengan solusinya”
>
> Paket tidak menghubungkan *Latihan 7.2* di modul dengan butir kedua `exercises` secara otomatis. Keduanya ditulis terpisah, dan hubungannya hanya lewat kebiasaan Anda: `id: "L7.2"` dan urutan yang sama. Bila Anda menyisipkan soal baru di tengah latihan, nomor soal di modul bergeser

dengan sendirinya, tetapi `id` di `exercises` tidak. Yang dijamin paket adalah hal lain: data dan kode solusi selalu dijalankan ulang terhadap data yang sama dengan data mahasiswa. Karena itu, setiap kali menambah atau menghapus soal, periksa ulang `id` dan urutan solusinya.

Coba sendiri

Hapus baris `points: 60` dari solusi kedua, lalu jalankan `sp7$validate()` setelah membaca ulang spesifikasinya. Baca pesannya, kemudian kembalikan baris itu.

9.6. Kunci berupa prosa panjang

Sebagian penugasan dijawab dengan uraian, bukan kode. Untuk kunci seperti itu, tulislah uraiannya di `body` → `kunci`. Butir-butir di sana dicetak apa adanya di kunci jawaban, sesudah semua solusi `exercises`, dan tidak pernah muncul di modul:

```yaml
body:
  kunci:
    - "#### Pembahasan Tugas 7.1"
    - >
      Laporan yang baik membandingkan median pengeluaran antarprovinsi, bukan hanya
      rata-ratanya, dan menyebut bahwa datanya simulasi.
```

9.7. Ringkasan

Soal ditulis di `body` → `latihan` dan `body` → `penugasan`, diberi nomor otomatis, dan tercetak di modul. Solusi ditulis di `exercises`, dengan `points` yang wajib, dan tercetak di kunci jawaban. Kode solusi dijalankan secara bawaan terhadap data yang sama dengan data mahasiswa. Hubungan antara nomor soal dan solusinya dijaga lewat `id` dan urutan, sehingga perlu diperiksa setiap kali soal ditambah atau dihapus.

Modul 7 kini lengkap dari sampul sampai kunci. Bab 10 merapikan tampilannya dengan kotak dan rumus.

9.8. Uji pemahaman

1. Di mana soal ditulis, dan di mana solusinya ditulis? Mengapa keduanya dipisah?
2. Mengapa blok kode berisi garis kosong `____` harus ditulis dengan `eval=FALSE`?
3. Apa beda bawaan `eval` pada kode contoh dan kode solusi?
4. (Transfer) Anda menyisipkan soal baru di antara Latihan 7.1 dan Latihan 7.2. Sebutkan dua hal yang harus Anda ubah di `exercises`, dan jelaskan mengapa paket tidak mengubahnya sendiri.

10 Kotak, Rumus, Tabel, dan Gambar

Mengapa tanda persen atau garis bawah bisa merusak sebuah kotak di modul? Karena di dalam kotak Anda menulis LaTeX secara langsung, dan bagi LaTeX kedua tanda itu punya arti khusus. Bab ini membahas sedikit LaTeX yang diperlukan untuk modul: kapan memakainya, kotak apa saja yang tersedia, cara menulis rumus dan tabel, dan karakter mana yang harus diloloskan.

Setelah membaca bab ini, Anda dapat menyisipkan kotak berwarna dan rumus ke Modul 7, serta mengenali dan memperbaiki galat LaTeX yang paling sering.

10.1. Kapan Markdown cukup dan kapan perlu LaTeX

Sampai bab ini, semua isi Modul 7 ditulis dengan Markdown: `**tebal**`, `*miring*`, `` `kode` ``, daftar, dan subjudul `###`. Markdown juga sanggup menulis rumus sebaris di antara dua tanda dolar, misalnya `$\bar{x}$`, dan rumus terpisah di antara dua pasang tanda dolar, `$$s^2 = \frac{1}{n-1}\sum (x_i - \bar{x})^2$$`. Untuk paragraf biasa, Markdown cukup.

LaTeX baru diperlukan untuk dua hal: **kotak berwarna**, yang tidak dapat dibuat dengan Markdown, dan rumus yang perlu disusun khusus, misalnya beberapa rumus sejajar di dalam satu kotak.

10.2. Menulis LaTeX di dalam spesifikasi

Kotak ditulis sebagai satu butir pada bagian mana pun di `body`, dengan bentuk `\begin{nama-kotak}` … `\end{nama-kotak}`. Cara paling aman menuliskannya adalah dengan penanda `|`, karena teks di bawahnya disalin apa adanya, termasuk garis miring terbalik:

```
- |
  \begin{kotaksalah}[Rata-rata pada data menceng]
  Melaporkan rata-rata saja untuk peubah yang menceng dapat menyesatkan.
  Bandingkan dahulu rata-rata dengan median; bila keduanya berbeda jauh,
  sajikan median beserta ukuran penyebarannya.
  \end{kotaksalah}
```

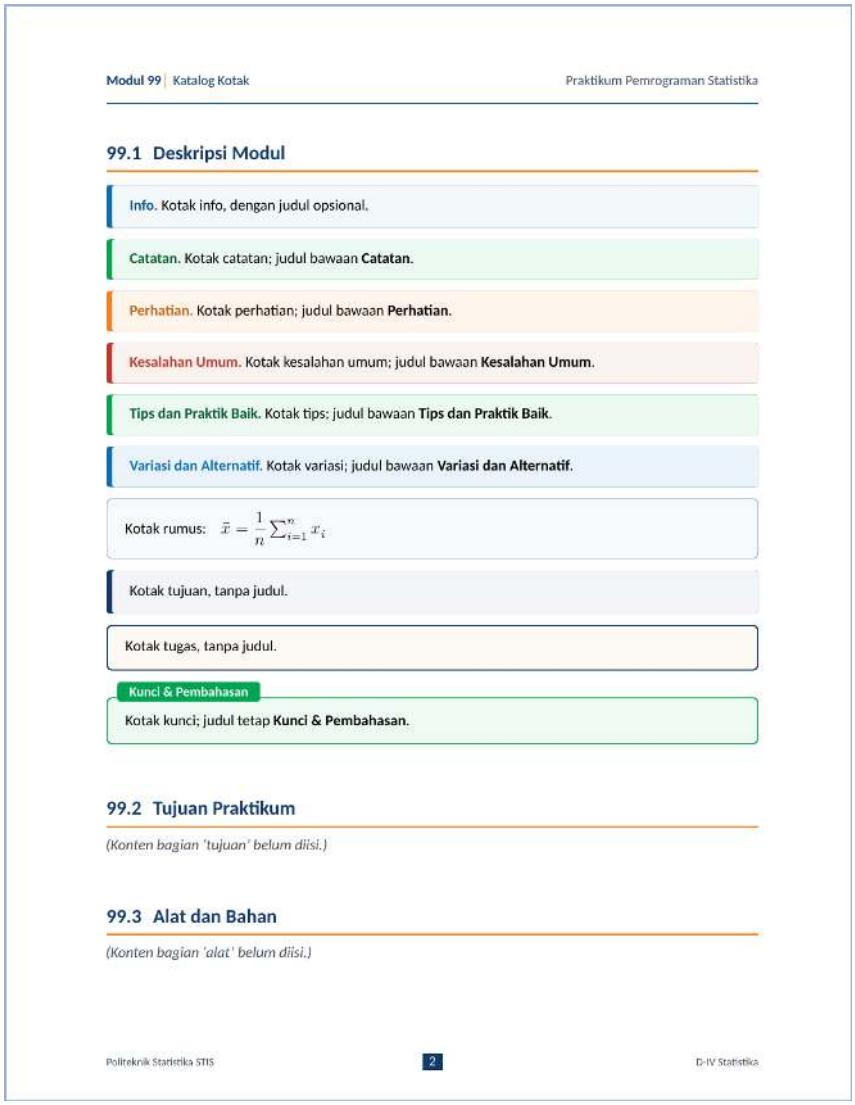
Garis miring terbalik di dalam tanda kutip

Spesifikasi bawaan paket menulis kotak di dalam tanda kutip ganda, seperti `"\\begin{kotakperhatian}[Tentang data]\n...\n\\end{kotakperhatian}"`. Di dalam tanda kutip ganda, YAML memperlakukan `\` sebagai awalan karakter khusus: `\n` berarti ganti baris, dan `\\` berarti satu garis miring terbalik. Karena itu, setiap perintah LaTeX harus ditulis dengan **dua** garis miring. Dengan penanda `|`, cukup satu garis miring, dan ganti baris ditulis sebagai ganti

baris biasa. Kedua cara menghasilkan modul yang sama; pilih | bila Anda menulis kotak sendiri.

10.3. Katalog kotak bawaan

Paket menyediakan sepuluh kotak untuk isi modul. Gambar 10.1 memperlihatkan tampilannya.



Gambar 10.1.: Sepuluh kotak bawaan, dirender dari spesifikasi peraga contoh/katalog-kotak.yml.

Tabel 10.1.: Kotak yang dapat dipakai di spesifikasi

Kotak	Judul bawaan	Judul dapat diganti?	Untuk
`kotakinfo`	tanpa judul	ya	keterangan umum, tabel ringkas
`kotakcatatan`	Catatan	ya	catatan sampingan

Kotak	Judul bawaan	Judul dapat diganti?	Untuk
`kotakperhatian`	Perhatian	ya	hal yang mudah keliru; kotak *Tentang data*
`kotaksalah`	Kesalahan Umum	ya	miskonsepsi beserta koreksinya
`kotaktips`	Tips dan Praktik Baik	ya	kebiasaan kerja yang baik
`kotakvariasi`	Variasi dan Alternatif	ya	cara lain menyelesaikan contoh
`kotakrumus`	tanpa judul	tidak	rumus pokok
`kotaktujuan`	tanpa judul	tidak	sasaran belajar yang ditonjolkan
`kotaktugas`	tanpa judul	tidak	konteks penugasan
`kotakkunci`	Kunci & Pembahasan	tidak	pembahasan di kunci jawaban

Judul diganti dengan menuliskannya dalam kurung siku sesudah nama kotak: `\begin{kotakperhatian}[Tentang data]`. Selain sepuluh kotak itu, ada tiga kotak yang dipakai paket sendiri dan tidak perlu Anda tulis: `kotakpenjelasan` dan `kotakinterpretasi` untuk contoh, serta `kotakistilah` untuk glosarium (Bab 13).

Kotak *Tentang data*

Setiap modul yang memakai data simulasi sebaiknya memuat kotak perhatian yang menyatakannya. Spesifikasi bawaan paket meletakkannya sebagai paragraf terakhir bagian `deskripsi`:

```
    - |
      \begin{kotakperhatian}[Tentang data]
      Berkas data pada modul ini adalah \textbf{data simulasi}, bukan data
↪ nyata
      dan bukan data resmi instansi mana pun.
      \end{kotakperhatian}
```

10.4. Rumus matematika

Di dalam LaTeX, rumus selalu diapit tanda dolar: `$...$` untuk rumus sebaris. Beberapa perintah yang paling sering dipakai di modul statistika:

Tabel 10.2.: Perintah rumus yang sering dipakai

Ditulis	Tercetak	Ditulis	Tercetak
`\bar{x}`	$\bar{x}$	`\hat{\beta}`	$\hat{\beta}$
`x_i`	x_i	`s^{2}`	s^2
`\sum_{i=1}^{n}`	$\sum_{i=1}^{n}$	`\sqrt{s^{2}}`	$\sqrt{s^2}$
`\dfrac{a}{b}`	$\dfrac{a}{b}$	`\mu, \sigma`	μ, σ
`\le, \ge, \ne`	$\le, \ge, \neq$	`\times`	$\times$

Untuk menyusun beberapa rumus di dalam satu kotak, akhiri setiap baris dengan `\\` dan beri jarak dengan `[5pt]`, seperti kotak rumus Modul 7:

```
    - |
      \begin{kotakrumus}
      \textbf{Rumus pokok.}\\[3pt]
      Ragam contoh:\quad $s^{2} = \dfrac{1}{n-1}\sum_{i=1}^{n}(x_i -
↪  \bar{x})^{2}$\\[5pt]
      Simpangan kuartil:\quad $IQR = Q_3 - Q_1$
      \end{kotakrumus}
```

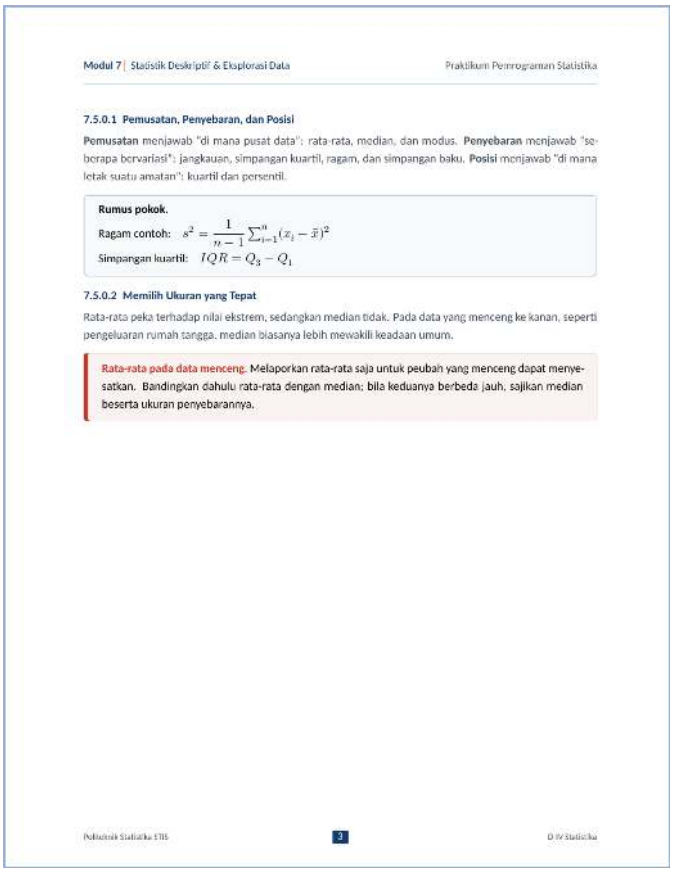
`\quad` memberi jarak mendatar, dan `\textbf{...}` menebalkan teks.

Langkah 1 **Tambahkan kotak rumus dan kotak salah paham ke Modul 7**

Tambahkan kedua kotak di atas ke bagian `teori` Modul 7: kotak rumus sesudah paragraf tentang pemusatan, dan kotak kesalahan umum sesudah paragraf tentang memilih ukuran. Lalu render:

```
library(modulstis)
sp7 <- module_spec_from_yaml("spec/modul-07-deskriptif.yml")
h <- render_module(sp7, output_dir = "build")
#> INFO  [07:23:42] Data dibangkitkan modul=7 dataset=1
#> INFO  [07:23:42] render modul mulai modul=7
#> INFO  [07:23:42] Render (bookdown/rmarkdown) modul=7 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:23:48] render modul selesai modul=7 durasi_dtk=5.99
#> INFO  [07:23:48] render kunci mulai modul=7
#> INFO  [07:23:48] Render (bookdown/rmarkdown) modul=7 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
↪  bailing
```

```
#> Warning:  out. on input line 1156.
#> INFO  [07:23:54] render kunci selesai modul=7 durasi_dtk=5.6
```



Gambar 10.2.: Bagian Teori Singkat Modul 7 dengan kotak rumus dan kotak kesalahan umum.

Hasil yang seharusnya Anda lihat

Log render hanya berisi baris `INFO` dan peringatan `tocloft` yang biasa. Tidak ada baris `WARN`. Di PDF, kotak rumus berbingkai biru tua tipis, dan kotak kesalahan umum berwarna merah muda dengan garis merah di kiri.

10.5. Tabel

Tabel sederhana ditulis dengan tabel Markdown di dalam butir `|`. Baris kedua, yang berisi tanda hubung, memisahkan judul kolom dari isinya:

```
- |
  | Awalan | Menghasilkan | Contoh |
  |:--|:--|:--|
  | `d` | kepekatan atau peluang | `dnorm(0)` |
```

```
| `p` | peluang kumulatif | `pnorm(1.96)` |
```

Tabel yang berasal dari data sebaiknya dibuat oleh kode di sebuah contoh, misalnya dengan `knitr::kable()`, supaya isinya ikut berubah bila datanya berubah.

10.6. Gambar

Gambar di modul paling baik dibuat oleh kode di sebuah contoh, seperti histogram Contoh 7.3. Gambar itu otomatis tercetak di bawah kodenya, dan ikut diperbarui setiap render. Bila Anda perlu menyisipkan gambar yang sudah jadi, misalnya diagram alur, simpan berkasnya di proyek dan tampilkan dengan `knitr::include_graphics()` di sebuah contoh. Jalur berkas ditulis lengkap, karena kode contoh dijalankan dari folder hasil render, bukan dari folder `spec/`.

10.7. Karakter yang harus diloloskan

Di dalam kotak, lima karakter berikut punya arti khusus bagi LaTeX. Untuk mencetaknya sebagai karakter biasa, beri garis miring terbalik di depannya. Cara ini disebut **meloloskan** (*escape*).

Tabel 10.3.: Karakter khusus LaTeX

Karakter	Arti bagi LaTeX	Tulis
_	subskrip, hanya boleh di dalam rumus	_
%	awal komentar; sisa baris dibuang	\%
&	pemisah kolom tabel	\&
#	parameter perintah	\#
$	awal dan akhir rumus	\$

Di luar kotak, yaitu pada paragraf Markdown biasa, Anda **tidak** perlu meloloskan apa pun; paket dan Markdown melakukannya untuk Anda. Aturan ini hanya berlaku di dalam `\begin{...}` … `\end{...}`.

> Salah paham umum: “Bila render berhasil, kotaknya pasti benar”
>
> Galat LaTeX di dalam kotak tidak selalu menghentikan render. Paket mencoba memulihkannya, dan PDF tetap terbentuk, tetapi isinya rusak di sekitar galat. Berikut yang terjadi bila sebuah kotak memuat `data_rumahtangga.csv` dan `50%` tanpa diloloskan:

```
h <- render_module(module_spec_from_yaml("kotak-rusak.yml"), output_dir =
↪  "rusak")
#> WARN  [07:23:57] render tunggal gagal; memulihkan via 2x xelatex modul=7
↪  pesan=LaTeX failed to compile
↪  /home/claude/gh/modulstis/pedoman/_kerja/bab10/rusak/modul-
↪  07/Modul_07_deskriptif.tex. See https://yihui.org/tinytex/r/#debugging for
↪  debugging tips. See Modul_07_deskriptif.log for more info.
#> WARN  [07:23:59] LaTeX melaporkan galat; isi PDF di sekitar baris ini rusak
↪  modul=7 galat=Missing $ inserted. baris=l.448 Data disimpan di data_
↪  jumlah=2 log=Modul_07_deskriptif.log
```

Baris pertama `WARN` menandakan bahwa render normal gagal dan paket beralih ke jalur pemulihan. Baris kedua menyebut galat LaTeX yang pertama, yaitu `Missing $ inserted`, dan letaknya, yang diawali `l.` dan nomor baris di berkas `.tex` antara. Potongan teks sesudah nomor baris itu berhenti tepat di garis bawah yang tidak diloloskan. Di PDF, teks sesudah garis bawah tercetak miring seperti rumus, dan teks sesudah `%` hilang. Karena itu, **setiap baris `WARN` harus ditindaklanjuti**, walaupun PDF sudah terbentuk.

Coba sendiri

Perbaiki kotak rusak di atas menjadi `data\_rumahtangga.csv` dan `50\%`, render ulang, dan pastikan tidak ada lagi baris `WARN`.

10.8. Ringkasan

Markdown cukup untuk paragraf, daftar, kode, rumus sebaris, dan tabel sederhana. LaTeX diperlukan untuk kotak berwarna dan susunan rumus khusus. Kotak ditulis sebagai butir `|` dengan `\begin{...}` dan `\end{...}`, memakai sepuluh kotak bawaan, dan judulnya dapat diganti dengan kurung siku. Di dalam kotak, lima karakter khusus harus diloloskan. Galat LaTeX di dalam kotak dapat menghasilkan PDF yang rusak tanpa menghentikan render, dan tandanya adalah baris `WARN` di log render.

Modul 7 kini lengkap. Bagian III membahas cara menerbitkan seluruh modul satu mata kuliah.

10.9. Uji pemahaman

1. Sebutkan dua hal yang memerlukan LaTeX dan tidak dapat ditulis dengan Markdown di modul.
2. Mengapa perintah LaTeX di spesifikasi bawaan paket ditulis dengan dua garis miring terbalik, sedangkan di buku ini dengan satu?
3. Tuliskan kotak perhatian berjudul *Tentang data* yang menyatakan bahwa 25% data adalah data hilang. Karakter mana yang harus diloloskan?
4. (Transfer) Render sebuah modul berhasil dan PDF terbentuk, tetapi log memuat baris `WARN` yang menyebut `Misplaced alignment tab character &`. Apa yang Anda cari di spesifikasi, dan di

bagian mana kemungkinan besar letaknya?

Bagian III

Menerbitkan

11 Merender: Satu Modul, Banyak Modul, Dua Backend

Bagaimana merender empat belas modul tanpa membuka empat belas berkas? Paket menyediakan tiga cara, dari yang paling sederhana sampai yang paling lengkap. Bab ini membahas ketiganya, kapan memakai masing-masing, dan cara memilih backend bookdown atau Quarto.

Setelah membaca bab ini, Anda dapat merender satu modul atau seluruh mata kuliah, membaca log render, dan berpindah backend.

11.1. Menyiapkan beberapa spesifikasi

Sampai bab lalu, proyek kita berisi spesifikasi contoh dan Modul 7. Untuk berlatih merender banyak modul, kita hapus spesifikasi contoh, lalu salin dua spesifikasi Pemrograman Statistika yang disertakan paket: Modul 3 dan Modul 5.

Langkah 1 **Salin dua spesifikasi bawaan**

```
library(modulstis)
file.remove("spec/modul-01-contoh.yml")
#> Warning: cannot remove file 'spec/modul-01-contoh.yml', reason 'No such file or
↪  directory'
#> [1] FALSE
bawaan <- system.file("specs_ps", package = "modulstis")
file.copy(file.path(bawaan, c("modul-03-pemrograman-r.yml", "modul-05-dplyr.yml")),
          "spec")
#> [1] TRUE TRUE
list.files("spec", pattern = "\\.yml$")
#> [1] "modul-03-pemrograman-r.yml" "modul-05-dplyr.yml"
#> [3] "modul-07-deskriptif.yml"
```

Seluruh empat belas spesifikasi bawaan dapat Anda lihat dengan `list.files(system.file("specs_ps", package = "modulstis"))`. Spesifikasi itu adalah titik awal yang baik untuk mata kuliah Anda sendiri: salin, lalu sunting.

11.2. Tiga cara merender

Tabel 11.1.: Tiga cara merender

Cara	Merender	Hasil di	Cocok untuk
`render_module()`	satu spesifikasi	`build/modul-NN/`, `build/kunci-NN/`	menyunting satu modul
`ModuleProject`	semua spesifikasi di satu folder	`build/modul-NN/`, `build/kunci-NN/`	membangun seluruh mata kuliah dari R
`render_modul.R`	semua, atau rentang tertentu	`modul/ModulNN_kode/`	membangun dari terminal

Ketiganya menghasilkan PDF yang sama. Yang berbeda adalah cakupannya dan susunan folder hasilnya.

11.2.1. Satu modul dengan `render_module()`

Cara ini sudah Anda pakai sejak Bab 5. Selama menyunting satu modul, inilah cara tercepat, karena hanya modul itu yang dirender:

```
sp5 <- module_spec_from_yaml("spec/modul-05-dplyr.yml")
h5 <- render_module(sp5, output_dir = "build")
#> INFO  [07:24:06] Data dibangkitkan modul=5 dataset=1
#> INFO  [07:24:07] render modul mulai modul=5
#> INFO  [07:24:07] Render (bookdown/rmarkdown) modul=5 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:24:18] render modul selesai modul=5 durasi_dtk=11.76
#> INFO  [07:24:18] render kunci mulai modul=5
#> INFO  [07:24:18] Render (bookdown/rmarkdown) modul=5 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:24:25] render kunci selesai modul=5 durasi_dtk=6.75
c(h5$modul$path, h5$kunci$path)
#> [1] "build/modul-05/Modul_05_dplyr.pdf" "build/kunci-05/Kunci_05_dplyr.pdf"
```

11.2.2. Seluruh folder dengan `ModuleProject`

`ModuleProject` membaca semua berkas `.yml` di satu folder, lalu memeriksa, membangkitkan data, dan merender semuanya:

Langkah 2 **Render seluruh mata kuliah**

```
proj <- ModuleProject$new(specs_dir = "spec", backend = "bookdown",
                          output_dir = "build")
#> INFO  [07:24:26] Muat spesifikasi jumlah=3
proj$validate_all()
#> [1] TRUE
hasil <- proj$render_all()
#> INFO  [07:24:26] render modul mulai modul=3
#> INFO  [07:24:26] Render (bookdown/rmarkdown) modul=3 jenis=modul
#>
#> Quitting from lines 364-369 [unnamed-chunk-9] (Modul_03_pemrograman_r.Rmd)
#> INFO  [07:24:26] render modul mulai modul=5
#> INFO  [07:24:26] Render (bookdown/rmarkdown) modul=5 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:24:38] render modul selesai modul=5 durasi_dtk=12.21
#> INFO  [07:24:38] render kunci mulai modul=5
#> INFO  [07:24:38] Render (bookdown/rmarkdown) modul=5 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:24:45] render kunci selesai modul=5 durasi_dtk=6.68
#> INFO  [07:24:45] render modul mulai modul=7
#> INFO  [07:24:45] Render (bookdown/rmarkdown) modul=7 jenis=modul
#>
#> Quitting from lines 109-113 [unnamed-chunk-1] (Modul_07_deskriptif.Rmd)
#> INFO  [07:24:45] OK modul=5
```

`validate_all()` memeriksa semua spesifikasi sekaligus dan mengembalikan `TRUE` hanya bila semuanya valid. Bila ada yang tidak valid, log menyebut nomor modul dan pesannya, sehingga Anda dapat memperbaiki semuanya sebelum mulai merender.

`render_all()` merender modul dan kunci untuk setiap spesifikasi, lalu menutup dengan satu baris `OK` atau `GAGAL` per modul. Baris itulah yang perlu Anda baca pertama kali.

Hasil yang seharusnya Anda lihat

Log diakhiri tiga baris `INFO ... OK modul=3`, `modul=5`, dan `modul=7`. Folder `build/` berisi enam subfolder: `modul-03`, `kunci-03`, `modul-05`, `kunci-05`, `modul-07`, dan `kunci-07`.

```
list.files("build", pattern = "\\.pdf$", recursive = TRUE)
#> [1] "kunci-05/Kunci_05_dplyr.pdf" "modul-05/Modul_05_dplyr.pdf"
```

11.2.3. Dari terminal dengan render_modul.R

Skrip render_modul.R yang dibuat bersama proyek (Bab 3) memeriksa setiap spesifikasi, melewati yang tidak valid, lalu merender sisanya ke folder modul/. Dari tab **Terminal**:

```
Rscript render_modul.R        # semua spesifikasi
Rscript render_modul.R 2 3    # spesifikasi kedua dan ketiga dalam urutan nama
 ↪  berkas
```

Angka pada perintah kedua adalah urutan berkas di folder spec/, bukan nomor modul. Dengan isi folder kita sekarang, 2 3 berarti modul-05-dplyr.yml dan modul-07-deskriptif.yml.

11.3. Ketika satu modul gagal

Salah paham umum: "Satu modul yang gagal menghentikan seluruh proses"

render_all() merender setiap modul secara terpisah. Bila satu modul gagal, modul lain tetap dirender, dan kegagalan itu dicatat. Untuk memperlihatkannya, kita tambahkan sementara satu spesifikasi yang contohnya pasti galat:

```
proj <- ModuleProject$new(specs_dir = "spec", output_dir = "build")
#> INFO  [07:24:46] Muat spesifikasi jumlah=4
#> INFO  [07:25:06] OK modul=5
#> ERROR [07:25:06] GAGAL modul=3 pesan=Render gagal: PDF tidak dihasilkan
 ↪  untuk modul 3. Penyebab: cannot open the connection
#> ERROR [07:25:06] GAGAL modul=7 pesan=Render gagal: PDF tidak dihasilkan
 ↪  untuk modul 7. Penyebab: cannot open the connection
#> ERROR [07:25:06] GAGAL modul=98 pesan=Render gagal: PDF tidak dihasilkan
 ↪  untuk modul 98. Penyebab: data belum dibaca
```

Keluaran di atas hanya menampilkan baris penutup render_all(). Modul 3, 5, dan 7 tetap selesai, sedangkan Modul 98 dicatat GAGAL lengkap dengan penyebabnya.

11.4. Membaca log render

Setiap baris log diawali tingkatnya:

Tabel 11.2.: Tingkat log

Tingkat	Arti	Yang perlu dilakukan
INFO	kemajuan biasa	tidak ada
WARN	ada yang tidak beres, tetapi proses berlanjut	baca pesannya; periksa PDF di bagian yang disebut

Tingkat	Arti	Yang perlu dilakukan
ERROR	satu modul gagal	perbaiki penyebabnya; lihat Bab 14

Pesan `Warning: Package tocloft Warning` yang muncul pada setiap render berasal dari LaTeX, bukan dari paket, dan dapat diabaikan (Bab 3).

`ModuleProject` menerima argumen `log_level` untuk mengatur tingkat terendah yang ditampilkan: `"info"` (bawaan), `"warn"`, atau `"error"`.

11.5. Bookdown atau Quarto

Paket memiliki dua **backend**, yaitu dua mesin yang mengubah spesifikasi menjadi PDF:

- **bookdown** (bawaan) menulis berkas antara `.Rmd` dan merendernya dengan R Markdown;
- **Quarto** menulis berkas antara `.qmd` dan merendernya dengan Quarto.

Isi modul, tampilan, dan warnanya sama. Untuk berpindah, cukup ubah satu argumen:

```
hq <- render_module(sp5, backend = "quarto", output_dir = "build-quarto")
#> INFO  [07:25:07] Data dibangkitkan modul=5 dataset=1
#> INFO  [07:25:07] render modul mulai modul=5
#> INFO  [07:25:07] Render (quarto) modul=5 jenis=modul
#> INFO  [07:25:18] render modul selesai modul=5 durasi_dtk=10.61
#> INFO  [07:25:18] render kunci mulai modul=5
#> INFO  [07:25:18] Render (quarto) modul=5 jenis=kunci
#> INFO  [07:25:25] render kunci selesai modul=5 durasi_dtk=7.57
list.files("build-quarto/modul-05")
#> [1] "data"               "Modul_05_dplyr.pdf" "Modul_05_dplyr.qmd"
#> [4] "preamble.tex"
```

Perhatikan berkas antaranya: `.qmd`, bukan `.Rmd`.

Tabel 11.3.: Perbandingan dua backend

	bookdown	Quarto
Perangkat tambahan	tidak ada	Quarto dan paket R `quarto`
Berkas antara	`.Rmd`	`.qmd`
Didukung `render_modul.R` dan `render_berlanjut()`	ya	tidak
Pemulihan galat LaTeX	ya	tidak; galat LaTeX menghentikan render

Saran kami: pakai bookdown kecuali Anda memang memerlukan Quarto, misalnya karena seluruh bahan

ajar Anda yang lain sudah memakai Quarto. Bila Quarto belum terpasang, paket hanya menulis berkas `.qmd` dan mencatat peringatan `Quarto tidak tersedia`, tanpa membuat PDF.

> **Coba sendiri**
>
> Render Modul 7 dengan backend Quarto ke folder `build-quarto`, lalu buka kedua PDF Modul 7 berdampingan. Carilah perbedaan tampilan di antara keduanya.

11.6. Ringkasan

`render_module()` merender satu modul dan paling cocok selama menyunting. `ModuleProject` memeriksa dan merender seluruh folder spesifikasi, dan satu modul yang gagal tidak menghentikan yang lain. `render_modul.R` melakukan hal serupa dari terminal dengan susunan folder hasil yang berbeda. Log render dibaca dari tingkatnya: `INFO` dapat dilewati, sedangkan `WARN` dan `ERROR` harus ditindaklanjuti. Backend bookdown adalah pilihan bawaan; Quarto dapat dipakai dengan satu argumen bila Quarto terpasang.

Modul-modul sudah menjadi PDF terpisah. Bab 12 merangkainya menjadi satu buku.

11.7. Uji pemahaman

1. Kapan Anda memakai `render_module()`, dan kapan `ModuleProject`?
2. Apa arti perintah `Rscript render_modul.R 2 3` untuk proyek yang folder `spec/`-nya berisi lima berkas?
3. Pada log `render_all()`, baris mana yang Anda baca pertama kali, dan mengapa?
4. (Transfer) Seorang asisten menjalankan `render_all()` untuk empat belas modul sebelum pulang. Pagi harinya ia menemukan dua belas modul berhasil. Bagaimana ia mengetahui dua modul mana yang gagal beserta penyebabnya, tanpa merender ulang semuanya?

12 Menyusun Buku Modul dan Buku Kunci

Mengapa nomor halaman kembali ke 1 di setiap modul, dan bagaimana memperbaikinya? Setiap modul dirender sendiri-sendiri, sehingga masing-masing memulai penomoran dari 1. Bila PDF-nya langsung digabung, buku Anda memuat banyak halaman bernomor 1, dan daftar isi tidak dapat dipakai untuk mencari halaman. Bab ini menyusun buku modul dan buku kunci dengan nomor halaman yang berlanjut.

Setelah membaca bab ini, Anda dapat menyusun buku satu semester lengkap dengan sampul, daftar modul, dan penomoran berlanjut, serta memeriksa hasilnya.

12.1. Susunan sebuah buku

Buku modul `modulstis` terdiri atas dua bagian:

1. **bagian awal**: sampul buku, kata pengantar, dan daftar modul, bernomor romawi (i, ii, ...);
2. **isi**: semua modul berurutan, bernomor arab yang berlanjut dari modul ke modul.

Buku kunci jawaban disusun dengan cara yang sama dari PDF kunci, dengan sampul berwarna hijau.

12.2. Mengapa penomoran perlu dua tahap

Nomor halaman pertama Modul 5 bergantung pada tebal Modul 3, dan tebal Modul 3 baru diketahui setelah dirender. Karena itu, paket bekerja dua tahap:

1. **Tahap 1** merender semua modul dan menghitung banyaknya halaman masing-masing.
2. Paket menghitung nomor awal setiap modul: satu ditambah jumlah halaman semua modul sebelumnya.
3. **Tahap 2** merender ulang semua modul dengan nomor awal yang sudah dihitung.

Nomor awal itu disimpan di `course` → `halaman_awal` (Bab 5), tetapi Anda tidak perlu mengisinya sendiri; fungsi `render_berlanjut()` mengerjakan ketiga langkah di atas. Karena setiap modul dirender dua kali, prosesnya memakan waktu dua kali render biasa. Jalankan hanya ketika menyusun buku, bukan setiap kali menyunting modul.

12.3. Menyiapkan bagian awal

Sampul buku dibuat oleh fungsi `sampul_buku()`, yang menghasilkan potongan LaTeX bernuansa warna yang sama dengan modul. Potongan itu disisipkan ke berkas R Markdown kecil bersama kata pengantar dan daftar modul.

Langkah 1 Buat berkas bagian awal

Buat berkas `buku/bagian_awal.Rmd` dengan isi berikut, lalu sesuaikan judul, penyusun, dan kata pengantarnya:

````
---
output:
  pdf_document:
    latex_engine: xelatex
geometry: "left=2.6cm,right=2.4cm,top=3.0cm,bottom=2.4cm"
fontsize: 11pt
header-includes:
  - \usepackage{fontspec}
  - \setmainfont{Carlito}
  - \usepackage{xcolor}
  - \definecolor{bpsnavy}{HTML}{123A6B}
  - \definecolor{bpsbiru}{HTML}{0B6FB4}
  - \definecolor{bpsbirumuda}{HTML}{6FB3E0}
  - \definecolor{bpsoranye}{HTML}{F58220}
  - \definecolor{bpshijau}{HTML}{00A651}
  - \usepackage{tikz}
  - \usetikzlibrary{calc}
  - \pagenumbering{roman}
params:
  jenis: "modul"
---

```{r, include=FALSE}
library(modulstis)
daftar <- readRDS(sprintf("daftar_%s.rds", params$jenis))
```

```{r, results="asis", echo=FALSE}
cat(sampul_buku(
  judul1 = "Pemrograman", judul2 = "Statistika",
  kode = "K242310", semester = "3 (Ganjil)",
  penyusun = "Achmad Syahrul Choir",
  jenis = if (params$jenis == "kunci") "BUKU KUNCI JAWABAN" else "BUKU MODUL
  ↪  PRAKTIKUM",
  ringkas = "Program Studi D-IV Statistika",
  varian = params$jenis))
```
````

```
# Kata Pengantar {-}

Buku ini memuat modul praktikum Pemrograman Statistika semester ganjil.

# Daftar Modul {-}

```{r, echo=FALSE}
knitr::kable(data.frame(Modul = daftar$judul, Halaman = daftar$mulai),
             col.names = c("Modul", "Halaman"))
```
```

Berkas ini dipakai untuk kedua buku. Parameter `jenis` menentukan apakah yang dibuat sampul modul (pita biru) atau sampul kunci (pita hijau). Daftar modul dibaca dari berkas `daftar_modul.rds` atau `daftar_kunci.rds`, yang dibuat oleh skrip di langkah berikutnya.

Tabel 12.1.: Argumen `sampul_buku()`

Argumen `sampul_buku()`	Isi
`judul1`, `judul2`	dua baris judul besar
`kode`, `semester`	kode mata kuliah dan semester
`penyusun`	satu nama atau vektor beberapa nama
`jenis`	label di atas judul
`ringkas`	satu baris keterangan, misalnya program studi
`varian`	`"modul"` atau `"kunci"`

12.4. Menjalankan penyusunan

Langkah 2 **Buat skrip penyusun buku**

Simpan skrip berikut sebagai `susun_buku.R` di folder proyek:

```
# ============================================================================
# susun_buku.R -- Menyusun buku modul dan buku kunci jawaban satu semester.
# Jalankan dari folder proyek:  Rscript susun_buku.R
# ============================================================================
suppressMessages(library(modulstis))

# 1. Spesifikasi, terurut menurut nama berkas (nomor dua digit menjaga urutan)
specs <- sort(list.files("spec", "\\.yml$", full.names = TRUE))
dirs  <- file.path("buku", sub("\\.yml$", "", basename(specs)))
```

```
susun <- function(jenis) {
  # 2. Render dua tahap dengan nomor halaman berlanjut
  tabel <- render_berlanjut(specs, dirs, jenis = jenis)
  tabel$judul <- vapply(specs, function(f) {
    s <- module_spec_from_yaml(f)
    sprintf("Modul %d  %s", s$number, s$title)
  }, character(1))
  saveRDS(tabel, file.path("buku", sprintf("daftar_%s.rds", jenis)))

  # 3. Bagian awal: sampul buku, kata pengantar, daftar modul
  awal <- sprintf("bagian_awal_%s.pdf", jenis)
  rmarkdown::render("buku/bagian_awal.Rmd", params = list(jenis = jenis),
                    output_file = awal, quiet = TRUE)

  # 4. Gabungkan
  pola <- if (jenis == "kunci") "^Kunci_.*\\.pdf$" else "^Modul_.*\\.pdf$"
  isi  <- vapply(dirs, function(d) list.files(d, pola, full.names = TRUE)[1],
                 character(1))
  keluar <- sprintf("Buku_%s_Pemrograman_Statistika.pdf",
                    if (jenis == "kunci") "Kunci" else "Modul")
  gabung_pdf(c(file.path("buku", awal), isi), keluar)
  message(sprintf("%s: %d halaman", keluar, pdf_halaman(keluar)))
  tabel
}

modul <- susun("modul")
kunci <- susun("kunci")
print(modul[, c("spec", "hal", "mulai", "akhir")])
```

Skrip ini mengerjakan empat hal untuk setiap jenis buku: merender modul dengan penomoran berlanjut, membuat bagian awal, mengumpulkan PDF dalam urutan yang benar, lalu menggabungkannya dengan `gabung_pdf()`.

Langkah 3 Jalankan skrip

Dari tab **Terminal**, jalankan `Rscript susun_buku.R`, atau dari Console:

```
source("susun_buku.R")
#> Buku_Modul_Pemrograman_Statistika.pdf: 71 halaman
#> Buku_Kunci_Pemrograman_Statistika.pdf: 22 halaman
#>                         spec hal mulai akhir
```

```
#> 1 modul-03-pemrograman-r.yml  29     1    29
#> 2         modul-05-dplyr.yml  30    30    59
#> 3    modul-07-deskriptif.yml  10    60    69
```

Tabel di akhir keluaran menunjukkan tebal setiap modul (`hal`) beserta nomor halaman pertama (`mulai`) dan terakhirnya (`akhir`) di dalam buku. Modul 5 dimulai tepat satu halaman sesudah Modul 3 berakhir, dan begitu seterusnya.

Hasil yang seharusnya Anda lihat

Folder proyek kini berisi `Buku_Modul_Pemrograman_Statistika.pdf` dan `Buku_Kunci_Pemrograman_Statistika.pdf`. Halaman kedua buku modul berisi kata pengantar dan daftar modul beserta nomor halamannya.



(a) Sampul buku modul (b) Daftar modul (c) Sampul buku kunci (d) Daftar isi Modul 5 dengan nomor halaman berlanjut

Gambar 12.1.: Buku modul dan buku kunci hasil susun_buku.R.

12.5. Memeriksa buku hasil gabungan

Langkah 4 **Periksa tiga hal**

Sebelum membagikan buku, periksa tiga hal berikut.

Banyaknya halaman. Buku modul harus setebal bagian awal ditambah jumlah halaman semua modul. Buku contoh kita 71 halaman: 2 halaman bagian awal (sampul serta kata pengantar dan daftar modul) ditambah 69 halaman modul.

```
pdf_halaman("Buku_Modul_Pemrograman_Statistika.pdf")
#> [1] 71
```

Urutan modul. Buka daftar modul di halaman kedua, lalu lompat ke nomor halaman yang tercantum untuk setiap modul. Sampul modul yang benar harus muncul di sana.

Nomor halaman. Buka dua halaman di perbatasan modul, misalnya halaman terakhir Modul 3 dan daftar isi Modul 5. Nomornya harus berurutan tanpa kembali ke 1.

Salah paham umum: "Cukup gabungkan PDF modul apa adanya"

`gabung_pdf()` hanya menempelkan berkas satu per satu; ia tidak mengubah nomor halaman. Bila PDF dari `render_module()` atau `render_modul.R` langsung digabung, setiap modul tetap bernomor mulai dari 1, dan nomor halaman di daftar isi setiap modul tidak cocok dengan letaknya di buku. Penomoran berlanjut hanya terjadi bila modul dirender ulang dengan `render_berlanjut()`.

Perhatian

Urutan modul di buku mengikuti urutan nama berkas di folder `spec/`. Inilah alasan nama berkas memakai nomor dua digit (Bab 5): tanpa itu, `modul-10` akan diletakkan sebelum `modul-2`.

Coba sendiri

Tambahkan satu kalimat ke kata pengantar di `buku/bagian_awal.Rmd`, lalu jalankan ulang `susun_buku.R`. Perhatikan bahwa semua modul ikut dirender ulang dua kali. Pikirkan: bila Anda hanya mengubah kata pengantar, bagian mana dari skrip yang sebenarnya perlu dijalankan?

12.6. Ringkasan

Buku modul terdiri atas bagian awal bernomor romawi dan modul-modul bernomor arab yang berlanjut. `render_berlanjut()` merender semua modul dua kali supaya setiap modul dimulai di nomor halaman yang benar. Bagian awal dibuat dari R Markdown kecil yang memakai `sampul_buku()`, lalu semuanya digabung dengan `gabung_pdf()`. Skrip `susun_buku.R` mengerjakan seluruh langkah untuk buku modul dan buku kunci sekaligus.

Bab berikutnya membahas cara mengubah tampilan seluruh modul tanpa menyunting setiap berkas.

12.7. Uji pemahaman

1. Mengapa `render_berlanjut()` merender setiap modul dua kali?
2. Apa yang terjadi bila PDF hasil `render_modul.R` langsung digabung dengan `gabung_pdf()`?

3. Pada tabel hasil `render_berlanjut()`, Modul 3 tebalnya 29 halaman dan dimulai di halaman 1. Di halaman berapa Modul 5 dimulai?
4. (Transfer) Di tengah semester, Anda memperbaiki satu salah ketik di Modul 5 tanpa mengubah tebalnya. Apakah buku perlu disusun ulang seluruhnya? Jelaskan risikonya bila salah ketik itu ternyata menambah satu halaman.

13 Menyesuaikan Tampilan

Bagaimana mengganti warna atau font semua modul sekaligus? Tampilan modul tidak disimpan di setiap spesifikasi, tetapi di satu **objek tema** yang dipakai saat merender. Mengubah tema berarti mengubah tampilan semua modul yang dirender dengannya, tanpa menyentuh satu pun berkas `.yml`.

Setelah membaca bab ini, Anda dapat membuat tema dengan palet warna dan font sendiri, memakainya pada ketiga cara merender, dan menambahkan daftar istilah ke sebuah modul.

13.1. Objek tema dan palet warna

Tema bawaan memakai enam warna yang diambil dari nuansa logo BPS:

```
library(modulstis)
str(bps_palette())
#> List of 6
#>  $ navy    : chr "#123A6B"
#>  $ biru    : chr "#0B6FB4"
#>  $ birumuda: chr "#6FB3E0"
#>  $ oranye  : chr "#F58220"
#>  $ hijau   : chr "#00A651"
#>  $ abuteks : chr "#45505C"
```

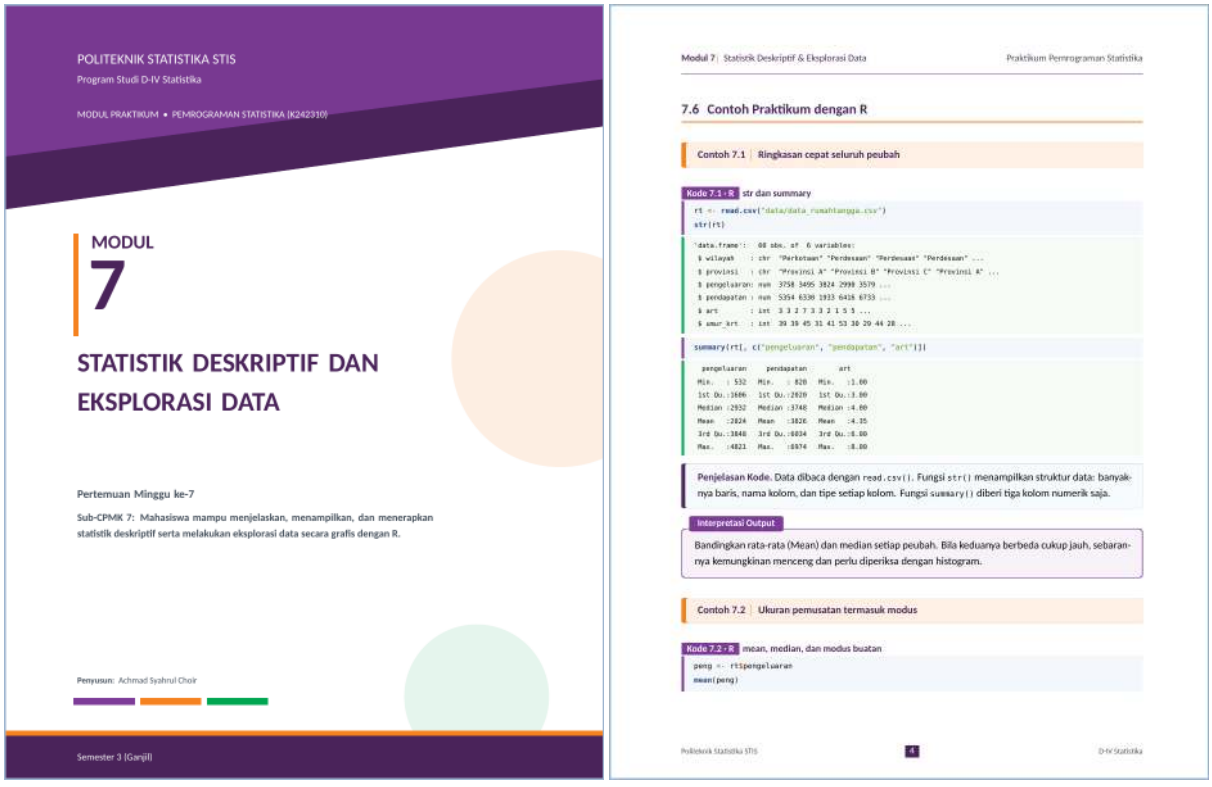
Tabel 13.1.: Enam warna palet

Nama	Dipakai untuk
`navy`	pita sampul, judul bagian, kotak rumus
`biru`	pita sampul, judul subbagian, kotak info dan interpretasi
`birumuda`	latar kotak penjelasan
`oranye`	aksen sampul, pita judul contoh, kotak perhatian
`hijau`	sampul kunci jawaban, kotak catatan dan tips
`abuteks`	warna teks

Tema dibuat dengan `theme_factory()`. Sebutkan hanya warna yang ingin diganti; warna lain tetap memakai warna bawaan. Warna ditulis dalam kode heksadesimal enam digit, seperti yang tampil di pemilih warna kebanyakan aplikasi.

Langkah 1 **Buat tema dan render dengannya**

```
tema_ungu <- theme_factory(palette = list(navy = "#4A235A", biru = "#7D3C98"))
sp7 <- module_spec_from_yaml("spec/modul-07-deskriptif.yml")
h <- render_module(sp7, theme = tema_ungu, output_dir = "build-ungu")
#> INFO  [07:27:18] Data dibangkitkan modul=7 dataset=1
#> INFO  [07:27:18] render modul mulai modul=7
#> INFO  [07:27:18] Render (bookdown/rmarkdown) modul=7 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:27:24] render modul selesai modul=7 durasi_dtk=6.33
#> INFO  [07:27:24] render kunci mulai modul=7
#> INFO  [07:27:24] Render (bookdown/rmarkdown) modul=7 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:27:30] render kunci selesai modul=7 durasi_dtk=5.65
```



(a) Sampul (b) Halaman contoh

Gambar 13.1.: Modul 7 dirender dengan tema berpalet ungu. Spesifikasinya tidak berubah.

Perhatian

Warna harus ditulis lengkap enam digit, boleh dengan atau tanpa tanda #. Nama warna seperti `"ungu"` ditolak dengan pesan `warna 'biru' harus berbentuk heksadesimal enam digit`. Pastikan juga warna `navy` dan `biru` cukup gelap, karena keduanya menjadi latar teks putih di sampul.

13.2. Mengganti font

Font utama diganti dengan argumen `font`. Font itu harus terpasang di sistem operasi, seperti Carlito pada Bab 2:

```
tema_dejavu <- theme_factory(font = "DejaVu Sans")
h <- render_module(sp7, theme = tema_dejavu, output_dir = "build-font")
#> INFO  [07:27:31] Data dibangkitkan modul=7 dataset=1
#> INFO  [07:27:31] render modul mulai modul=7
#> INFO  [07:27:31] Render (bookdown/rmarkdown) modul=7 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:27:38] render modul selesai modul=7 durasi_dtk=6.68
#> INFO  [07:27:38] render kunci mulai modul=7
#> INFO  [07:27:38] Render (bookdown/rmarkdown) modul=7 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:27:43] render kunci selesai modul=7 durasi_dtk=5.51
```



Gambar 13.2.: Halaman Deskripsi Modul 7 dengan font DejaVu Sans.

Bila nama font salah ketik atau font belum terpasang, PDF tetap terbentuk dengan font cadangan, tetapi log memuat dua baris `WARN`. Baris kedua menyebut penyebabnya, misalnya `Package fontspec Error: The font "Calibrri" cannot be found`. Render semacam ini juga jauh lebih lambat dari biasanya. Karena itu, uji font baru pada satu modul lebih dahulu dan pastikan log bebas dari `WARN` sebelum merender semuanya.

13.3. Memakai tema pada ketiga cara merender

Tema yang sama dapat dipakai di ketiga cara merender dari Bab 11:

```
# satu modul
render_module(sp7, theme = tema_ungu, output_dir = "build")

# seluruh folder
proj <- ModuleProject$new(specs_dir = "spec", theme = tema_ungu, output_dir =
 ↪  "build")

# buku dengan nomor halaman berlanjut
render_berlanjut(specs, dirs, jenis = "modul", theme = tema_ungu)
```

Untuk `render_modul.R`, ubah satu baris di skrip tersebut:

```
th <- theme_factory(palette = list(navy = "#4A235A", biru = "#7D3C98"))
```

Salah paham umum: “Tampilan diubah dengan menyunting berkas .tex hasil render”

Setiap folder hasil render memuat `preamble.tex`, yang berisi warna dan font modul. Menyunting berkas itu memang mengubah PDF, tetapi hanya sampai render berikutnya, karena paket menulis ulang `preamble.tex` setiap kali merender. Perubahan tampilan yang bertahan hanya dapat dibuat melalui objek tema.

13.4. Menambahkan daftar istilah

Modul yang memperkenalkan banyak istilah baru dapat diberi bagian **Daftar Istilah**. Bagian ini tidak termasuk sebelas bagian bawaan, sehingga perlu dua langkah.

Langkah 2 Tulis susunan bagian lengkap

Tambahkan kunci tingkat atas `sections` yang menyebut semua bagian modul, termasuk satu bagian berjenis `glossary`. Urutan di daftar ini menjadi urutan di PDF:

```
# Susunan bagian ditulis lengkap supaya Daftar Istilah dapat disisipkan
sections:
  - {id: deskripsi,  type: prose,      title: "Deskripsi Modul"}
  - {id: tujuan,     type: objectives, title: "Tujuan Praktikum"}
  - {id: alat,       type: tools,      title: "Alat dan Bahan"}
  - {id: instruksi,  type: procedure,  title: "Instruksi Pengerjaan"}
  - {id: teori,      type: theory,     title: "Teori Singkat"}
  - {id: contoh,     type: example,    title: "Contoh Praktikum dengan R"}
  - {id: latihan,    type: exercise,   title: "Latihan Mandiri"}
  - {id: penugasan,  type: assignment, title: "Penugasan"}
  - {id: ringkasan,  type: summary,    title: "Ringkasan Modul"}
  - {id: istilah,    type: glossary,   title: "Daftar Istilah"}
  - {id: referensi,  type: references, title: "Referensi"}
```

Kunci `sections` menggantikan susunan bawaan sepenuhnya. Bagian yang tidak Anda sebut di sini tidak dicetak, walaupun isinya ada di `body`.

Langkah 3 Tulis istilahnya

Isi bagian itu di `body`, dengan kunci yang sama dengan `id`-nya, yaitu `istilah`:

```
  istilah:
    - istilah: "median"
```

```
      inggris: "median"
      definisi: "Nilai tengah data yang sudah diurutkan."
    - istilah: "simpangan kuartil"
      inggris: "interquartile range"
      definisi: "Selisih kuartil ketiga dan kuartil pertama."
    - istilah: "pencilan"
      inggris: "outlier"
      definisi: "Amatan yang letaknya jauh dari sebagian besar data."
```

Setiap istilah memiliki `istilah`, `inggris`, `definisi`, dan boleh ditambah `kbbi` untuk catatan kebakuan. Paket mengurutkan istilah menurut abjad.

```
sp7 <- module_spec_from_yaml("spec/modul-07-deskriptif.yml")
sp7$validate()
#> □ Spesifikasi valid.
h <- render_module(sp7, output_dir = "build")
#> INFO  [07:27:44] Data dibangkitkan modul=7 dataset=1
#> INFO  [07:27:44] render modul mulai modul=7
#> INFO  [07:27:44] Render (bookdown/rmarkdown) modul=7 jenis=modul
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:27:51] render modul selesai modul=7 durasi_dtk=6.3
#> INFO  [07:27:51] render kunci mulai modul=7
#> INFO  [07:27:51] Render (bookdown/rmarkdown) modul=7 jenis=kunci
#> Warning: Package tocloft Warning: \@starttoc has already been redefined; tocloft
 ↪  bailing
#> Warning:  out. on input line 1156.
#> INFO  [07:27:56] render kunci selesai modul=7 durasi_dtk=5.7
```



Gambar 13.3.: Bagian Daftar Istilah pada Modul 7, terurut menurut abjad.

> **Catatan**
>
> Bila bagian `glossary` dicantumkan, tetapi tidak diisi, paket memakai bank istilah bawaannya, yang disusun untuk mata kuliah Analisis Peubah Ganda. Isilah daftar istilah sendiri untuk mata kuliah lain.

> **Coba sendiri**
>
> Pindahkan baris `istilah` di `sections` ke posisi kedua, sesudah `deskripsi`, lalu render ulang. Di mana Daftar Istilah kini muncul, dan berapa nomor bagiannya?

13.5. Ringkasan

Tampilan modul ditentukan oleh objek tema, bukan oleh spesifikasi. `theme_factory()` membuat tema dengan palet warna dan font sendiri, dan tema itu dipakai lewat argumen `theme` pada `render_module()`, `ModuleProject`, dan `render_berlanjut()`. Menyunting `preamble.tex` hasil render tidak bertahan. Daftar istilah ditambahkan dengan menuliskan susunan bagian lengkap di `sections`, termasuk satu bagian berjenis `glossary`, lalu mengisinya di `body`.

13.6. Uji pemahaman

1. Di mana tampilan modul disimpan, dan mengapa tidak di spesifikasi?

2. Anda membuat tema dengan `palette = list(oranye = "#E67E22")`. Warna apa saja yang berubah, dan mana yang tetap?
3. Setelah menambahkan `sections`, bagian Tambahan di modul Anda menghilang. Mengapa?
4. (Transfer) Program studi lain ingin memakai `modulstis` dengan warna kampusnya sendiri. Apa saja yang perlu mereka ubah, dan apa yang tidak perlu diubah sama sekali?

14 Ketika Render Gagal

Dari mana mulai mencari ketika render berhenti dengan pesan merah yang panjang? Pesan galat render sering terlihat menakutkan karena memuat keluaran tiga program sekaligus: YAML, R, dan LaTeX. Kabar baiknya, hampir semua galat berasal dari salah satu dari empat sumber, dan setiap sumber meninggalkan tanda yang khas.

Setelah membaca bab ini, Anda dapat mengenali sumber sebuah galat dari pesannya, menemukan baris yang salah di spesifikasi, dan memeriksa hasil yang tampak benar, tetapi keliru.

14.1. Urutan penelusuran

Bila render gagal atau hasilnya mencurigakan, telusuri dengan urutan berikut. Setiap langkah lebih murah daripada langkah sesudahnya.

1. **Baca baris `WARN` dan `ERROR`**, bukan baris terakhir saja. Kenali sumbernya dengan
    1.
2. **Baca ulang spesifikasi dan periksa** dengan `module_spec_from_yaml()` lalu `validate()`. Langkah ini hanya beberapa detik dan menemukan semua galat YAML dan isian.
3. **Render satu modul saja** dengan `render_module()`, bukan seluruh proyek, supaya log hanya memuat modul yang bermasalah.
4. **Buka berkas antara** di folder hasil: `.Rmd` untuk galat kode R, `.log` untuk galat LaTeX.

Tabel 14.1.: Mengenali sumber galat dari pesannya

Tanda di pesan	Sumber	Bahas di
`Scanner error`, `Parser error`, `line ..., column ...`	YAML	Bagian 14.2
`✖ Spesifikasi tidak valid`	isian spesifikasi	Bagian 14.3
`Quitting from lines ...` lalu `Render gagal ... Penyebab: ...`	kode R di contoh atau solusi	Bagian 14.4
`WARN ... LaTeX melaporkan galat`	LaTeX di dalam kotak, atau font	Bagian 14.5
`there is no package called ...`, `xelatex not found`	pemasangan	Bab 2

Salah paham umum: “Baris yang disebut pesan galat adalah baris yang salah”

Nomor baris pada pesan galat menunjuk tempat **program menyadari** ada masalah, bukan selalu tempat kesalahan dibuat. Pada Bab 4 Anda melihat galat yang ada di baris 1, tetapi dilaporkan di baris 2. Nomor pada `Quitting from lines` merujuk berkas `.Rmd` antara, dan nomor `l.` pada galat LaTeX merujuk berkas `.tex` antara, bukan spesifikasi Anda. Gunakan nomor itu sebagai titik awal pencarian, lalu cocokkan isinya dengan spesifikasi.

14.2. Galat YAML

Galat YAML muncul saat spesifikasi dibaca, sebelum render dimulai. Pesannya selalu diawali nama berkas dalam kurung, lalu `Scanner error` atau `Parser error`, dan menyebut baris serta kolomnya. Bab 4 membahas lima galat yang paling sering beserta pesannya (Tabel 4.1).

14.3. Galat isian spesifikasi

Spesifikasi yang lolos dibaca YAML belum tentu lengkap. `validate()` memeriksa isian yang wajib dan melaporkan **semua** masalah sekaligus, bukan hanya yang pertama. Berikut Modul 7 yang kehilangan kunci `number` dan skor salah satu solusinya:

```
library(modulstis)
sp <- module_spec_from_yaml("isian-salah.yml")
sp$validate()
#> ✖ Spesifikasi tidak valid:
#>   - `number` harus bilangan bulat 1..99
#>   - exercises[2].points hilang
```

Tabel 14.2.: Pesan `validate()` dan artinya

Pesan `validate()`	Arti
`` `number` harus bilangan bulat 1..99 ``	`number` hilang atau bukan angka
`` `title` tidak boleh kosong ``	`title` hilang
`exercises[2].points hilang`	solusi kedua tidak memiliki `points`
`dataset '...' tidak punya recipe.kind`	resep data tanpa `kind`
`dataset '...': berkas tidak ditemukan (...)`	`path` pada `kind: file` salah
`jenis bagian tidak dikenal: ...`	`type` pada `sections` salah ketik

14.4. Galat kode R

Kode di contoh atau solusi dijalankan saat render. Bila kode itu galat, render berhenti dan PDF tidak dibuat. Berikut Modul 7 dengan salah ketik ekstensi berkas data, `.cvs` alih-alih `.csv`:

```
sp <- module_spec_from_yaml("kode-salah.yml")
#> Quitting from lines 109-113 [unnamed-chunk-1] (Modul_07_deskriptif.Rmd)
#> Error: Render gagal: PDF tidak dihasilkan untuk modul 7. Penyebab: cannot open
 ↪  the connection
```

Pesan ini terdiri atas dua bagian:

- `Quitting from lines ...` menyebut baris di berkas **`.Rmd` antara** tempat R berhenti;
- `Render gagal: ... Penyebab: ...` menyebut pesan galat R-nya.

Langkah 1 **Temukan contoh yang galat**

Buka berkas `.Rmd` di folder hasil, yaitu `hasil/modul-07/Modul_07_deskriptif.Rmd`, lalu lompat ke nomor baris yang disebut lewat menu **Code → Go To Line** (**Shift+Alt+G**). Anda akan menemukan kode contoh yang galat, lengkap dengan judul contohnya di atasnya. Judul itu menunjukkan contoh mana di spesifikasi yang harus diperbaiki.

```
rmd <- readLines("hasil/modul-07/Modul_07_deskriptif.Rmd")
grep("data_rumahtangga.cvs", rmd)
#> [1] 109 185
```

> Catatan
>
> Bila kode yang galat ada di **solusi**, PDF modul tetap terbentuk, tetapi kunci jawabannya gagal. Pesan yang sama muncul dengan `Kunci_07_...Rmd` sebagai nama berkasnya.

14.5. Galat LaTeX

Galat LaTeX berbeda dari tiga galat sebelumnya: **PDF biasanya tetap terbentuk**. Paket memulihkan render dengan menjalankan XeLaTeX sekali lagi tanpa berhenti pada galat, lalu mencetak dua baris `WARN` (Bab 10):

- `render tunggal gagal; memulihkan via 2x xelatex` menandakan bahwa pemulihan berjalan;
- `LaTeX melaporkan galat; isi PDF di sekitar baris ini rusak` menyebut galat pertama beserta potongan teks tempat galat itu muncul.

Potongan teks sesudah `baris=l.NNN` adalah petunjuk terkuat: carilah teks itu di spesifikasi Anda. Penyebab yang paling sering adalah karakter khusus yang tidak diloloskan di dalam kotak (Tabel 10.3), lingkungan kotak yang tidak ditutup dengan `\end{...}`, dan font yang tidak terpasang (Bab 13).

Tabel 14.3.: Pesan LaTeX dan penyebabnya

Pesan LaTeX	Penyebab yang paling mungkin
`Missing $ inserted`	_ atau ^ di luar rumus di dalam kotak
`Misplaced alignment tab character &`	`&` tidak diloloskan
`Environment kotak... undefined`	nama kotak salah ketik
`\begin{...} ended by \end{document}`	`\end{...}` lupa ditulis
`The font "..." cannot be found`	font belum terpasang atau namanya salah

Berkas `.log` di folder hasil memuat laporan lengkap XeLaTeX. Setiap galat diawali tanda seru di awal baris, sehingga mudah dicari dengan **Ctrl+F** dan tanda `!`.

14.6. Hasil tampak salah walau tanpa galat

Kesalahan yang paling berbahaya adalah yang tidak menimbulkan pesan apa pun. Buku ini telah menunjukkan beberapa di antaranya. Gunakan daftar berikut ketika memeriksa PDF:

Tabel 14.4.: Gejala hasil yang keliru tanpa pesan galat

Gejala di PDF	Penyebab	Bab
sampul menyebut *Analisis Peubah Ganda*	blok `course` hilang	Bab 5
bagian bertuliskan *belum diisi* padahal sudah diisi	nama bagian di `body` salah ketik	Bab 6
kode contoh tanpa keluaran	`eval: true` tidak ditulis	Bab 7
keluaran kode tidak masuk akal, misalnya `NULL` atau `NA`	nama kolom salah ketik di kode	Bab 7
kata terpenggal tidak sesuai EYD	pola pemenggalan Indonesia belum terpasang	Bab 2
teks miring seperti rumus, atau kalimat terpotong	galat LaTeX yang dipulihkan	Bab 10
soal dan solusi tidak berpasangan	`id` dan urutan `exercises` tidak diperbarui	Bab 9
nomor halaman kembali ke 1 di buku	PDF digabung tanpa `render_berlanjut()`	Bab 12

> **Coba sendiri**
>
> Ambil Modul 7 Anda dan buat sendiri satu kesalahan dari setiap baris 1, satu per satu. Untuk setiap kesalahan, catat pesan yang muncul sebelum membaca penjelasannya di bab ini. Latihan ini membuat Anda mengenali pesan-pesan itu ketika kelak muncul tanpa sengaja.

14.7. Meminta bantuan

Bila Anda tetap buntu, siapkan tiga hal sebelum bertanya kepada rekan atau melapor ke repositori paket:

1. spesifikasi terkecil yang masih menimbulkan galat, misalnya Modul 7 yang dikurangi sampai tinggal satu contoh;
2. seluruh keluaran render, termasuk baris `WARN` dan `Quitting from`;
3. hasil `cek_kesiapan()` dan `packageVersion("modulstis")`.

Laporan masalah paket dapat dikirim ke https://github.com/madsyair/modulstis/issues.

14.8. Ringkasan

Galat render berasal dari empat sumber utama: YAML, isian spesifikasi, kode R, dan LaTeX, ditambah masalah pemasangan. Galat YAML dan isian ditemukan dalam hitungan detik dengan `module_spec_from_yaml()` dan `validate()`. Galat kode R menghentikan render dan ditelusuri lewat berkas `.Rmd` antara. Galat LaTeX biasanya tidak menghentikan render, tetapi ditandai baris `WARN` yang harus ditindaklanjuti. Sebagian kesalahan tidak menimbulkan pesan apa pun, sehingga PDF tetap perlu diperiksa dengan Tabel 14.4.

14.9. Uji pemahaman

1. Sebutkan urutan empat langkah penelusuran galat dan alasan urutannya.
2. Pesan render menyebut `Quitting from lines 109-113`. Di berkas mana baris itu berada, dan bagaimana Anda menemukan contoh yang harus diperbaiki di spesifikasi?
3. Mengapa galat LaTeX lebih mudah terlewat daripada galat kode R?
4. (Transfer) Seorang rekan mengirim PDF modul yang tampak rapi dan mengatakan render berhasil tanpa galat. Tiga pemeriksaan apa yang Anda lakukan pada PDF itu sebelum modul dibagikan, dan mengapa ketiganya?

15 Alur Kerja Satu Semester

Bagaimana menjaga empat belas modul tetap rapi dari awal sampai akhir semester? Bab-bab sebelumnya membahas setiap langkah secara terpisah. Bab ini merangkainya menjadi kebiasaan kerja: apa yang dilakukan sebelum semester, selama semester, dan sebelum modul dibagikan, serta cara menyimpan dan berbagi proyek dengan tim pengampu.

Setelah membaca bab ini, Anda dapat menyusun alur kerja semester Anda sendiri dan memakai daftar periksa sebelum menerbitkan modul.

15.1. Sebelum semester dimulai

1. **Siapkan komputer** dan jalankan `cek_kesiapan()` (Bab 2).
2. **Buat proyek** untuk mata kuliah (Bab 3). Bila mata kuliahnya sudah memiliki spesifikasi bawaan, salin dari `system.file("specs_ps", package = "modulstis")` dan sejenisnya (Bab 11).
3. **Tulis satu modul sampai lengkap**, termasuk data, latihan, dan kunci, lalu periksa PDF-nya dengan teliti. Modul pertama ini menjadi pola bagi modul-modul lain.
4. **Salin pola itu** untuk modul berikutnya. Blok `course` cukup disalin (Bagian 5.6).
5. **Susun buku** modul dan kunci bila Anda membagikannya sebagai satu berkas (Bab 12).

15.2. Menyunting satu modul di tengah semester

Perubahan di tengah semester sebaiknya kecil dan terlacak. Alurnya:

1. sunting spesifikasi modul yang bersangkutan saja;
2. baca ulang dan periksa: `module_spec_from_yaml()` lalu `validate()`;
3. render modul itu saja dengan `render_module()`, lalu periksa PDF-nya;
4. bila modul dibagikan dalam bentuk buku, susun ulang buku dengan `susun_buku.R`.

> **Perhatian**
>
> Mengubah `seed`, resep data, atau urutan soal di tengah semester mengubah angka dan nomor yang sudah dilihat mahasiswa (Bagian 8.6, Bab 9). Bila perubahan semacam itu tidak terhindarkan, umumkan kepada mahasiswa dan tandai versi modulnya, misalnya dengan menambahkan tanggal revisi di bagian `deskripsi`.

15.3. Menyimpan proyek di Git

Git adalah program pencatat riwayat perubahan berkas. Setiap kali Anda menyimpan *commit*, Git mencatat isi semua berkas proyek saat itu, sehingga Anda dapat melihat apa yang berubah dan kembali ke

versi lama bila perlu. Proyek `modulstis` sangat cocok disimpan di Git karena sumbernya berupa berkas teks.

Langkah 1 Aktifkan Git untuk proyek

Di RStudio, pilih **Tools → Version Control → Project Setup**, lalu pada **Version control system** pilih **Git**. RStudio akan dimulai ulang dan menampilkan tab **Git** di panel kanan atas. Bila pilihan Git tidak tersedia, pasang Git lebih dahulu dari https://git-scm.com.

Langkah 2 Kecualikan hasil render

Buat berkas `.gitignore` di folder proyek. Berkas ini menyebut berkas dan folder yang **tidak** dicatat Git:

```
writeLines(c(
  "# hasil render: selalu dapat dibuat ulang dari spec/",
  "modul/", "build*/", "buku/*/", "buku/*.pdf", "buku/*.rds", "*.pdf",
  "# berkas sementara R dan RStudio",
  ".Rhistory", ".RData", ".Rproj.user/"
), ".gitignore")
cat(readLines(".gitignore"), sep = "\n")
#> # hasil render: selalu dapat dibuat ulang dari spec/
#> modul/
#> build*/
#> buku/*/
#> buku/*.pdf
#> buku/*.rds
#> *.pdf
#> # berkas sementara R dan RStudio
#> .Rhistory
#> .RData
#> .Rproj.user/
```

Tabel 15.1.: Isi proyek yang dicatat dan tidak dicatat Git

Dicatat Git	Tidak dicatat Git
`spec/*.yml`	`modul/`, `build/`, `build-*/`
`render_modul.R`, `susun_buku.R`	PDF hasil render
`buku/bagian_awal.Rmd`	folder modul di dalam `buku/`
`data-asli/` (bila datanya boleh disimpan)	`.Rhistory`, `.RData`
`README.md`, `.gitignore`, `*.Rproj`	

Salah paham umum: “PDF hasil render perlu disimpan di Git”

PDF selalu dapat dibuat ulang dari spesifikasi, jadi menyimpannya di Git tidak menambah informasi. Sebaliknya, setiap render menghasilkan PDF yang berbeda sedikit, misalnya tanggal pembuatan yang tersimpan di dalam berkas, sehingga riwayat Git penuh perubahan yang tidak berarti dan ukurannya membengkak. Simpan di Git sumbernya saja. PDF yang dibagikan kepada mahasiswa disimpan di tempat pembagian, misalnya Google Classroom atau folder bersama, dengan nama yang memuat tanggal atau versi.

Langkah 3 **Simpan perubahan**

Di tab **Git**, centang berkas yang berubah, klik **Commit**, tulis pesan singkat yang menjelaskan perubahannya, misalnya *Modul 7: tambah contoh histogram*, lalu klik **Commit** sekali lagi.

15.4. Daftar periksa sebelum menerbitkan

Jalankan daftar periksa berikut setiap kali modul atau buku akan dibagikan.

Tabel 15.2.: Daftar periksa sebelum menerbitkan

No.	Periksa	Cara
1	Semua spesifikasi valid	`ModuleProject$new(specs_dir = "spec")$validate_all()` bernilai `TRUE`
2	Log render bersih	tidak ada baris `WARN` atau `ERROR` selain peringatan `tocloft`
3	Sampul benar	mata kuliah, kode, semester, dan penyusun sesuai
4	Tidak ada bagian kosong	cari tulisan *belum diisi* di PDF
5	Setiap kode contoh berkeluaran	periksa setiap kotak kode
6	Keluaran masuk akal	tidak ada `NULL`, `NA`, atau pesan galat yang tak disengaja
7	Kotak *Tentang data* ada	untuk setiap modul yang memakai data simulasi
8	Soal dan solusi berpasangan	jumlah dan urutan Latihan sama dengan `exercises`
9	Skor berjumlah benar	jumlah `points` sesuai rencana penilaian
10	Nomor halaman buku berlanjut	periksa dua halaman di perbatasan modul
11	Berkas data ikut dibagikan	folder `data/` disertakan bersama modul

Pemeriksaan nomor 4 dapat dilakukan untuk semua modul sekaligus dengan `pdftools`:

```
pdf <- list.files("build", "^Modul_.*\\.pdf$", recursive = TRUE, full.names = TRUE)
kosong <- vapply(pdf, function(f) any(grepl("belum diisi", pdftools::pdf_text(f))),
                 logical(1))
pdf[kosong]
#> character(0)
```

Hasil `character(0)` berarti tidak ada modul yang memuat bagian kosong.

15.5. Berbagi proyek dengan tim pengampu

Mata kuliah yang diampu beberapa dosen dapat memakai satu proyek bersama. Beberapa kesepakatan membuat kerja bersama lebih lancar:

- **Satu modul, satu penanggung jawab.** Karena setiap modul ada di berkas sendiri, dua dosen yang menyunting modul berbeda tidak akan saling menimpa.
- **Versi paket yang sama.** Catat versi `modulstis` di `README.md` proyek, dan minta setiap anggota memeriksanya dengan `packageVersion("modulstis")`. Versi yang berbeda dapat menghasilkan tampilan yang sedikit berbeda.
- **Satu orang menyusun buku.** Render seluruh modul dan penyusunan buku dilakukan oleh satu orang dari versi spesifikasi yang sudah disepakati.
- **Tempat bersama.** Simpan proyek di repositori Git bersama, misalnya GitHub atau GitLab kampus. Bila tim belum memakai Git, folder bersama tetap dapat dipakai, asalkan setiap orang hanya menyunting berkas modulnya sendiri.

15.6. Penutup

Anda sudah menempuh seluruh perjalanan: dari folder kosong, satu modul contoh, Modul 7 yang dibangun bagian demi bagian, sampai buku modul dan buku kunci satu semester. Polanya selalu sama: **sunting spesifikasi, periksa, render, lalu baca hasilnya**. Paket mengerjakan penyusunan, tampilan, dan sinkronisasi; Anda tetap memegang isi dan mutunya.

Lampiran berikut merangkum semua kunci spesifikasi, menyediakan templat kosong, dan memuat kunci Uji Pemahaman setiap bab.

15.7. Uji pemahaman

1. Sebutkan empat langkah menyunting satu modul di tengah semester.
2. Mengapa folder `modul/` dan `build/` dikecualikan dari Git?
3. Pada daftar periksa, mengapa pemeriksaan nomor 6 tidak dapat digantikan oleh pemeriksaan nomor 2?

4. (Transfer) Tim pengampu Anda terdiri atas tiga dosen, dan dua di antaranya memakai versi `modulstis` yang berbeda. Apa yang mungkin terjadi pada buku modul, dan bagaimana Anda mengatasinya sebelum semester dimulai?

A Rujukan Cepat Spesifikasi YAML

Lampiran ini merangkum semua kunci spesifikasi dalam satu tempat. Kolom **Bab** menunjuk tempat pembahasannya.

Kunci tingkat atas

Kunci	Wajib	Isi	Bab
`number`	ya	nomor modul, 1 sampai 99	5
`title`	ya	judul lengkap	5
`short_code`		kode singkat untuk nama berkas; bila kosong diturunkan dari judul	5
`topic`		judul ringkas di kepala halaman; bila kosong memakai `title`	5
`pertemuan`		pertemuan ke berapa menurut RPS	5
`sub_cpmk`		capaian pembelajaran modul	5
`packages`		paket yang dipakai; catatan, tidak dicetak	5
`contoh_title`		judul pengganti bagian Contoh	6
`tambahan_title`		judul pengganti bagian Tambahan	6
`course`	dianjurkan	identitas mata kuliah; lihat di bawah	5
`datasets`		daftar resep data	8
`body`		isi bagian-bagian modul	6
`exercises`		solusi untuk kunci jawaban	9
`sections`		susunan bagian kustom; menggantikan susunan bawaan	13

Kunci	Wajib	Isi	Bab
`references`		disimpan, tidak dicetak; pakai `body` → `referensi`	6

Blok `course`

Kunci	Isi	Bawaan bila tidak ditulis
`matkul`	nama mata kuliah	Analisis Peubah Ganda
`matkul_besar`	nama dalam huruf kapital	ANALISIS PEUBAH GANDA
`kode`	kode mata kuliah	K203318
`semester`	semester	5 (Ganjil)
`prodi`	program studi	D-IV Komputasi Statistik
`penyusun`	daftar nama	Achmad Syahrul Choir
`halaman_awal`	nomor halaman pertama	1; diatur `render_berlanjut()`

Blok `datasets`

Setiap butir: `name` (nama berkas CSV), `role` (keterangan), dan `recipe`.

`kind`	Kunci resep	Bab
`tabular`	`"n"`, `seed`, `columns`	8
`mvn`	`"n"`, `seed`, `vars`, `mu`, `Sigma`	8
`grouped_mvn`	`seed`, `vars`, `groups`, `means`, `Sigma`	8
`file`	`path`, `columns`, `rename`, `"n"`, `na`	8

Jenis kolom `tabular`: `numeric` (`min`, `max`, `digits`), `integer` (`min`, `max`), `category` (`levels`, `probs`), `balanced` (`levels`, `each`).

Bagian di bawah `body`

Kunci	Judul di modul	Bentuk isi
`deskripsi`	Deskripsi Modul	daftar paragraf
`tujuan`	Tujuan Praktikum	daftar paragraf
`alat`	Alat dan Bahan	daftar paragraf
`instruksi`	Instruksi Pengerjaan	daftar paragraf
`teori`	Teori Singkat	daftar paragraf, boleh dengan `###`

Kunci	Judul di modul	Bentuk isi
`contoh`	Contoh Praktikum dengan R	daftar contoh atau paragraf
`latihan`	Latihan Mandiri	daftar soal (`judul`, `isi`) atau paragraf
`penugasan`	Penugasan	daftar tugas (`judul`, `isi`) atau paragraf
`tambahan`	Tambahan: Penyelesaian dengan Fungsi R	seperti `contoh`; dilewati bila kosong
`ringkasan`	Ringkasan Modul	daftar paragraf
`referensi`	Referensi	daftar paragraf
`kunci`	(hanya di kunci jawaban)	daftar paragraf

Kunci pada satu contoh

Kunci	Isi
`contoh`	judul contoh
`narasi`	paragraf pengantar sebelum kode
`judul_r`, `kode_r`	judul dan kode R
`judul_py`, `kode_py`	judul dan kode Python
`eval`	`true` agar kode dijalankan; bawaan `false`
`penjelasan`	isi kotak Penjelasan Kode
`interpretasi`	isi kotak Interpretasi Output

Kunci pada satu solusi (`exercises`)

Kunci	Isi
`id`	label soal, misalnya `L7.1`
`points`	skor; **wajib**
`solution`	ringkasan jawaban
`judul_r`, `solusi_r`	judul dan kode solusi R
`judul_py`, `solusi_py`	judul dan kode solusi Python
`eval`	`false` agar kode tidak dijalankan; bawaan dijalankan
`interpretasi`	pembahasan untuk pengajar

Jenis bagian untuk `sections`

Setiap butir `sections` memiliki `id`, `type`, dan `title`. Jenis yang sah: `prose`, `objectives`, `tools`, `procedure`, `theory`, `example`, `exercise`, `assignment`, `summary`, `references`, dan `glossary`.

B Templat Spesifikasi Siap Salin

Templat berikut berisi semua bagian yang umum dipakai. Salin ke folder `spec/`, beri nama sesuai nomor modul, ganti teks di dalam tanda kutip, lalu hapus bagian yang tidak diperlukan. Templat ini lolos `validate()` dan dapat langsung dirender, sehingga Anda dapat memulai dari keadaan yang pasti berhasil.

```yaml
# ======================================================================
# Templat spesifikasi modul modulstis
# Ganti semua teks di dalam tanda kutip, lalu hapus bagian yang tidak perlu.
# Indentasi: dua spasi per tingkat. Jangan memakai tab.
# ======================================================================
number: 1                                # 1 sampai 99
title: "Judul Lengkap Modul"
short_code: "kode_singkat"               # huruf kecil, angka, garis bawah
topic: "Judul Ringkas untuk Kepala Halaman"
pertemuan: 1
sub_cpmk: >
  Sub-CPMK 1: Mahasiswa mampu ...
packages: [dplyr]                        # catatan saja, tidak dicetak
# contoh_title: "Contoh Praktikum dengan R dan Python"

course:                                  # salin sama persis ke semua modul
  matkul: "Nama Mata Kuliah"
  matkul_besar: "NAMA MATA KULIAH"
  kode: "K000000"
  semester: "1 (Ganjil)"
  prodi: "D-IV Statistika"
  penyusun:
    - "Nama Penyusun"

datasets:
  - name: data_latihan                   # menjadi data/data_latihan.csv
    role: contoh
    recipe:
      kind: tabular
      "n": 50
      seed: 2026                         # ganti per modul
      columns:
```

```
        kelompok: {type: category, levels: [A, B], probs: [0.5, 0.5]}
        nilai: {type: numeric, min: 0, max: 100, digits: 1}

body:
  deskripsi:
    - >
      Gambaran modul: apa yang dibahas, mengapa penting, dan kaitannya dengan
      modul sebelum dan sesudahnya.
    - |
      \begin{kotakperhatian}[Tentang data]
      Berkas data pada modul ini adalah \textbf{data simulasi}, bukan data nyata.
      \end{kotakperhatian}
  tujuan:
    - "Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:"
    - "1. **Menjelaskan** ..."
    - "2. **Menerapkan** ..."
  alat:
    - "Perangkat dan bahan yang diperlukan:"
    - "- **R** dan **RStudio**."
    - "- Berkas data `data/data_latihan.csv`."
  instruksi:
    - "Kerjakan Contoh berurutan, lalu Latihan Mandiri dan Penugasan."
  teori:
    - "### Subjudul Teori"
    - >
      Teori singkat ditulis di sini.
  contoh:
    - contoh: "Judul contoh pertama"
      judul_r: "Judul kode"
      kode_r: |
        d <- read.csv("data/data_latihan.csv")
        summary(d)
      eval: true                          # tanpa baris ini kode tidak dijalankan
      penjelasan: >
        Apa yang dikerjakan kode, baris demi baris.
      interpretasi: >
        Apa arti keluarannya bagi persoalan yang dibahas.
  latihan:
    - judul: "Judul latihan pertama"
      isi: |
        Perintah latihan.
```

```
        ```{r eval=FALSE}
        d <- read.csv("data/data_latihan.csv")
        ____(d$nilai)   # rata-rata
        ```
  penugasan:
    - judul: "Judul tugas"
      isi: >
        Perintah tugas.
  ringkasan:
    - >
      Rangkuman modul dan kaitannya dengan modul berikutnya.
  referensi:
    - "Penulis, A. (Tahun). *Judul Buku*. Penerbit."

exercises:
  - id: "L1.1"
    points: 100
    solution: "Ringkasan jawaban dalam satu kalimat."
    judul_r: "Solusi Latihan 1.1"
    solusi_r: |
      d <- read.csv("data/data_latihan.csv")
      mean(d$nilai)
    interpretasi: >
      Pembahasan untuk pengajar.
```

Templat ini juga tersimpan di repositori paket, di berkas `pedoman/contoh/templat-kosong.yml`.

C Katalog Kotak, Warna, dan Karakter Khusus

Kotak

Kotak ditulis sebagai butir `|` di `body`:

```
- |
  \begin{kotakperhatian}[Judul pengganti]
  Isi kotak.
  \end{kotakperhatian}
```

Kotak	Judul bawaan	Judul dapat diganti
`kotakinfo`	tanpa judul	ya
`kotakcatatan`	Catatan	ya
`kotakperhatian`	Perhatian	ya
`kotaksalah`	Kesalahan Umum	ya
`kotaktips`	Tips dan Praktik Baik	ya
`kotakvariasi`	Variasi dan Alternatif	ya
`kotakrumus`	tanpa judul	tidak
`kotaktujuan`	tanpa judul	tidak
`kotaktugas`	tanpa judul	tidak
`kotakkunci`	Kunci & Pembahasan	tidak

Tampilannya ada di Gambar 10.1.

Warna palet

Nama palet	Nama warna LaTeX	Bawaan
`navy`	`bpsnavy`	`#123A6B`
`biru`	`bpsbiru`	`#0B6FB4`
`birumuda`	`bpsbirumuda`	`#6FB3E0`
`oranye`	`bpsoranye`	`#F58220`
`hijau`	`bpshijau`	`#00A651`
`abuteks`	`abuteks`	`#45505C`

Nama warna LaTeX dapat dipakai di dalam kotak, misalnya `\textcolor{bpsoranye}{teks}`.

Karakter yang harus diloloskan di dalam kotak

Karakter	Tulis	Karakter	Tulis
_	`\_`	#	`\#`
%	`\%`	$	`\$`
&	`\&`	~	`\textasciitilde{}`

Di luar kotak, pada paragraf Markdown biasa, karakter-karakter ini ditulis apa adanya.

D Fungsi Paket untuk Penyusun Modul

Lampiran ini mencantumkan fungsi yang dipakai di buku ini. Bantuan lengkap setiap fungsi dapat dibuka di RStudio dengan `?nama_fungsi`, misalnya `?render_module`.

Menyiapkan

Fungsi	Kegunaan	Bab
`cek_kesiapan()`	memeriksa perangkat yang diperlukan	2
`buat_proyek_modul()`	membuat folder proyek beserta spesifikasi contoh	3
`buka_pedoman()`	membuka buku pedoman ini	–

Membaca dan memeriksa

Fungsi	Kegunaan	Bab
`module_spec_from_yaml(path)`	membaca spesifikasi menjadi objek	3
`spec$validate()`	memeriksa isian spesifikasi	3, 14
`default_sections()`	menampilkan sebelas bagian bawaan	6
`generate_module_data(spec)`	membangkitkan data tanpa merender	8
`generate_dataset(recipe)`	membangkitkan satu data dari resep	8

Merender

Fungsi	Kegunaan	Bab
`render_module(spec, backend, theme, output_dir)`	merender satu modul dan kuncinya	5, 11
`ModuleProject$new(specs_dir, backend, theme, output_dir, log_level)`	membuat proyek dari satu folder spesifikasi	11
`proj$validate_all()`	memeriksa semua spesifikasi	11

Fungsi	Kegunaan	Bab
`proj$generate_data()`	membangkitkan data semua modul	11
`proj$render_all()`	merender semua modul dan kunci	11

Menerbitkan

Fungsi	Kegunaan	Bab
`render_berlanjut(specs, out_dirs, jenis, theme, mulai)`	render dua tahap dengan nomor halaman berlanjut	12
`sampul_buku(...)`	potongan LaTeX sampul buku	12
`gabung_pdf(berkas, keluaran)`	menggabung PDF	12
`pdf_halaman(path)`	menghitung halaman PDF	12

Tampilan

Fungsi	Kegunaan	Bab
`bps_palette()`	palet warna bawaan	13
`theme_factory(palette, font)`	membuat tema	13
`load_glossary(path)`	membaca bank istilah dari berkas YAML	13

E Kunci Uji Pemahaman

Bab 1

1. Modul praktikum (PDF), kunci jawaban (PDF), dan berkas data latihan (CSV).
2. Perbaikan itu hilang, karena PDF dibuat ulang dari spesifikasi pada setiap render. Ia seharusnya memperbaiki berkas `.yml` di folder `spec/`.
3. Keduanya dibangkitkan dari resep yang sama dengan `seed` yang sama, dalam render yang sama.
4. Keluaran kode yang memakai `dplyr` dapat berubah, misalnya format tabel atau pesan. Perubahan ini diinginkan karena modul menampilkan apa yang akan dilihat mahasiswa dengan versi paket terbaru.

Bab 2

1. Ketiadaan pola pemenggalan tidak menimbulkan galat; XeLaTeX memakai kaidah bahasa Inggris. Anda mengetahuinya dari `cek_kesiapan()`, yang menandai baris pola pemenggalan dengan `[BELUM]`.
2. Font kemungkinan dipasang hanya untuk akun pengguna. Hapus font itu, lalu pasang ulang dengan **Install for all users**.
3. RStudio (dianjurkan, tetapi tidak wajib) dan Quarto (hanya untuk backend Quarto).
4. `install.packages(c("tinytex", "rmarkdown", "bookdown", "knitr", "pdftools", "qpdf", "remotes"))`, `tinytex::install_tinytex()`, `tinytex::tlmgr_install("hyphen-indonesian")`, pemasangan font Carlito, `remotes::install_github("madsyair/modulstis")`, lalu `modulstis::cek_kesiapan()`. Perintah terakhir memastikan semuanya sudah benar.

Bab 3

1. `spec/` untuk spesifikasi, `modul/` untuk hasil render, `buku/` untuk bagian awal dan akhir buku gabungan.
2. `validate()` hanya beberapa detik, sedangkan render memakan waktu lebih lama; kesalahan isian lebih murah ditemukan sebelum render.
3. Berkas itu ditulis ulang, sehingga suntingannya hilang.
4. Folder `modul/Modul02_dasar/` dengan berkas `Modul_02_dasar.pdf`.

Bab 4

1. `>` menggabungkan baris menjadi satu paragraf; `|` mempertahankan pergantian baris. Kode R ditulis dengan `|`; bila tertukar, baris-baris kode digabung menjadi satu dan R gagal menjalankannya.
2. `#` mengawali komentar, sehingga `#revisi` dan teks sesudahnya dibuang.

3. Keduanya terbaca sebagai `TRUE` dan `FALSE`. Tulis `["on", "off"]`.
4. Periksa baris sebelumnya: kesalahan seperti tidak adanya spasi sesudah titik dua atau tanda kutip yang tidak ditutup baru disadari YAML pada baris berikutnya atau di akhir berkas.

Bab 5

1. `number` dan `title`.
2. `topic`; bila dihilangkan, kepala halaman memakai `title`.
3. Tanpa blok `course`, paket memakai identitas bawaannya, yaitu mata kuliah Analisis Peubah Ganda.
4. `prodi` di blok `course`, dan biasanya juga `title`, `sub_cpmk`, atau isinya. Bila keduanya bernomor 7 dan dirender ke folder yang sama, keduanya ditulis ke `build/modul-07/` dan data salah satunya tertimpa. Proyek terpisah menghindari tabrakan ini.

Bab 6

1. `instruksi` yang kosong tetap tercetak dengan tulisan *belum diisi*; `tambahan` yang kosong dilewati sepenuhnya.
2. Setiap butir YAML menjadi satu paragraf, dan butir yang diawali `1.` membentuk daftar bernomor.
3. Tanda hubung pertama menandai butir YAML; tanda hubung kedua, di dalam tanda kutip, menandai butir daftar Markdown.
4. Nama bagian salah ketik di `body` (misalnya `ringkasn`), atau indentasinya membuat `ringkasan` tidak berada di bawah `body`.

Bab 7

1. Judul contoh, kode beserta keluarannya, Penjelasan Kode, dan Interpretasi Output.
2. Kodenya tidak dijalankan; di PDF, kotak kode tidak diikuti keluaran.
3. Render berhenti dengan galat, karena objek `rt` belum ada ketika Contoh 7.2 dijalankan.
4. Tulis contoh itu dengan `eval: false`. Di PDF, kodenya tampil tanpa keluaran; sebaiknya jelaskan juga di `penjelasan` bahwa kode itu hanya untuk dijalankan sendiri oleh mahasiswa.

Bab 8

1. Kunci `name` pada butir `datasets`.
2. Kolom `numeric` pada resep `tabular` tersebar merata. Untuk dua peubah yang berkorelasi, pakai resep `mvn` dengan peragam yang tidak nol di `Sigma`.
3. `path: "data/survei.csv"`, karena jalur dihitung dari letak berkas spesifikasi.
4. Angka di modul baru tidak lagi cocok dengan pekerjaan mahasiswa yang sudah mengerjakan modul lama. Sarannya: pertahankan `seed`, atau umumkan perubahan dan tandai versi modulnya.

Bab 9

1. Soal di `body` → `latihan` dan `penugasan`; solusi di `exercises`. Pemisahan ini membuat modul tidak memuat jawaban, sementara keduanya tetap berasal dari berkas yang sama.
2. Supaya kode yang belum lengkap tidak dijalankan; tanpa itu render gagal.
3. Kode contoh dijalankan hanya bila `eval: true`; kode solusi dijalankan kecuali `eval: false`.
4. Sisipkan solusi baru pada urutan yang sesuai dan perbarui `id` solusi sesudahnya. Paket tidak melakukannya karena tidak ada hubungan otomatis antara soal di `body` dan solusi di `exercises`.

Bab 10

1. Kotak berwarna dan susunan rumus khusus, misalnya beberapa rumus sejajar dalam satu kotak.
2. Spesifikasi bawaan menulis kotak di dalam tanda kutip ganda, tempat YAML memakai `\` sebagai awalan karakter khusus. Buku ini memakai penanda `|`, yang menyalin teks apa adanya.
3. Contoh:

   ```
   - |
     \begin{kotakperhatian}[Tentang data]
     Sebanyak 25\% data adalah data hilang.
     \end{kotakperhatian}
   ```

   Tanda `%` harus diloloskan.
4. Cari tanda `&` yang tidak diloloskan, kemungkinan besar di dalam sebuah kotak, misalnya pada nama seperti *Statistik & Data*.

Bab 11

1. `render_module()` selama menyunting satu modul; `ModuleProject` untuk memeriksa dan merender seluruh folder.
2. Merender spesifikasi kedua dan ketiga menurut urutan nama berkas di `spec/`.
3. Baris `OK` dan `GAGAL` di akhir log, karena baris itu merangkum hasil setiap modul.
4. Dari baris `GAGAL modul=... pesan=...` di log `render_all()`, atau dengan memeriksa unsur `error` pada objek hasil `render_all()` untuk setiap modul.

Bab 12

1. Tahap pertama mengukur tebal setiap modul; tahap kedua merender ulang dengan nomor halaman awal yang dihitung dari tebal modul-modul sebelumnya.
2. Setiap modul tetap bernomor mulai dari 1, dan nomor di daftar isi modul tidak cocok dengan letaknya di buku.
3. Halaman 30.

4. Bila tebalnya tetap, cukup render ulang Modul 5 dengan `halaman_awal` yang sama, tetapi cara paling aman tetap menyusun ulang buku. Bila tebalnya bertambah satu halaman, semua modul sesudah Modul 5 harus bergeser satu halaman; tanpa penyusunan ulang, nomor halaman akan bertabrakan.

Bab 13

1. Di objek tema, supaya satu perubahan berlaku untuk semua modul tanpa menyunting setiap spesifikasi.
2. Hanya `oranye` yang berubah; lima warna lain tetap memakai warna bawaan.
3. Kunci `sections` menggantikan susunan bawaan sepenuhnya; bagian Tambahan tidak disebut di sana.
4. Mereka perlu membuat tema dengan palet warna kampusnya dan memakainya saat merender. Spesifikasi modul tidak perlu diubah sama sekali.

Bab 14

1. Baca baris `WARN` dan `ERROR`; baca ulang dan periksa spesifikasi; render satu modul; buka berkas antara. Urutannya dari yang paling murah ke yang paling mahal.
2. Di berkas `.Rmd` antara di folder hasil. Buka baris itu dan lihat judul contoh di atasnya.
3. Galat LaTeX biasanya tidak menghentikan render; PDF tetap terbentuk dan tandanya hanya baris `WARN`.
4. Misalnya, cari tulisan *belum diisi*; periksa bahwa setiap kotak kode berkeluaran dan keluarannya masuk akal; periksa sampul dan blok `course`. Ketiganya mendeteksi kesalahan yang tidak menimbulkan pesan galat.

Bab 15

1. Sunting spesifikasinya; baca ulang dan periksa; render modul itu saja dan periksa PDF-nya; susun ulang buku bila perlu.
2. Isinya selalu dapat dibuat ulang dari spesifikasi.
3. Kode yang keliru dapat berjalan tanpa galat dan menghasilkan keluaran yang tidak bermakna, misalnya `NULL`; log yang bersih tidak menjamin isinya benar.
4. Tampilan atau perilaku modul dapat berbeda antarbagian buku. Sepakati satu versi, catat di `README.md`, dan minta semua anggota memasang versi itu.